

**BỘ GIÁO DỤC ĐÀO TẠO
TRƯỜNG ĐẠI HỌC SƯ PHẠM VÀ KỸ THUẬT TP. HCM
KHOA CHẤT LƯỢNG CAO**



HCMUTE

BÁO CÁO CUỐI KỲ
Môn học: Đồ án Công nghệ thông tin

Đề tài:

**TÌM HIỂU VÀ CÀI ĐẶT CÁC BÀI
MAPREDUCE
TRÊN APACHE SPARK**

Mã môn học: PROJ215879

Giảng viên hướng dẫn: ThS. LÊ THỊ MINH CHÂU

Sinh viên thực hiện: Cao Thị Ngọc Phụng 21110276

Nguyễn Phú Thành 21110299

Trần Văn Tiến 20110319

Đinh Thị Thúy Quỳnh 21110284

Tp. Hồ Chí Minh, tháng 5 năm 2025

DANH SÁCH SINH VIÊN THAM GIA

Mã học phần: BDES333877_23_2_03CLC

Nhóm: 02

Tên đề tài: Tìm hiểu và cài đặt các bài MapReduce trên Apache Spark

STT	HỌ VÀ TÊN THÀNH VIÊN	MÃ SỐ SINH VIÊN	TỶ LỆ THAM GIA
1	Nguyễn Phú Thành	21110299	100%
2	Cao Thị Ngọc Phụng	21110276	100%
3	Trần Văn Tiến	21110319	100%
4	Đinh Thị Thúy Quỳnh	21110284	100%

Ghi chú: Tỷ lệ %: Mức độ phân trăm hoàn thành của từng sinh viên tham gia.

Trưởng nhóm: Nguyễn Phú Thành

Điểm số:

Nhận xét của giáo viên

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Tp. Hồ Chí Minh - Tháng 5 năm 2024

BẢNG KẾ HOẠCH LÀM VIỆC VÀ PHÂN CÔNG NHIỆM VỤ

STT	Nội dung công việc	Thời gian	Thành viên thực hiện	
			Họ tên	MSSV
1	- Lý thuyết về 1.1+ 1.2 +1.3 + 1.4 + 2.1 + 2.2 + 3.4 +3.5 , phần mở đầu, kết luận và tổng hợp Word. - Cài đặt Apache Spark và thiết lập cụm Spark Cluster Master-Slave - Cài đặt và demo chương trình FriendCount, RDD Actions, Spark SQL	26/4-7/5	Nguyễn Phú Thành	21110299
2	- Tổng hợp 3.2 + 3.3 + 3.6 - Cài đặt Apache Spark và thiết lập cụm Spark Cluster Master-Slave - Cài đặt và demo Comprehension, Spark Streaming	26/4-7/5	Cao Thị Ngọc Phụng	21110276
3	- Lý thuyết về 2.3, tổng hợp 3.1 + 3.3 - Cài đặt Apache Spark và thiết lập cụm Spark Cluster Master-Slave - Cài đặt và Demo chương trình WordCount.	26/4-7/5	Trần Văn Tiến	21110319
4	- Lý thuyết 1.5. - Cài đặt Apache Spark và thiết lập cụm Spark Cluster Master-Slave - Cài đặt và Demo chương trình WordLength	26/4-7/5	Đinh Thị Thúy Quỳnh	21110284

LỜI CẢM ƠN

Lời đầu tiên, nhóm em xin gửi lời cảm ơn chân thành nhất đến *Cô Lê Thị Minh Châu* – giảng viên bộ môn *Big Data Essentials* của chúng em. Trong quá trình học tập và tìm hiểu bộ môn, chúng em đã nhận được sự quan tâm giúp đỡ, hướng dẫn rất tận tình và tâm huyết từ Cô. Cô đã giúp chúng em tích lũy thêm nhiều kiến thức để có cái nhìn sâu sắc và hoàn thiện hơn trong lĩnh vực Công nghệ thông tin. Để từ đó, ứng dụng những kiến thức mà Cô truyền tải, nhóm em xin trình bày lại những gì mà mình đã học hỏi được thông qua việc thực hiện đề tài “*Tìm hiểu và cài đặt các bài MapReduce trên Apache Spark*”.

Kiến thức là vô hạn và sự tiếp nhận kiến thức của bản thân mỗi người luôn tồn tại những hạn chế nhất định. Do đó, trong phạm vi khả năng của bản thân, nhóm em đã rất cố gắng để hoàn thành đề tài một cách tốt nhất. Tuy nhiên, chắc chắn không tránh khỏi những thiếu sót, nhóm chúng em rất mong nhận được sự cảm thông và những ý kiến đóng góp đến từ Cô để đề tài của nhóm em được hoàn thiện hơn.

Một lần nữa, *nhóm em xin chân thành cảm ơn Cô* đã tận tình hướng dẫn, chỉ bảo các thành viên nhóm em trong suốt quá trình học tập và thực hiện đồ án này.

Kính chúc Cô sức khỏe, hạnh phúc thành công trên con đường sự nghiệp giảng dạy.

Trân trọng
Đại diện nhóm
Nguyễn Phú Thành

MỤC LỤC

PHẦN MỞ ĐẦU	1
1. LỜI MỞ ĐẦU	1
2. MỤC ĐÍCH, NHIỆM VỤ CỦA ĐỀ TÀI	1
3. PHẠM VI THỰC HIỆN ĐỀ TÀI	1
CHƯƠNG 1: GIỚI THIỆU APACHE SPARK.....	2
1.1 LỊCH SỬ HÌNH THÀNH CỦA APACHE SPARK	2
1.2 CÁC THÀNH PHẦN CỦA SPARK.....	3
1.2.1 <i>Spark Core</i>	3
1.2.2 <i>SparkSQL</i>	4
1.2.3 <i>Spark Streaming</i>	4
1.2.4 <i>MLlib (Machine Learning Library)</i>	5
1.2.5 <i>GraphX</i>	6
1.3 NỀN TẢNG XỬ LÝ DỮ LIỆU ĐƯỢC ƯA CHUỘNG BỞI CÁC CÔNG TY CÔNG NGHỆ HÀNG ĐẦU	6
1.4 TÍNH NĂNG CỦA APACHE SPARK	8
1.4.1 <i>Ưu điểm</i>	8
1.4.2 <i>Nhược điểm</i>	9
1.5 SPARK VÀ HADOOP MAPREDUCE	10
1.5.1 <i>Hadoop MapReduce</i>	10
1.5.2 <i>Spark</i>	11
CHƯƠNG 2: KHÁM PHÁ APACHE SPARK.....	12
2.1 SPARK CORE VÀ RDD	12
2.1.1 <i>Xử lý dữ liệu phân tán</i>	12
2.1.2 <i>Spark Application</i>	13
2.1.3 <i>Spark Session</i>	14
2.1.4 <i>RDD</i>	15
2.1.4.1 <i>RDD Transformations</i>	16
2.1.4.1 <i>RDD Actions</i>	21
2.2 SPARK SQL VỚI DATAFRAME VÀ DATASET	24
2.2.1 <i>Hiệu năng</i>	24
2.2.2 <i>DataFrame</i>	26
2.2.3 <i>DataFrame và DataSet</i>	28
2.3 SPARK STREAMING.....	32
CHƯƠNG 3: CÀI ĐẶT HADOOP VÀ HADOOP CLUSTER.....	35
3.1 CÀI ĐẶT JDK	35
3.1.1 <i>Tải JDK 1.8</i>	35
3.1.2 <i>Cài đặt môi trường JDK</i>	35
3.2 CÀI ĐẶT HADOOP	36
3.3 CẤU HÌNH CÁC FILE CỦA HADOOP	38
3.3.1 <i>Cấu hình địa chỉ IP</i>	38
3.3.2 <i>Cấu hình core-site.xml (cấu hình trên cả master và slave)</i>	40
3.3.3 <i>Cấu hình mapred-site.xml (cấu hình trên cả master và slave)</i>	41

3.3.4 Cấu hình <i>hdfs-site.xml</i> (cấu hình trên cả master và slave).....	42
3.3.5 Cấu hình <i>yarn-site.xml</i> (cấu hình trên cả master và slave)	44
3.3.6 Cấu hình <i>hadoop-env.cmd</i> (cấu hình trên cả master và slave)	45
3.3.7 Cập nhật file <i>BIN</i>	47
3.3.8 Format hệ thống.....	48
3.3.9 Chạy thử.....	49
CHƯƠNG 4: THIẾT KẾ VÀ CÀI ĐẶT.....	51
4.1 CÀI ĐẶT APACHE SPARK	51
4.1.1 Cài đặt <i>JDK</i>	51
4.1.2 Cài đặt <i>Scala</i>	53
4.1.3 Cài đặt <i>Python</i>	56
4.1.4 Cài đặt <i>Spark</i>	58
4.1.5 Chạy thử <i>Spark</i>	61
4.1.6 Chạy thử <i>Pyspark</i>	63
4.2 CÀI ĐẶT SPARK CLUSTER.....	65
4.3 DEMO CÁC BÀI MAP REDUCE	70
4.3.1 <i>WordCount</i>	70
4.3.2 <i>WordLength</i>	74
4.3.3 <i>FriendCount</i>	77
4.3.4 <i>Comprehension</i>	80
4.4 DEMO RDD ACTIONS.....	82
4.5 DEMO SPARKSQL	90
4.6 DEMO SPARK STREAMING	97
PHẦN KẾT LUẬN.....	99
1. ƯU ĐIỂM.....	99
2. NHƯỢC ĐIỂM	99
3. HƯỚNG PHÁT TRIỂN.....	99
TÀI LIỆU THAM KHẢO	100

PHẦN MỞ ĐẦU

1. Lời mở đầu

Với sự phát triển mạnh mẽ của công nghệ thông tin, nhu cầu xử lý dữ liệu lớn ngày càng tăng cao trong nhiều lĩnh vực. Apache Spark là một nền tảng mã nguồn mở phổ biến cho xử lý dữ liệu lớn, cung cấp hiệu suất cao và khả năng mở rộng tốt. MapReduce là một mô hình lập trình được sử dụng phổ biến trong Spark để xử lý dữ liệu song song. Đề tài này nhằm giới thiệu về mô hình lập trình MapReduce và hướng dẫn cài đặt, cấu hình, thực hành các bài MapReduce cơ bản trên Apache Spark. Qua đó, giúp học viên hiểu rõ về nguyên tắc hoạt động, cách thức áp dụng MapReduce trong Spark, đồng thời có khả năng tự cài đặt và sử dụng Spark để giải quyết các bài toán thực tế.

2. Mục đích, nhiệm vụ của đề tài

Hiểu rõ về mô hình lập trình MapReduce và cách thức hoạt động của nó trong Apache Spark. Cài đặt và cấu hình môi trường Spark để thực hành các bài MapReduce. Viết các chương trình Spark bằng Scala hoặc Python để thực hiện các bài toán MapReduce cơ bản như đếm từ, tính tổng, tìm giá trị trung bình,...Đánh giá hiệu quả của các bài MapReduce trên Spark.

3. Phạm vi thực hiện đề tài

Đề tài này thực hiện chủ yếu các lý thuyết cơ bản về Apache Spark, cài đặt Apache Spark và thiết lập cụm Spark Cluster Master Slave và cài đặt các chương trình của Apache Spark như SparkSQL, Spark Streaming, các bài toán MapReduce trong Spark.

CHƯƠNG 1: GIỚI THIỆU APACHE SPARK

1.1 Lịch sử hình thành của Apache Spark

Apache Spark là một hệ thống xử lý dữ liệu lớn mã nguồn mở, được biết đến với hiệu suất cao, khả năng mở rộng và dễ sử dụng. Hành trình phát triển của Spark bắt đầu từ một dự án nghiên cứu tại Đại học California, Berkeley vào năm 2009, và trải qua nhiều giai đoạn quan trọng để trở thành công cụ phổ biến như hiện nay. Năm 2010, một nhóm 5 nhà nghiên cứu thuộc AMPLab - UC Berkeley đã cho ra đời một bài báo mang tính đột phá mang tên [“Spark: Cluster Computing with Working Sets”](#). Lúc bấy giờ, Hadoop MapReduce đang thống trị lĩnh vực lập trình song song, trở thành công cụ open-source đầu tiên để xử lý dữ liệu quy mô lớn trên các cụm máy chủ với hàng ngàn node. Nhóm AMPLab, nhận thức được tiềm năng to lớn của Cluster Computing, đã bắt tay vào nghiên cứu để phát triển một nền tảng lập trình mạnh mẽ và linh hoạt hơn MapReduce.

Trong suốt quá trình khảo sát và phân tích, họ đã xác định được hai điểm chính

1. *Tiềm năng to lớn của Cluster Computing*: Mô hình này ẩn chứa tiềm năng phát triển to lớn, cho phép xây dựng và vận hành các ứng dụng mới hoàn toàn dựa trên hệ thống và dữ liệu hiện có.
2. *Hạn chế của MapReduce*: Việc sử dụng MapReduce để xây dựng các ứng dụng lớn trở nên khó khăn và kém hiệu quả.

Nhóm AMPLab quyết định phát triển Spark, đặt mục tiêu cốt lõi là tạo ra một nền tảng lập trình dễ dàng sử dụng, có khả năng mở rộng linh hoạt để xử lý các tập dữ liệu khổng lồ. Spark được thiết kế dựa trên API lập trình hàm (Functional Programming), cho phép viết ứng dụng một cách ngắn gọn và hiệu quả. Nền tảng này còn tích hợp công cụ chia sẻ dữ liệu trong bộ nhớ hiệu quả, cho phép tái sử dụng dữ liệu qua các bước tính toán.

Spark cung cấp một hệ thống xử lý dữ liệu song song mạnh mẽ, có thể hoạt động trên hệ thống hàng ngàn máy chủ. Nền tảng này hỗ trợ đa ngôn ngữ lập trình phổ biến như Python, Java, Scala và R, cùng với các thư viện dành cho nhiều tác vụ khác nhau, từ SQL đến Streaming, Machine Learning và tính toán đồ thị song song. Năm 2013, dự án Spark được trao tặng cho Apache Software Foundation, trở thành dự án open-source thu hút sự quan tâm và đóng góp mạnh mẽ từ cộng đồng lập trình viên. Cuối cùng, vào ngày 26/5/2014, Apache chính thức phát hành Spark phiên bản 1.0, sẵn sàng cho ứng dụng trong môi trường sản xuất.

1.2 Các thành phần của Spark



1.2.1 Spark Core

Là nền tảng cho tất cả các chức năng khác của Spark. Nó cung cấp khả năng tính toán phân tán hiệu quả trên bộ nhớ RAM và cả bộ dữ liệu lưu trữ trên các hệ thống lưu trữ ngoài (như HDFS). Tất cả các chức năng được cung cấp bởi Apache Spark đều được xây dựng trên nền tảng của Spark Core. Spark Core cung cấp khả năng tính toán trong bộ nhớ (in-memory computation) để mang lại tốc độ xử lý nhanh. Nó là nền tảng của việc xử lý song song và phân tán các tập dữ liệu khổng lồ. Spark Core đóng vai trò trụ cột cho các chức năng I/O cần thiết và đóng vai trò quan trọng trong việc lập trình và giám sát cụm Spark. Spark Core bao gồm các thành phần liên quan đến việc lập lịch, phân phối và giám sát các tác vụ trên cụm, phân (phân vaii nhiệm vụ - task dispatching), và phục hồi lỗi.

Các chức năng của Spark Core bao gồm:

- Hỗ trợ nhiều mô hình lập lịch: local (chạy cục bộ trên một máy), cluster (chạy trên cụm máy tính) và YARN (dành cho các ứng dụng quy mô lớn).
- Là nơi lưu trữ API để định nghĩa RDD (Các tập dữ liệu đàn hồi phân tán).
- Bộ dữ liệu phân tán có khả năng phục hồi), Spark Context (ngữ cảnh Spark) và các

primitive operations (phép toán nguyên thủy) cho RDD.

1.2.2 SparkSQL

Spark SQL được xây dựng trên nền tảng Spark Core, cung cấp một giao diện SQL để truy vấn và xử lý dữ liệu có cấu trúc và bán cấu trúc. Nó đóng vai trò như một lớp trung gian giúp truy cập và thao tác với nhiều nguồn dữ liệu khác nhau một cách dễ dàng. Spark SQL hỗ trợ đọc dữ liệu từ các định dạng phổ biến như Hive table, Parquet, JSON, và cho phép truy vấn dữ liệu bằng cả ngôn ngữ SQL và HiveQL.

Điểm mạnh của Spark SQL là cung cấp các tính năng phân tích mạnh mẽ, tương tác, hoạt động trên cả dữ liệu thời gian thực (streaming) và dữ liệu lịch sử. Spark SQL có thể được coi như một thành phần mở rộng của Spark, tích hợp xử lý dữ liệu theo quan hệ với các API lập trình của Spark.

Các chức năng chính của Spark SQL:

- Xử lý dữ liệu có cấu trúc trong Spark. Giới thiệu khái niệm SchemaRDD: cung cấp một biểu diễn trừu tượng cho dữ liệu có cấu trúc, bao gồm cả schema (sơ đồ dữ liệu).
- Đọc dữ liệu từ nhiều nguồn khác, tích hợp với các hệ thống lưu trữ dữ liệu khác nhau: Hive, HBase, Cassandra, CSV, JSON, Parquet, ORC và hơn thế nữa.
- Cho phép lập trình viên kết hợp các truy vấn SQL với thao tác dữ liệu theo chương trình được hỗ trợ bởi RDD trong Python, Scala và Java.
- Cho phép thực thi các truy vấn SQL phức tạp: JOIN, aggregation, filtering, windowing và subqueries.

1.2.3 Spark Streaming

Spark Streaming cung cấp khả năng xử lý luồng dữ liệu trực tiếp với tính mở rộng, thông lượng cao và chịu lỗi. Spark có thể truy cập dữ liệu từ các nguồn như Flume, socket TCP. Nó có thể xử lý các thuật toán khác nhau trên dữ liệu được nhận và kết quả có thể được lưu trữ ở nhiều định dạng như hệ thống tập tin, cơ sở dữ liệu hoặc hiển thị trên bảng điều khiển trực tiếp.

Spark Streaming sử dụng kỹ thuật Micro-batching để xử lý luồng dữ liệu thời gian thực. Micro-batching phân chia luồng dữ liệu thành các nhóm nhỏ (micro-batch) để xử lý hiệu quả hơn. Mỗi micro-batch được xử lý giống như một batch dữ liệu thông thường

trong Spark Core, giúp tận dụng tối đa sức mạnh tính toán của Spark.

Chức năng chính của Spark Streaming:

- Cho phép xử lý luồng dữ liệu trực tiếp, ví dụ như các file log được tạo bởi các dịch vụ web.
- Cung cấp API xử lý luồng dữ liệu tương tự như API RDD của Spark Core, giúp dễ dàng học hỏi và sử dụng.
- Cung cấp tính năng xử lý dữ liệu theo thời gian thực (real-time) trên Spark.
- Lấy dữ liệu theo các mini-batch nhỏ và áp dụng các phép biến đổi RDD trên mỗi mini-batch.
- Hỗ trợ nhiều nguồn dữ liệu thời gian thực: Kafka, Flume, Twitter và hơn thế nữa.
- Cho phép thực hiện các phép tính phân tích phức tạp: cập nhật trạng thái, phân tích dòng chảy, phát hiện bất thường và hơn thế nữa

1.2.4 MLlib (Machine Learning Library)

Thư viện Học máy (Machine Learning) mở rộng trên nền Spark – MLlib, một thư viện học máy mở rộng tích hợp trên Spark, cung cấp nhiều thuật toán học máy khác nhau. Mục tiêu chính của việc xây dựng MLlib là đơn giản hóa việc triển khai các thuật toán học máy. MLlib bao gồm các công cụ cần thiết cho học máy cùng với việc triển khai nhiều thuật toán phổ biến. Ví dụ, các thuật toán thường dùng trong MLlib bao gồm: cụm (clustering), dự báo (regression), phân loại (classification) và đề xuất (collaborative filtering).

Những điểm chính cần lưu ý:

- MLlib là một thư viện mở rộng, tận dụng khả năng tính toán phân tán của Spark để tăng tốc độ đào tạo và dự đoán mô hình.
- MLlib cung cấp nhiều thuật toán học máy phổ biến, giúp người dùng dễ dàng lựa chọn và triển khai thuật toán phù hợp cho bài toán của mình.
- Mục tiêu của MLlib là đơn giản hóa việc sử dụng học máy, giúp người dùng có thể tập trung vào việc xây dựng các mô hình hiệu quả.
- Có thể tích hợp với các thư viện học máy phổ biến khác: TensorFlow, scikit-learn và hơn thế nữa.

1.2.5 GraphX

Là một framework xử lý đồ thị phân tán được xây dựng trên Spark Core. Cung cấp một API để thực hiện các phép tính đồ thị

- Phân tích mạng lưới: Lưu trữ, truy vấn và phân tích dữ liệu dạng đồ thị, bao gồm các đỉnh (vertex) và cạnh (edge) kết nối chúng.
- Hỗ trợ nhiều loại đồ thị: directed graphs (đồ thị có hướng), undirected graphs (đồ thị vô hướng), weighted graphs (đồ thị có trọng số) và hơn thế nữa. Cho phép thực hiện các thuật toán đồ thị phức tạp: PageRank, community detection (phát hiện cộng đồng) và social network analysis (phân tích mạng xã hội)
- Tối ưu hóa hiệu suất: Tối ưu cách lưu trữ và biểu diễn đồ thị, hỗ trợ xử lý dữ liệu đồ thị lớn hiệu quả.
- Hỗ trợ các phép toán: Cung cấp các phép toán cơ bản để thao tác với dữ liệu đồ thị như lấy tập con, nối đỉnh, tổng hợp tin nhắn.
- API Pregel: Cung cấp phiên bản tối ưu hóa của API Pregel để xử lý song song các thuật toán trên đồ thị.

1.3 Nền tảng xử lý dữ liệu được ưa chuộng bởi các công ty công nghệ hàng đầu

Apache Spark đang ngày càng khẳng định vị thế là nền tảng xử lý dữ liệu hàng đầu, được tin dùng bởi nhiều công ty công nghệ lớn trên thế giới. Nhờ khả năng xử lý dữ liệu lớn hiệu quả, linh hoạt và dễ sử dụng, Spark mang đến giải pháp tối ưu cho các bài toán phức tạp trong nhiều lĩnh vực.

Alibaba: Alibaba một trong những nền tảng thương mại điện tử lớn nhất thế giới, sử dụng Apache Spark để phân tích dữ liệu khổng lồ với hàng trăm petabyte dữ liệu trên nền tảng thương mại điện tử của mình, cung cấp thông tin chi tiết về hành vi khách hàng, xu hướng thị trường và hiệu quả hoạt động. Spark giúp Alibaba trích xuất tính năng từ dữ liệu hình ảnh một cách nhanh chóng và chính xác, hỗ trợ cho các ứng dụng như nhận dạng sản phẩm, phân loại ảnh và đề xuất sản phẩm phù hợp. Ngoài ra, Spark được sử dụng để xử lý các đồ thị phức tạp mô tả mối quan hệ giữa người dùng, sản phẩm và nhà bán hàng, giúp Alibaba đưa ra các quyết định kinh doanh sáng suốt hơn.

Ebay: eBay sử dụng Apache Spark để cung cấp các ưu đãi được nhắm mục tiêu đến từng khách hàng, giúp họ dễ dàng tìm kiếm sản phẩm phù hợp và gia tăng tỷ lệ chuyển

đổi. Xử lý lượng lớn dữ liệu một cách nhanh chóng và hiệu quả, góp phần nâng cao hiệu suất hoạt động của toàn hệ thống. Apache Spark được tận dụng tại eBay thông qua Hadoop YARN.

Uber: Spark giúp Uber dự đoán nhu cầu đi xe một cách chính xác, hỗ trợ tối ưu hóa việc phân bổ tài xế và đáp ứng nhu cầu của khách hàng hiệu quả hơn. Spark được sử dụng để phân tích dữ liệu di chuyển của người dùng, giúp Uber hiểu rõ hơn về hành vi di chuyển và đưa ra các giải pháp cải thiện dịch vụ.

Netflix: Sử dụng Spark để phân tích sở thích của người dùng và đề xuất nội dung phù hợp với từng cá nhân, nâng cao trải nghiệm người dùng và gia tăng tỷ lệ giữ chân khách hàng. Phân tích hiệu suất của các nội dung trên Netflix, giúp công ty đánh giá hiệu quả các chiến dịch marketing và điều chỉnh nội dung phù hợp với thị hiếu người xem.

Ngoài ra, Spark còn được ứng dụng rộng rãi trong nhiều lĩnh vực khác như:

- Phân tích khoa học: Spark được sử dụng bởi các tổ chức như NASA, CERN và Broad Institute of MIT và Harvard để phân tích dữ liệu khoa học khổng lồ, hỗ trợ nghiên cứu khoa học và phát triển công nghệ.
- Xử lý ngôn ngữ tự nhiên: Spark được sử dụng để xử lý và phân tích dữ liệu ngôn ngữ tự nhiên, hỗ trợ các ứng dụng như dịch máy, tóm tắt văn bản và chatbot.
- Internet vạn vật (IoT): Spark được sử dụng để xử lý và phân tích dữ liệu từ các thiết bị IoT, giúp theo dõi và quản lý các thiết bị hiệu quả hơn.

Với những ưu điểm vượt trội, Apache Spark đang trở thành nền tảng xử lý dữ liệu không thể thiếu cho các doanh nghiệp và tổ chức trong thời đại dữ liệu bùng nổ. Khả năng xử lý dữ liệu lớn hiệu quả, linh hoạt và dễ sử dụng giúp Spark đáp ứng mọi nhu cầu xử lý dữ liệu phức tạp, đồng thời mang lại lợi thế cạnh tranh cho các doanh nghiệp trong thị trường ngày càng cạnh tranh.

1.4 Tính năng của Apache Spark



1.4.1 Ưu điểm

Apache Spark là một nền tảng xử lý dữ liệu mã nguồn mở, nổi tiếng với khả năng phân tích dữ liệu mạnh mẽ, tốc độ xử lý nhanh chóng và hỗ trợ đa ngôn ngữ. Spark được sử dụng rộng rãi trong nhiều lĩnh vực như khoa học dữ liệu, phân tích dữ liệu lớn, học máy, xử lý ngôn ngữ tự nhiên, Internet vạn vật (IoT), ... Một số điểm nổi bật của Apache Spark như:

Advanced Analytics: Spark không chỉ giới hạn trong các phép toán "Map" và "Reduce" truyền thống, mà còn cung cấp các tính năng phân tích dữ liệu tiên tiến như hỗ trợ Spark truy vấn SQL, Streaming data, Machine learning (ML) và các thuật toán xử lý đồ thị đóng vai trò như một bộ công cụ phân tích dữ liệu cực kì mạnh mẽ.

Speed: Spark giúp chạy một ứng dụng với tốc độ xử lý vượt trội. So với Hadoop cluster, Spark có thể chạy ứng dụng nhanh hơn tới 100 lần khi xử lý dữ liệu trong bộ nhớ và 10 lần khi xử lý dữ liệu trên đĩa.. Điều này do Spark tối ưu hóa việc sử dụng bộ nhớ, hạn chế tối đa việc đọc/ghi dữ liệu vào ổ đĩa, giúp tăng tốc độ xử lý và giảm thiểu thời gian chờ.

Supports multiple languages: Spark cung cấp API tích hợp sẵn cho các ngôn ngữ

lập trình phổ biến như Java, Scala, Python và R. Điều này cho phép người dùng lựa chọn ngôn ngữ phù hợp với sở thích và kỹ năng, có thể code Spark applications với nhiều lựa chọn về ngôn ngữ lập trình. Bên cạnh đó Spark còn cung cấp nhiều toán tử cấp cao (high-level operators) cho việc truy vấn dữ liệu, giúp đơn giản hóa việc viết mã và tăng năng suất lập trình.

1.4.2 Nhược điểm

Mặc dù Apache Spark là một nền tảng xử lý dữ liệu mạnh mẽ và linh hoạt, nhưng nó cũng có một số hạn chế nhất định cần được cân nhắc trước khi sử dụng:

Phụ thuộc vào hệ thống tệp: Spark không có hệ thống tệp riêng mà phụ thuộc vào các hệ thống tệp khác như HDFS (Hadoop Distributed File System) hoặc các hệ thống lưu trữ đám mây như S3, Google Cloud Storage,... Điều này có thể dẫn đến một số hạn chế về khả năng tương thích và hiệu suất. Spark yêu cầu hệ thống tệp phải hỗ trợ truy cập dữ liệu song song và hiệu quả để đảm bảo hiệu suất tối ưu.

Chi phí cao: Apache Spark được thiết kế để xử lý dữ liệu trong bộ nhớ, do đó yêu cầu lượng RAM lớn để hoạt động hiệu quả. Điều này có thể dẫn đến chi phí phần cứng cao hơn, đặc biệt khi xử lý các tập dữ liệu lớn.

Khả năng xử lý luồng dữ liệu: Spark Streaming được thiết kế để xử lý luồng dữ liệu với độ trễ thấp, nhưng nó không hoàn toàn là thời gian thực. Luồng dữ liệu được chia thành các micro-batch nhỏ để xử lý, dẫn đến một số độ trễ nhất định. Để đạt được hiệu suất xử lý luồng dữ liệu cao, Spark Streaming cần được cấu hình và tối ưu hóa cẩn thận, đòi hỏi chuyên môn kỹ thuật cao.

Tối ưu hóa thủ công: Việc tối ưu hóa, tinh chỉnh để phù hợp với các bộ dữ liệu cụ thể cần có kinh nghiệm và kiến thức để điều chỉnh hiệu suất Spark phù hợp với từng trường hợp sử dụng.

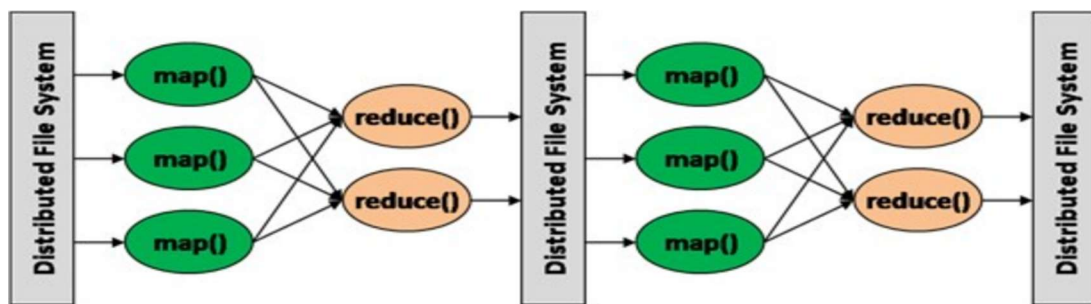
1.5 Spark và Hadoop Mapreduce

MapReduce và Spark là hai hệ thống xử lý dữ liệu lớn phổ biến, tuy nhiên có những điểm khác biệt cơ bản trong cách thức hoạt động dẫn đến ưu và nhược điểm riêng biệt. Bài viết này sẽ so sánh chi tiết cơ chế hoạt động của hai hệ thống này để giúp bạn hiểu rõ hơn về điểm mạnh và điểm yếu của từng hệ thống.

1.5.1 Hadoop MapReduce

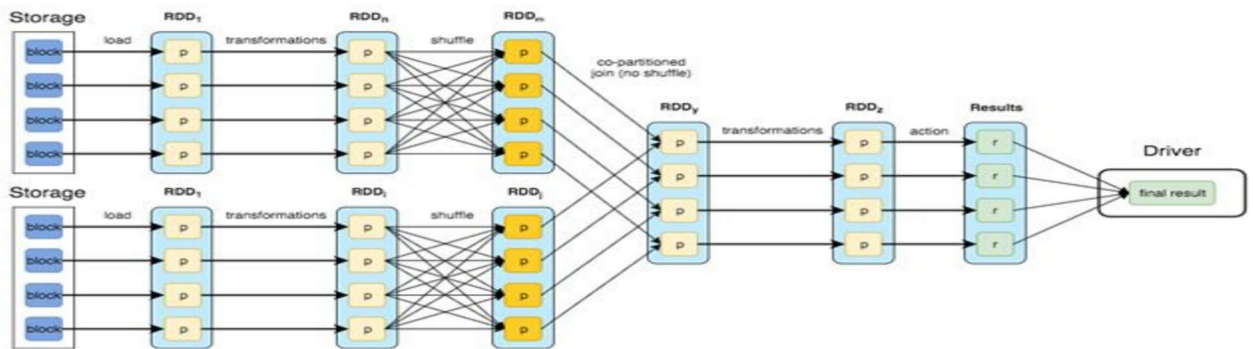
Về cơ chế hoạt động của MapReduce: input data được đọc từ HDFS (component phụ MapReduce chia nhỏ dữ liệu đầu vào thành nhiều khối độc lập, sau đó phân phối các khối này cho nhiều máy tính để xử lý song song. Sau khi xử lý, kết quả được ghi lại vào HDFS (Hệ thống tệp phân tán Hadoop). Quy trình này được lặp lại cho đến khi hoàn thành toàn bộ công việc.

- Ưu điểm: Khả năng mở rộng cao: Dễ dàng mở rộng hệ thống bằng cách thêm máy tính để xử lý lượng dữ liệu lớn. Chi phí thấp: Sử dụng phần cứng giá rẻ thay vì máy tính cao cấp.
- Nhược điểm: Do dữ liệu trung gian được ghi vào HDFS sau mỗi bước xử lý, dẫn đến lãng phí tài nguyên lưu trữ và tăng độ trễ. Việc viết code MapReduce có thể phức tạp và khó khăn để gỡ lỗi. Không hiệu quả cho các ứng dụng cần xử lý lặp đi lặp lại do mỗi bước xử lý đều ghi dữ liệu vào HDFS, việc xử lý lặp đi lặp lại dữ liệu có thể dẫn đến hiệu suất thấp.



1.5.2 Spark

Spark sử dụng RDD (Resilient Distributed Dataset) - một tập dữ liệu phân tán có khả năng phục hồi cao. Dữ liệu đầu vào chỉ được tải một lần duy nhất vào bộ nhớ, sau đó Spark lên kế hoạch và tối ưu hóa các bước xử lý dữ liệu liên tục cho đến khi tạo ra kết quả cuối cùng. Toàn bộ quá trình đó được diễn ra trên bộ nhớ RAM (khi hết RAM sẽ được chuyển sang xử lý trên Disk) tận dụng được hiệu suất I/O cao từ đó có thể giảm thời gian thực thi nhỏ hơn 10-100 lần Hadoop MapReduce.



CHƯƠNG 2: KHÁM PHÁ APACHE SPARK

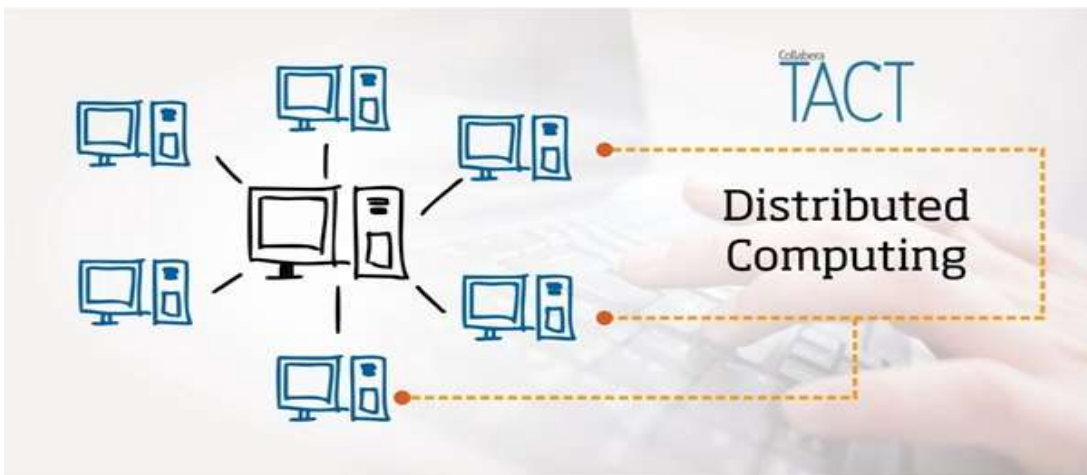
2.1 Spark Core và RDD

2.1.1 Xử lý dữ liệu phân tán

Trong kỷ nguyên dữ liệu bùng nổ, việc lưu trữ và xử lý lượng dữ liệu khổng lồ trở thành thách thức lớn cho các doanh nghiệp và tổ chức. Xử lý dữ liệu phân tán (distributed computing) ra đời như một giải pháp hiệu quả, giúp chia nhỏ dữ liệu trên nhiều máy tính, song song thực hiện các phép toán và tổng hợp kết quả, mang lại nhiều ưu điểm vượt trội so với phương pháp xử lý truyền thống

Trong Hadoop, Hdfs là thành phần giữ vai trò lưu trữ dữ liệu, và dữ liệu này là phân tán. Vậy khi muốn xử lý dữ liệu thì phải làm sao? Rõ ràng ta không thể kéo dữ liệu phân tán về xử lý và sau đó lại “phân tán” output ra để lưu trữ được đúng không. Hiểu đơn giản thì Apache Spark là 1 framework hỗ trợ xử lý dữ liệu phân tán, Spark có khả năng sinh và quản lý các task chạy song song, ổn định và chịu lỗi rất tốt.

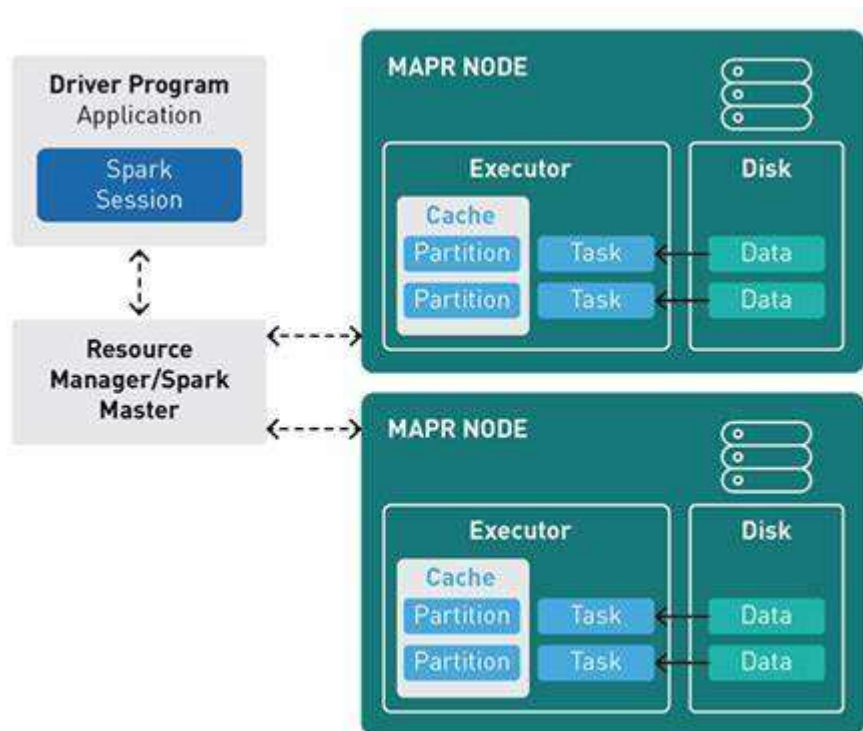
Nếu như theo truyền thống, ta sẽ “kéo” dữ liệu từ nhiều nơi về xử lý tập trung, thì với Spark, ta sẽ đưa code xử lý đến dữ liệu và xử lý ở đó luôn. Nếu hơi khó hiểu thì các bạn có thể xem hình dưới đây:



Giả sử các máy tính màu xanh chứa dữ liệu và phân tán trong 1 cụm, thông thường, khi muốn xử lý dữ liệu này, ta sẽ “kéo” dữ liệu liên quan từ các máy màu xanh về máy màu đen để xử lý. Tuy nhiên, với Spark sẽ ngược lại, ta sẽ gửi code xử lý từ máy màu đen đến các máy màu xanh, sau đó dữ liệu sẽ được xử lý trên các máy màu xanh, rồi lại lưu luôn tại máy màu xanh. Vừa không mất công chuyển đổi dữ liệu qua lại, lại giảm nhẹ gánh nặng tính toán lên 1 máy tính.

Tất nhiên, tổng quan là như vậy, có những tính toán vẫn cần phải xử lý tập trung (VD count số bản ghi mà phân tán thì tất nhiên cần phải 1 máy tính tổng từng phần từ các máy khác), Spark cung cấp các API linh hoạt, tối ưu việc gửi nhận dữ liệu giữa các máy tính.

2.1.2 Spark Application



Một ứng dụng Spark sẽ gồm 2 thành phần chính:

- **Driver Program:** Là 1 JVM (Java Virtual Machine Process), chứa hàm main() như bất kì 1 chương trình JVM nào khác, nó đóng vai trò điều phối code/ logic xử lý trên driver, tạo Spark Session, quản lý Executor và thu thập dữ liệu. Driver program chứa Spark Session
- **Executor:** Là các worker, đóng vai trò thực hiện các tác vụ tính toán được giao bởi Driver Program. Mỗi Executor được khởi chạy trên một nút trong cụm Spark và có quyền truy cập vào dữ liệu được lưu trữ cục bộ trên nút đó. Dữ liệu cần xử lý có thể được load trực tiếp vào memory của Executor..

2.1.3 Spark Session

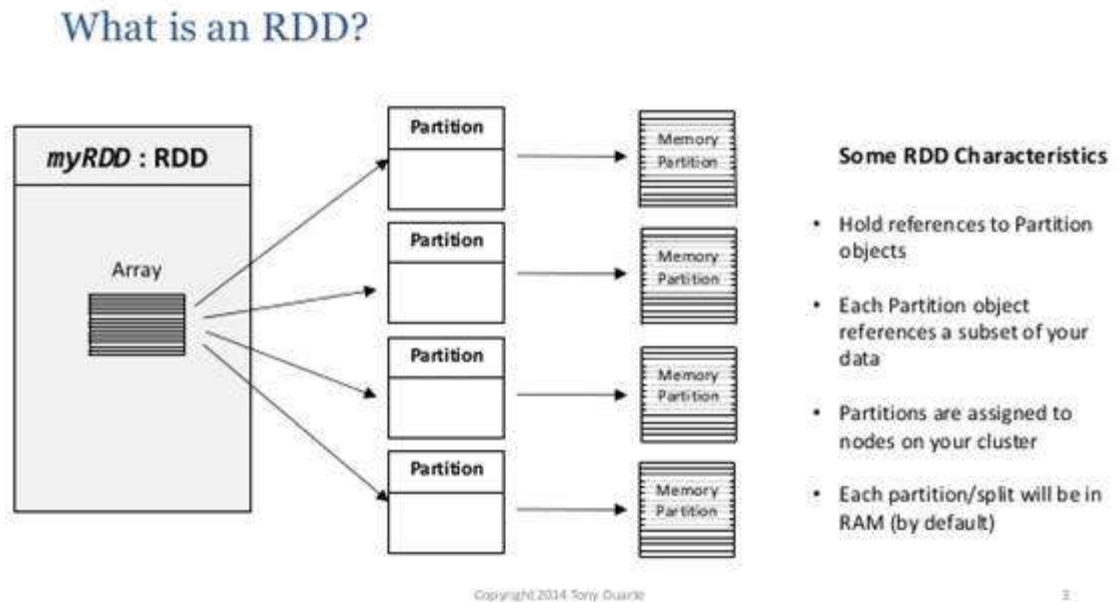
SparkSession là một điểm vào thống nhất cho các ứng dụng Spark, được giới thiệu trong Spark 2.0. Nó hoạt động như một đầu nối đến tất cả các chức năng nền tảng của Spark, bao gồm RDD, DataFrame và Dataset, cung cấp một giao diện thống nhất để xử lý dữ liệu có cấu trúc. Đây là một trong những đối tượng đầu tiên bạn tạo khi phát triển ứng dụng Spark SQL. Là một nhà phát triển Spark, bạn tạo SparkSession bằng phương thức SparkSession.builder().

Đại diện cho khả năng tương tác với executors trong 1 chương trình. Spark session chính là entry point của mọi chương trình Spark. Từ SparkSession, có thể tạo RDD/ DataFrame/ DataSet, thực thi SQL... từ đó thực thi tính toán phân tán. Khi chạy, từ logic của chương trình (chính là code xử lý thông qua việc gọi các API), Driver sẽ sinh ra các task tương ứng và lên lịch chạy các task, sau đó gửi xuống Executor để thực thi. Dữ liệu được lưu trên memory của Executor nên việc thực thi tính toán sẽ nhanh hơn rất nhiều.

SparkSession hợp nhất một số ngữ cảnh riêng biệt trước đây, chẳng hạn như SQLContext, HiveContext và StreamingContext, thành một điểm vào duy nhất, giúp đơn giản hóa việc tương tác với Spark và các API khác nhau của nó. Nó cho phép người dùng thực hiện các hoạt động khác nhau như đọc dữ liệu từ các nguồn khác nhau, thực thi các truy vấn SQL, tạo DataFrame và Dataset và thực hiện các hành động trên các tập dữ liệu phân tán một cách hiệu quả.

Đối với những người tham gia Spark thông qua giao diện dòng lệnh spark-shell, biến spark tự động cung cấp một SparkSession mặc định, loại bỏ nhu cầu tạo thủ công trong ngữ cảnh này.

2.1.4 RDD



Spark RDD (Resilient Distributed Datasets - Tập dữ liệu phân tán đàn hồi) là một cấu trúc dữ liệu nền tảng của Spark. Nó là một tập hợp các đối tượng không thể thay đổi được phân tán trên các nút của cụm. Mỗi tập dữ liệu trong RDD được chia thành các phân vùng logic, có thể được tính toán trên các nút khác nhau của cụm. Trong 1 chương trình Spark, RDD là đại diện cho tập dữ liệu phân tán.

Một ví dụ cho các bạn mới dễ hiểu nhé.

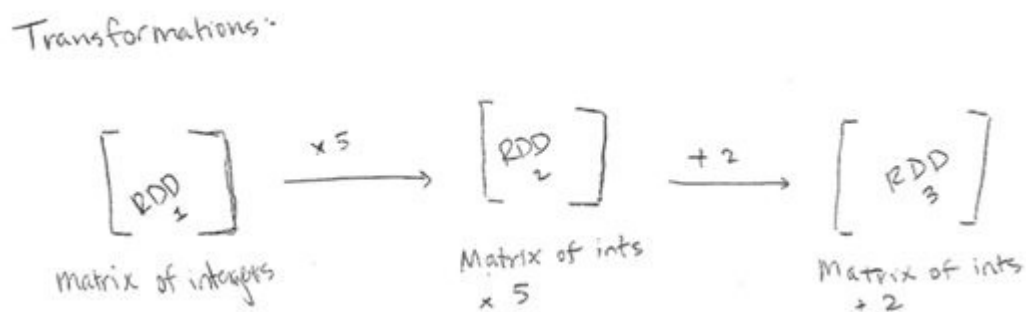
Ta có 1 object `colors = Array {red, blue, black, white, yellow}`. Trong chương trình thông thường, phần dữ liệu (red, blue, black, white, yellow) nằm trên 1 máy tính duy nhất và thông qua biến `colors`, bạn có thể truy cập đến dữ liệu. Tuy nhiên trong hệ phân tán, nếu phần dữ liệu red, blue nằm trên máy tính A còn black, white, yellow lại nằm trên máy tính B thì sao? RDD sẽ giúp bạn truy cập 2 phần dữ liệu rời rạc này như 1 đối tượng thông thường. Đó là lý do tại sao mình nói RDD là đại diện cho tập dữ liệu phân tán.

Đặc điểm quan trọng của 1 RDD là số partitions. Một RDD bao gồm nhiều partition nhỏ, mỗi partition này đại diện cho 1 phần dữ liệu phân tán. Khái niệm partition là logical, tức là 1 node xử lý có thể chứa nhiều hơn 1 RDD partition. Theo mặc định, dữ liệu các partitions sẽ lưu trên memory. Thử tưởng tượng bạn cần xử lý 1TB dữ liệu, nếu lưu hết trên mem tính ra thì cũng khá tốn kém. Tất nhiên nếu bạn có 1TB ram để xử lý thì tốt quá nhưng điều đó không cần thiết. Với việc chia nhỏ dữ liệu thành các partition và cơ chế lazy evaluation của Spark bạn có thể chỉ cần vài chục GB ram và 1 chương trình được thiết kế tốt để xử lý 1TB dữ liệu, chỉ là sẽ chậm hơn có nhiều RAM thôi. Mỗi Executor có thể chứa dữ liệu của 1 hoặc 1 vài partition của 1 RDD.

Việc xử lý dữ liệu, nhìn rộng ra, chính là việc biến đổi từ tập dữ liệu này sang tập dữ liệu khác, hay với Spark, là biến đổi từ RDD này sang RDD khác. Ví dụ bạn có 1 tập web log rất lớn gồm cả log call từ app và web, cần tìm xem số lượng log sử dụng trên app là bao nhiêu, từ tập dữ liệu ban đầu sẽ lọc ra tập dữ liệu nhỏ hơn chỉ chứa log call trên di động, rồi count trên tập dữ liệu đó. Đó chính là minh họa cho việc biến đổi các tập dữ liệu.

Làm việc với RDD, Spark có 2 loại operations là Transformation và Actions.

2.1.4.1 RDD Transformations

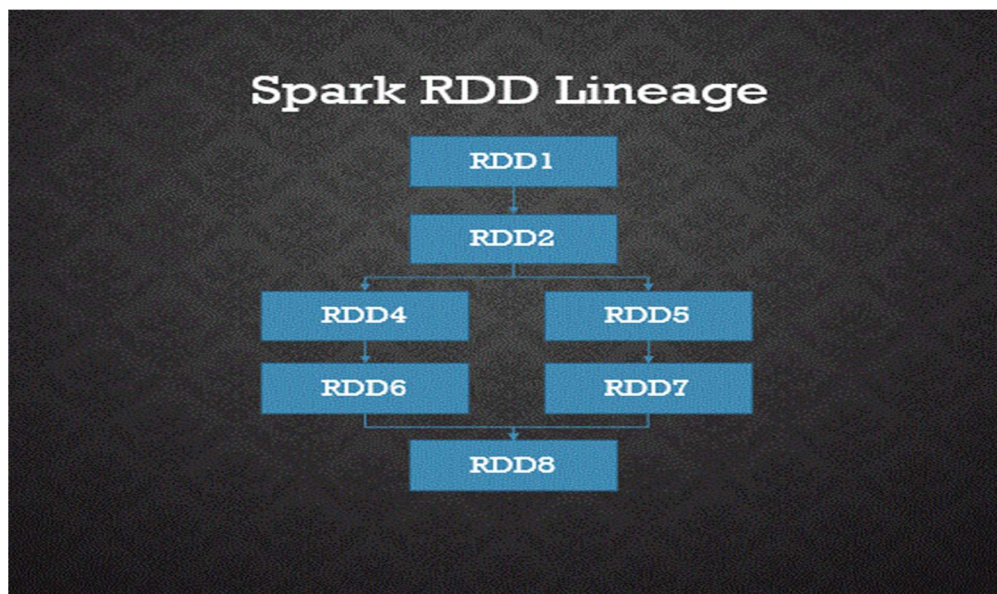


Phép biến đổi từ RDD này sang RDD khác là 1 transformation, như việc biến đổi tập web log ban đầu sang tập web log chỉ chứa log gọi qua app là 1 transformation.

Trong Apache Spark, các phép biến đổi (transformations) được áp dụng lên RDD (Resilient Distributed Dataset) làm kích hoạt quá trình tính toán trên dữ liệu phân tán. Mỗi khi một phép biến đổi được áp dụng, nó không thay đổi RDD gốc mà thay vào đó tạo ra một hoặc nhiều RDD mới. Điều này là do tính bất biến (immutable) của RDD, một đặc điểm cốt lõi trong Spark.

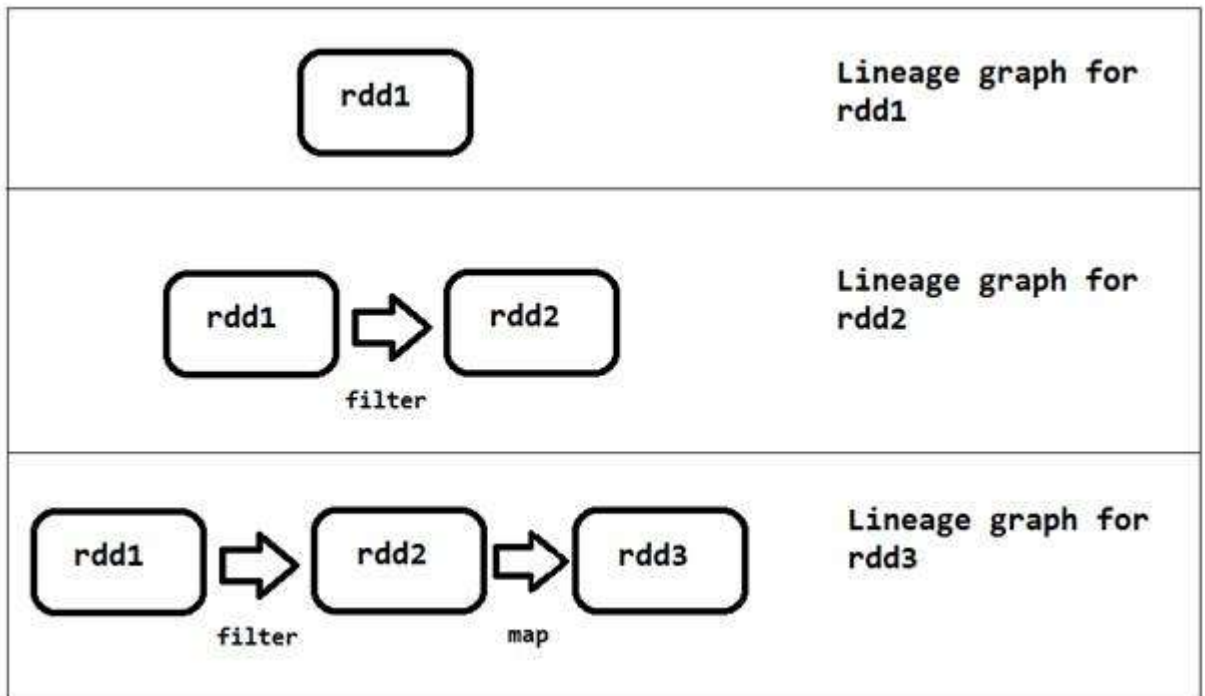
Tạo ra các RDD mới thông qua các phép biến đổi tạo ra một loạt các RDD liên kết gọi là RDD lineage hoặc RDD dependency. RDD lineage ghi lại cách mà các RDD được tạo ra từ các RDD gốc thông qua các phép biến đổi. Điều này không chỉ làm cho việc tái tính toán các RDD trở nên dễ dàng, mà còn cho phép Spark tái phục hồi dữ liệu trong trường hợp cần thiết, như mất mát dữ liệu hoặc lỗi trong quá trình tính toán.

Tóm lại, việc sử dụng RDD transformations trong Spark tạo ra một hệ thống linh hoạt và an toàn về mặt dữ liệu, bằng cách tạo ra các RDD mới mỗi khi cần thiết mà không làm thay đổi dữ liệu gốc và ghi lại lịch sử của các phép biến đổi thông qua RDD lineage.



RDD Transformations are Lazy

Khi thực thi, việc gọi các transformations, Spark sẽ không ngay lập tức thực thi các tính toán mà sẽ lưu lại thành 1 lineage, tức là tập hợp các biến đổi từ RDD này thành RDD khác qua mỗi transformation. Khi có 1 action được gọi, Spark lúc này mới thực sự thực hiện các biến đổi để trả ra kết quả.



Quay lại ví dụ lọc web log ban đầu. Nếu gọi đến đâu chạy đến đấy thì sau bước đầu tiên, ứng dụng cần load sẵn 1TB ram vào Mem, sẵn sàng thực hiện bước tiếp theo. Tuy nhiên, nếu lazy, Driver sẽ có “cái nhìn” toàn cảnh từ đầu đến cuối về output đầu ra, lúc này Driver sẽ sinh các task đọc từng phần nhỏ của 1TB, thực hiện lọc trên tập dữ liệu nhỏ và giữ lại kết quả count bản ghi là log app, load dần dần cho tới khi hết 1TB thì sum tổng các phần nhỏ lại sẽ ra được phần lớn.

Như vậy ta chỉ cần lượng ram nhỏ nhưng vẫn có thể xử lý được lượng dữ liệu lớn gấp nhiều lần. Trong Apache Spark, các phép biến đổi (transformations) trên RDD là các hoạt động lười biếng (lazy operations), có nghĩa là không có bất kỳ phép biến đổi nào được thực thi cho đến khi bạn gọi một hành động (action) trên RDD của Spark. Do RDD là bất biến (immutable), bất kỳ phép biến đổi nào trên nó cũng tạo ra một RDD mới, không làm thay đổi RDD hiện tại.

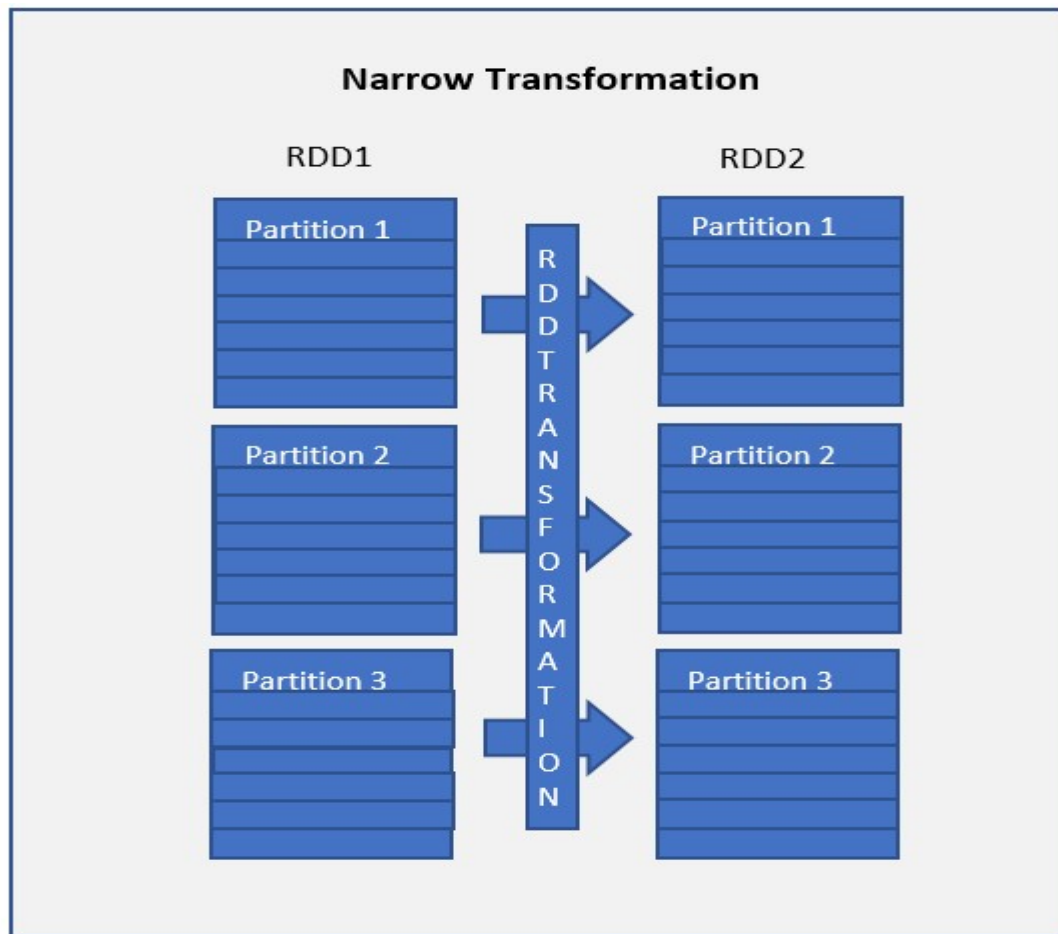
Có hai loại phép biến đổi trên RDD:

1. Narrow Transformation

Narrow transformations là kết quả của các hàm `map()` và `filter()`, và chúng tính toán dữ liệu tồn tại trên một phân vùng duy nhất, nghĩa là không có sự di chuyển dữ liệu giữa các phân vùng để thực hiện các phép biến đổi này.

Ví dụ, khi bạn áp dụng `map()` hoặc `filter()` lên một RDD, kết quả sẽ được tính toán

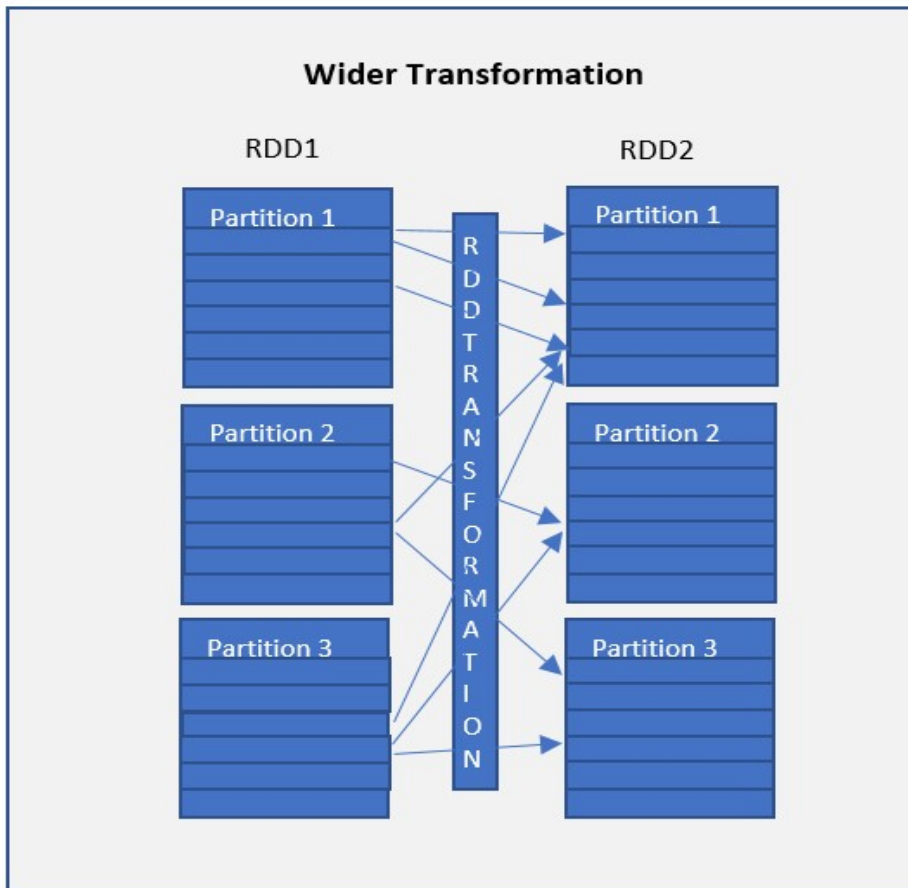
độc lập trên từng phân vùng của RDD mà không cần di chuyển dữ liệu giữa các phân vùng. Điều này làm giảm chi phí của phép biến đổi và tăng hiệu suất tính toán trong Spark.



2. Wide Transformation

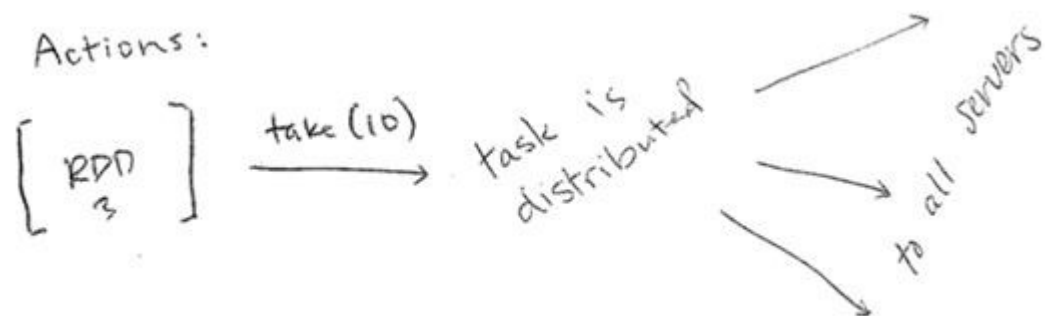
Wide transformations là kết quả của các hàm như **groupByKey()** và **reduceByKey()**. Những phép biến đổi này tính toán dữ liệu trên nhiều phân vùng của RDD, có nghĩa là có sự di chuyển dữ liệu giữa các phân vùng để thực hiện chúng. Do yêu cầu phải di chuyển dữ liệu giữa các phân vùng, những phép biến đổi này còn được gọi là shuffle transformations.

Ví dụ, khi sử dụng **groupByKey()** hoặc **reduceByKey()**, dữ liệu sẽ được tổ chức lại và các phần tử sẽ được nhóm lại hoặc tổng hợp trên nhiều phân vùng. Điều này đòi hỏi di chuyển dữ liệu giữa các executor để thực hiện các phép biến đổi này, tạo ra một quá trình gọi là shuffle. Quá trình shuffle có thể tốn kém về hiệu suất và làm giảm hiệu suất của ứng dụng Spark.



Phương thức Biến đổi	Sử dụng và Mô tả
cache()	Lưu trữ RDD trong bộ nhớ cache để sử dụng lại.
filter()	Trả về một RDD mới sau khi áp dụng hàm lọc trên tập dữ liệu nguồn.
flatMap()	Trả về một danh sách các phần tử đã được làm phẳng, từng phần tử trong một mảng sẽ được chuyển đổi thành một hàng. Nói cách khác, nó trả về 0 hoặc nhiều mục đầu ra cho mỗi phần tử trong tập dữ liệu.
map()	Áp dụng hàm biến đổi lên tập dữ liệu và trả về cùng số lượng phần tử trong tập dữ liệu phân phối.
mapPartitions()	Tương tự như map, nhưng thực hiện hàm biến đổi trên mỗi phân vùng. Điều này cung cấp hiệu suất tốt hơn so với hàm map.
mapPartitionsWithIndex()	Tương tự như mapPartitions, nhưng cũng cung cấp func với một giá trị số nguyên đại diện cho chỉ mục của phân vùng.
randomSplit()	Chia RDD thành các phần theo trọng số được chỉ định trong đối số. Ví dụ: <code>rdd.randomSplit(0.7, 0.3)</code>
union()	Kết hợp các phần tử từ tập dữ liệu nguồn và đối số và trả về tập dữ liệu kết hợp. Tương tự như hàm union trong các phép toán tập hợp toán học.
sample()	Trả về tập dữ liệu mẫu.
intersection()	Trả về tập dữ liệu chứa các phần tử có trong cả tập dữ liệu nguồn và đối số.
distinct()	Trả về tập dữ liệu bằng cách loại bỏ tất cả các phần tử trùng lặp.
repartition()	Trả về một tập dữ liệu với số lượng phân vùng được chỉ định trong đối số. Phép biến đổi này làm xáo trộn lại RDD ngẫu nhiên, có thể trả về RDD với số lượng phân vùng ít hơn hoặc nhiều hơn tùy thuộc vào đầu vào được cung cấp.
coalesce()	Tương tự như repartition nhưng hoạt động tốt hơn khi chúng ta muốn giảm số lượng phân vùng. Sự cải thiện được đạt được bằng cách xáo trộn dữ liệu từ ít nút hơn so với tất cả các nút bằng repartition.

2.1.4.1 RDD Actions



RDD actions là các hoạt động hoạt động trả về các giá trị cơ bản, tức là bất kỳ hàm RDD nào trả về giá trị khác RDD[T] được coi là một action trong lập trình Spark. Trong hướng dẫn này, chúng ta sẽ tìm hiểu về các hoạt động RDD với các ví dụ Scala.

Như đã đề cập trong RDD Transformations, tất cả các biến đổi đều là biến đổi lười biếng, có nghĩa là chúng không được thực hiện ngay lập tức và các hàm action kích hoạt để thực hiện các biến đổi.

Trong ngữ cảnh của Spark, RDD transformations thực hiện các thay đổi trên dữ liệu trong RDD, chẳng hạn như ánh xạ, lọc hoặc nhóm dữ liệu. Nhưng những thay đổi này không được thực hiện ngay lập tức khi gọi một hàm biến đổi. Thay vào đó, chúng chỉ đơn giản là mô tả các bước biến đổi mà Spark sẽ thực hiện khi một hành động được gọi. Điều này giúp tối ưu hóa hiệu suất và quản lý tài nguyên trong quá trình xử lý dữ liệu lớn.

Khi một hàm action được gọi trên RDD, Spark sẽ thực hiện tất cả các biến đổi liên quan đến RDD đó và trả về kết quả dưới dạng các giá trị cơ bản, chẳng hạn như các phần tử trong RDD, các giá trị tổng hợp, hoặc các dữ liệu được lấy từ RDD. Điều này làm cho các hàm action trở thành điểm kết thúc của một luồng tính toán trong Spark, là nơi mà các kết quả cuối cùng được trả về và xử lý.

Phương thức Action của RDD	Định nghĩa phương thức
aggregate[U](zeroValue: U)(seqOp: (U, T) ⇒ U, combOp: (U, U) ⇒ U)(implicit arg0: ClassTag[U]): U	Tổng hợp các phần tử của mỗi phân vùng và sau đó tổng hợp kết quả cho tất cả các phân vùng.
collect(): Array[T]	Trả về tập dữ liệu hoàn chỉnh dưới dạng một Mảng.
count(): Long	Trả về số lượng phần tử trong tập dữ liệu.
countApprox(timeout: Long, confidence: Double = 0.95): PartialResult[BoundedDouble]	Trả về số lượng phần tử xấp xỉ trong tập dữ liệu, phương pháp này trả về kết quả không hoàn chỉnh khi thời gian thực hiện đạt đến giới hạn thời gian.
countApproxDistinct(relativeSD: Double = 0.05): Long	Trả về số lượng xấp xỉ các phần tử độc nhất trong tập dữ liệu.
countByValue(): Map[T, Long]	Trả về một Map[T, Long] với khóa đại diện cho mỗi giá trị duy nhất trong tập dữ liệu và giá trị đại diện cho số lần xuất hiện của mỗi giá trị.
countByValueApprox(timeout: Long, confidence: Double = 0.95)(implicit ord: Ordering[T] = null): PartialResult[Map[T, BoundedDouble]]	Tương tự như countByValue() nhưng trả về kết quả xấp xỉ.
first(): T	Trả về phần tử đầu tiên trong tập dữ liệu.

fold(zeroValue: T)(op: (T, T) ⇒ T): T	Tổng hợp các phần tử của mỗi phân vùng và sau đó tổng hợp kết quả cho tất cả các phân vùng.
foreach(f: (T) ⇒ Unit): Unit	Lặp qua tất cả các phần tử trong tập dữ liệu bằng cách áp dụng hàm f cho tất cả các phần tử.
foreachPartition(f: (Iterator[T]) ⇒ Unit): Unit	Tương tự như foreach, nhưng áp dụng hàm f cho mỗi phân vùng.
min()(implicit ord: Ordering[T]): T	Trả về giá trị nhỏ nhất từ tập dữ liệu.
max()(implicit ord: Ordering[T]): T	Trả về giá trị lớn nhất từ tập dữ liệu.
reduce(f: (T, T) ⇒ T): T	Giảm các phần tử của tập dữ liệu bằng cách sử dụng toán tử nhị phân đã cho.
saveAsObjectFile(path: String): Unit	Lưu RDD dưới dạng các đối tượng được serialize vào hệ thống lưu trữ.
saveAsTextFile(path: String, codec: Class[_ <: CompressionCodec]): Unit	Lưu RDD dưới dạng một tập tin văn bản nén.
saveAsTextFile(path: String): Unit	Lưu RDD dưới dạng một tập tin văn bản.
take(num: Int): Array[T]	Trả về num phần tử đầu tiên của tập dữ liệu.
takeOrdered(num: Int)(implicit ord: Ordering[T]): Array[T]	Trả về num phần tử nhỏ nhất (hoặc lớn nhất) từ tập dữ liệu, đây là hoạt động ngược của hàm take().
takeSample(withReplacement: Boolean, num: Int, seed: Long = Utils.random.nextLong): Array[T]	Trả về một phần tử con của tập dữ liệu dưới dạng Mảng.
toLocalIterator(): Iterator[T]	Trả về tập dữ liệu hoàn chỉnh dưới dạng một Iterator.
top(num: Int)(implicit ord: Ordering[T]): Array[T]	Trả về top num phần tử từ tập dữ liệu, hãy sử dụng phương pháp này chỉ khi mảng kết quả nhỏ, vì toàn bộ dữ liệu sẽ được tải vào bộ nhớ của trình điều khiển.
treeAggregate	Tổng hợp các phần tử của RDD trong một mô hình cây đa cấp.
treeReduce	Giảm các phần tử của RDD trong một mô hình cây đa cấp.

Sau tất cả các phép biến đổi, khi muốn tương tác với kết quả cuối cùng (VD xem kết quả, collect kết quả, ghi kết quả...) ta gọi 1 action.

Có thể kể đến 1 số Action như:

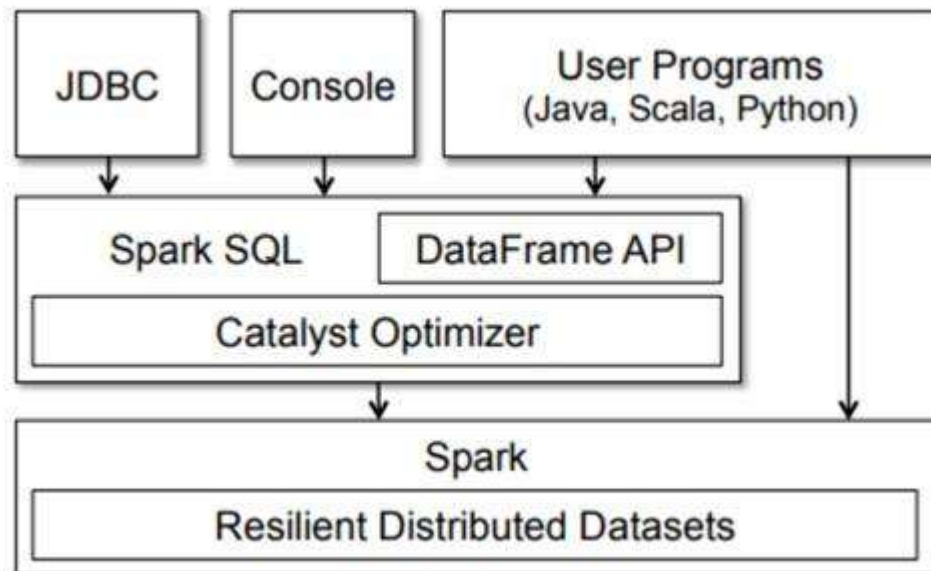
- take(n): lấy n bản ghi từ RDD về driver
- collect: lấy tất cả RDD về driver
- saveAsTextFile("path"): ghi dữ liệu RDD ra file
- count: đến số bản ghi của RDD

2.2 Spark SQL với DataFrame và DataSet

Apache Spark SQL là một thư viện mã nguồn mở được xây dựng trên nền tảng Spark Core, cung cấp các chức năng mạnh mẽ cho truy vấn và phân tích dữ liệu có cấu trúc. Spark SQL mở rộng khả năng của Spark Core bằng cách hỗ trợ các ngôn ngữ truy vấn quen thuộc như SQL và DataFrame API, giúp người dùng dễ dàng truy cập, thao tác và phân tích dữ liệu một cách hiệu quả.

Khi thực thi, Spark SQL vẫn sẽ gọi xuống lớp Core bên dưới, sử dụng RDD để tính toán. Có thể tóm gọn một số đặc điểm quan trọng của Spark SQL như sau:

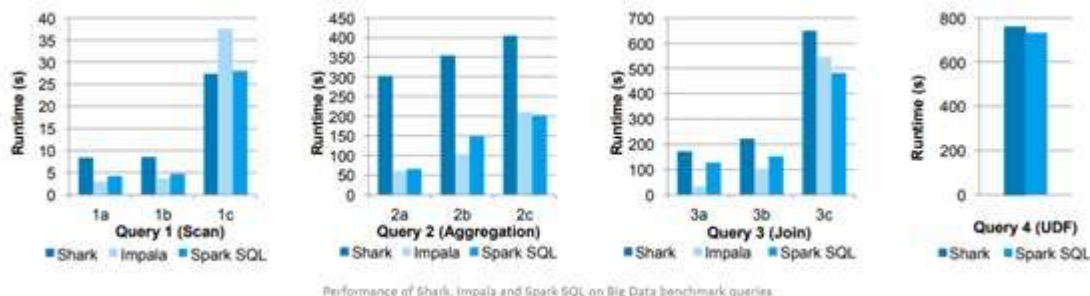
- Được xây dựng phía trên tầng Spark Core, thừa hưởng tất cả các tính năng mà RDD có.
- Làm việc với tập dữ liệu là DataSet hoặc DataFrame (tập dữ liệu phân tán, có cấu trúc)
- Hiệu năng cao, khả năng mở rộng và chịu lỗi tốt
- Tương tích với các thành phần khác trong tổng thể Spark Framework (như Streaming/ Mllib, GraphX)
- Bao gồm 2 thành phần là DataSet API và Catalyst Optimizer.



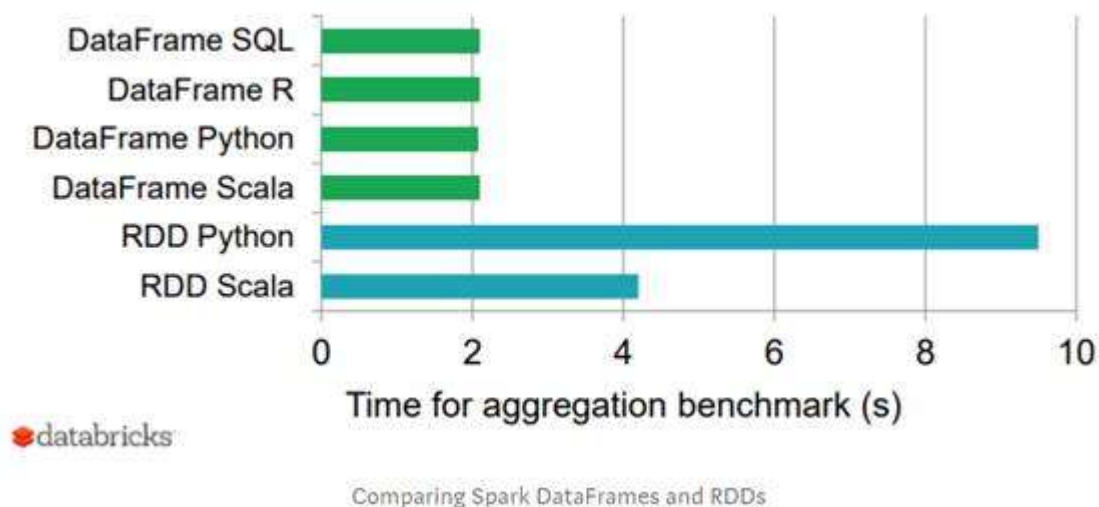
2.2.1 Hiệu năng

Trong việc biến đổi và tổng hợp dữ liệu, Spark SQL với DataFrame luôn có hiệu

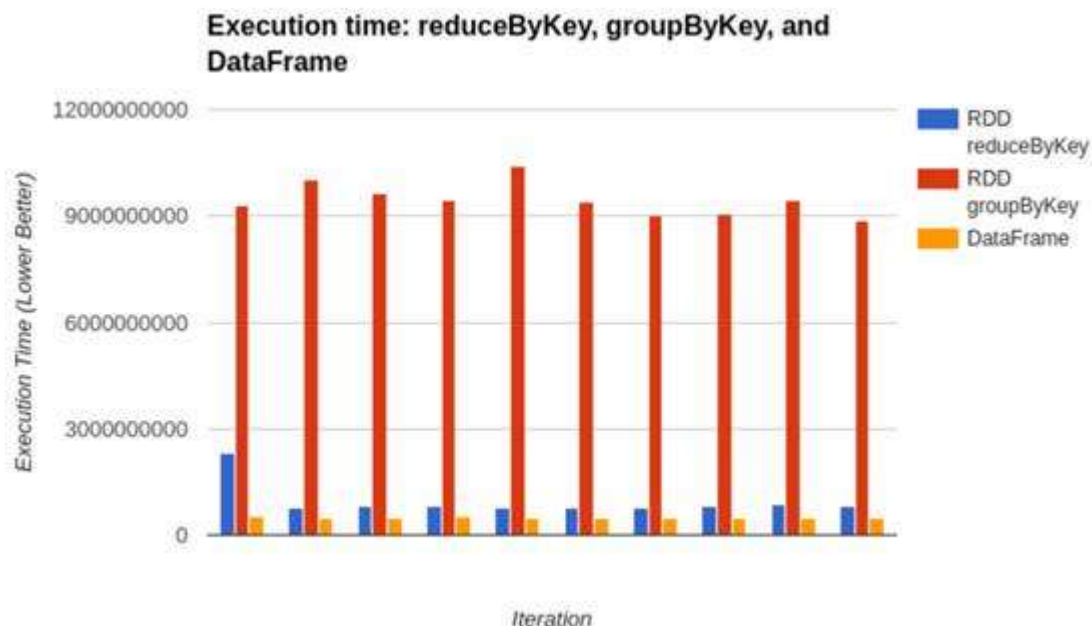
năng cao hơn rất nhiều so với sử dụng RDD. Còn so với các công nghệ tương tự khác như Impala hay Shark thì thời gian thực thi với Spark SQL cũng luôn rất ấn tượng.



Spark SQL, 1 thư viện nguồn mở có hiệu năng ngang ngửa với Impala, 1 query engine rất “xịn” đến từ Cloudera và tốt hơn Shark trong phần lớn các bài thử nghiệm.



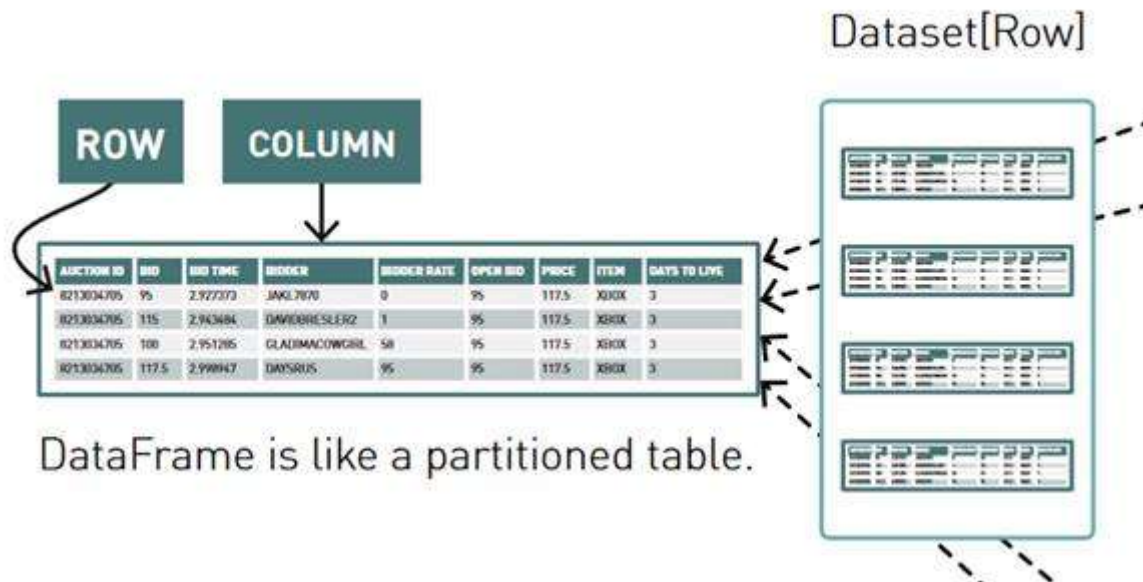
So với người anh là RDD trên cùng 1 công việc tổng hợp, Spark cho thời gian chạy nhanh hơn ít nhất 2 lần. Nếu chạy qua RDD Python là tới gần 5 lần.



Một bài thử nghiệm nữa là thực thi các aggregation thông qua RDD với 2 API là `reduceByKey` và `RDD groupByKey` với `DataFrame` và kết quả là `DataFrame` có thời gian xử lý nhanh hơn cả.

2.2.2 DataFrame

“`DataFrame` được định nghĩa là một tập dữ liệu phân tán, không thay đổi được, được tổ chức thành các hàng và cột. Mỗi hàng đại diện cho một bản ghi dữ liệu, mỗi cột có tên và kiểu dữ liệu riêng biệt.”

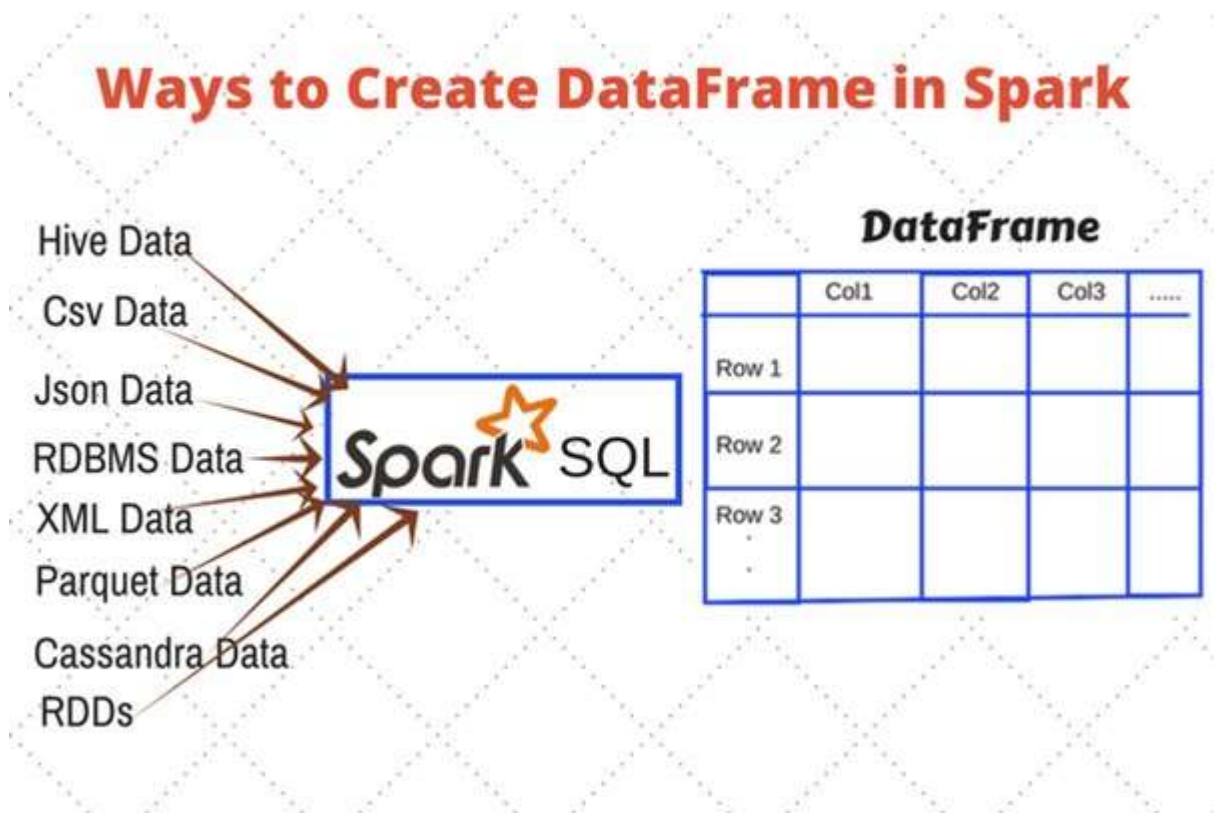


Nói một cách dễ hiểu: DataFrame tương tự như một bảng trong hệ quản trị cơ sở dữ liệu quan hệ (RDBMS), nhưng với khả năng lưu trữ và xử lý lượng dữ liệu khổng lồ vượt trội. Spark SQL đã từng xử lý thành công DataFrame chứa hàng chục tỷ bản ghi mà không gặp bất kỳ vấn đề nào.

Các đặc điểm bao gồm:

- **Tính bất biến (Immutability):** Dữ liệu trong DataFrame không thể thay đổi sau khi tạo. Nếu cần chỉnh sửa, bạn cần tạo một DataFrame mới từ DataFrame ban đầu thông qua API.
- **Hàng (Rows):** Mỗi hàng đại diện cho một bản ghi dữ liệu cụ thể. Toàn bộ DataFrame là tập hợp các hàng phân tán trên nhiều nút trong cụm Spark.
- **Cột (Columns):** Mỗi cột có tên và kiểu dữ liệu riêng biệt, giúp định nghĩa cấu trúc của dữ liệu.
- **Hỗ trợ nhiều kiểu dữ liệu:** Spark SQL hỗ trợ nhiều kiểu dữ liệu đa dạng, từ các kiểu cơ bản như số nguyên, chuỗi, đến các kiểu phức tạp hơn như ngày tháng, thời gian, địa điểm, v.v.
- **Đọc dữ liệu từ nhiều nguồn:** DataFrame Reader là công cụ mạnh mẽ cho phép bạn đọc dữ liệu từ nhiều nguồn và định dạng khác nhau như CSV, JSON, Parquet, ORC, Hive tables, v.v.
- **API phong phú:** Spark SQL cung cấp API phong phú cho phép bạn dễ dàng thao

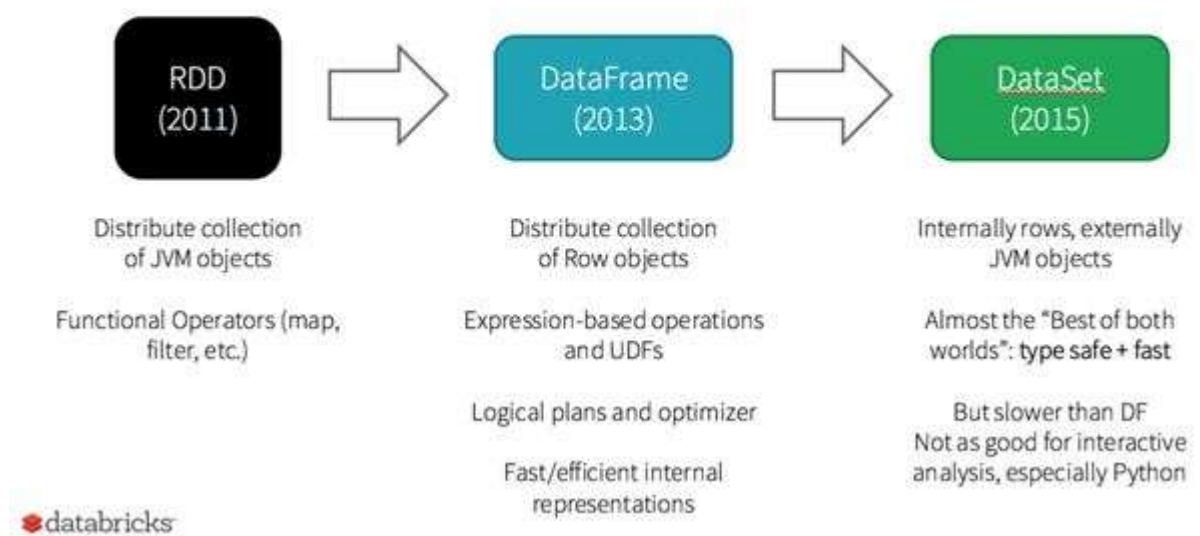
tác với DataFrame như filter, sort, aggregate, join,...



2.2.3 DataFrame và DataSet

DataFrame và Dataset là hai khái niệm quan trọng trong Spark SQL, thường xuyên khiến người mới bắt đầu cảm thấy bối rối.

History of Spark APIs

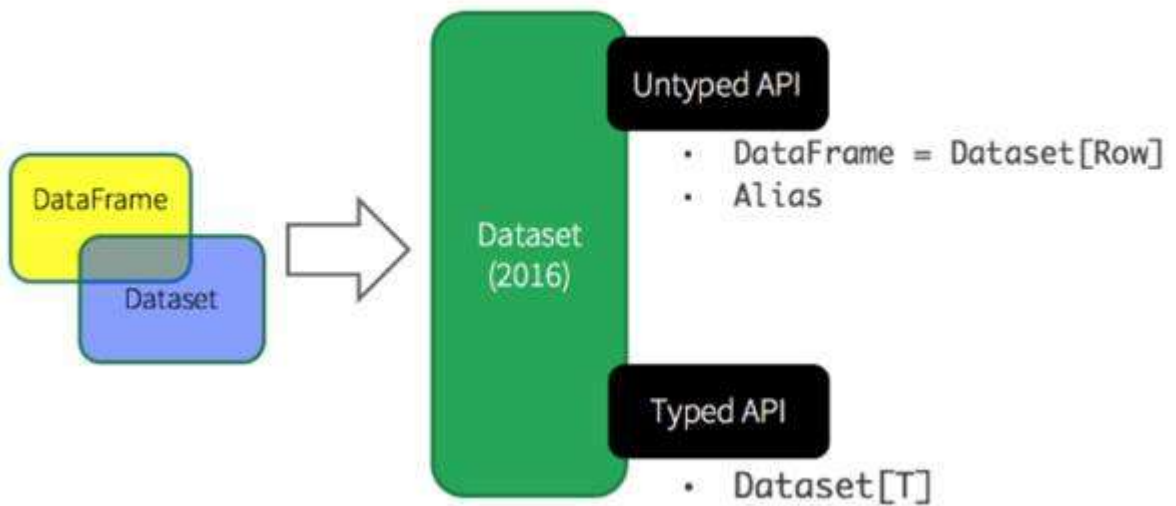


Theo chiều trái qua phải là sự cải tiến dần các đối tượng dữ liệu phân tán trong Spark

Về lịch sử phát triển, DataFrame được giới thiệu vào năm 2013 trong Spark 1.0 và DataSet là 2015 trong Spark 1.6. Tới 2016, Spark 2.0 ra mắt thống nhất DataFrame và Dataset thành một API Dataset chung.

DataFrame và Dataset đều là các tập dữ liệu phân tán được sử dụng trong Spark SQL để truy vấn và phân tích dữ liệu, hỗ trợ nhiều kiểu dữ liệu và cung cấp API phong phú cho phép thao tác với dữ liệu như lọc, sắp xếp, tổng hợp, join, ... Có thể được đọc từ nhiều nguồn dữ liệu khác nhau như CSV, JSON, Parquet, ORC, Hive tables, v.v.

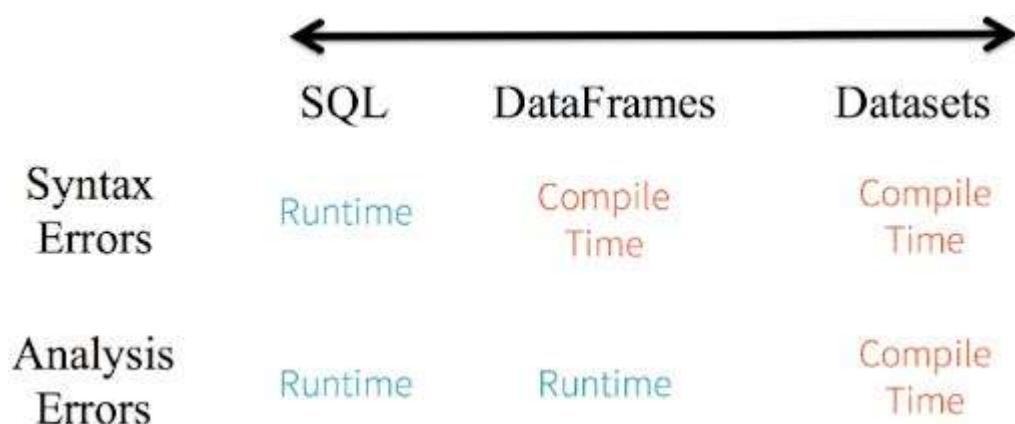
Unified Apache Spark 2.0 API



- DataFrame = Dataset [Row] (Untyped API)
- DataSet = Dataset [Type]

Đặc điểm	DataFrame	Dataset
Kiểu dữ liệu của bản ghi	Mặc định là Row (không xác định kiểu)	Có thể tùy chỉnh bằng case class (xác định kiểu)
Truy cập dữ liệu	Dựa vào tên cột	Dựa vào thuộc tính của case class
Kiểm tra lỗi	Lỗi xảy ra khi runtime	Lỗi được phát hiện khi compile
Hiệu suất	Có thể chậm hơn Dataset do cần suy luận kiểu dữ liệu	Nhanh hơn DataFrame do kiểu dữ liệu được xác định rõ ràng
Tính linh hoạt	Thích hợp cho các trường hợp không cần xác định rõ kiểu dữ liệu	Thích hợp cho các trường hợp cần kiểm soát chặt chẽ kiểu dữ liệu và phát hiện lỗi sớm

Một nhược điểm của DF là do kiểu dữ liệu được fix là row và truy cập dữ liệu trong DF thông qua row name nên nếu có sai sót trong việc truyền tên cột, trình biên dịch sẽ không thể phát hiện ra lỗi mà khi thực thi mới có exeption (Runtime exception). DS có thể khắc phục nhược điểm này do có sẵn định nghĩa của 1 bản ghi, dữ liệu trong DS truy cập thông qua member của case class chứ không phải truyền tên như DF.



Hãy xem 1 ví dụ:

Giả sử ta có tập dữ liệu phim ảnh với cột `movie_title` kiểu `String`. Khi trừ một số khỏi cột này, ta sẽ thấy sự khác biệt giữa `DataFrame` và `Dataset`:

actor_name	movie_title	produced_year
Cooper, Chris (I)	The Town	2010
Jolie, Angelina	Salt	2010
Jolie, Angelina	The Tourist	2010
Danner, Blythe	Little Fockers	2010
Byrne, Michael (I)	Harry Potter and ...	2010

Dữ liệu cột `movie_title` là kiểu `string`, thử dùng toán tử trừ với cột dữ liệu này, sử dụng `DF` và `DS`:

```
// demonstrating a type-safe transformation that fails at compile time,
performing subtraction on a column with string type

// a problem is not detected for DataFrame until runtime
movies.select('movie_title - 'movie_title)

// a problem is detected at compile time
moviesDS.map(m => m.movie_title - m.movie_title)
error: value - is not a member of String
```

Có thể thấy rằng khi dùng DF, ta vẫn build được code nhưng khi run, ứng dụng sẽ báo lỗi do khi chạy, DF mới biết được kiểu dữ liệu thật của cột `movie_title` là string cho dùng ở schema bạn đã chỉ định cột đó là string, trong khi nếu đã định nghĩa 1 class có thuộc tính `movie_title` là string thì ứng dụng sẽ báo lỗi ngay tại thời điểm compile. Do đó nên sử dụng Dataset khi cần xác định rõ ràng kiểu dữ liệu của bản ghi, kiểm soát chặt chẽ kiểu dữ liệu và phát hiện lỗi sớm. Nên sử dụng DataFrame khi không cần xác định rõ kiểu dữ liệu hoặc cần sự linh hoạt trong thao tác dữ liệu.

2.3 Spark Streaming

Apache Spark Streaming là một thành phần quan trọng của hệ sinh thái Apache Spark, được thiết kế để xử lý dữ liệu dòng liên tục (streaming data) trong thời gian thực. Vai trò chính của Apache Spark Streaming là cung cấp một cách tiếp cận phong phú và linh hoạt để xây dựng các ứng dụng xử lý dữ liệu dòng liên tục trong môi trường phân tán.

Dữ liệu dòng liên tục thường là những dữ liệu được tạo ra và truyền đi một cách liên tục trong thời gian thực từ các nguồn như cảm biến, logs hệ thống, hoặc luồng dữ liệu trực tuyến từ các ứng dụng web. Spark Streaming cho phép xử lý và phân tích dữ liệu này trong thời gian thực, giúp các nhà phân tích và nhà phát triển ứng dụng đưa ra quyết định và hành động nhanh chóng dựa trên dữ liệu mới nhận được.

Spark Streaming hoạt động bằng cách nhận và xử lý dữ liệu dòng liên tục theo các bước cơ bản sau:



Bước 1: Nhận Dữ Liệu Dòng Liên Tục: Spark Streaming nhận dữ liệu dòng liên tục từ các nguồn như Kafka, Kinesis, Flume, hoặc TCP sockets.

Bước 2: Chia Dữ Liệu thành Các Lô: Dữ liệu nhận được được chia thành các lô (batches) nhỏ, thường với kích thước cố định. Các lô này được xử lý riêng lẻ bởi Spark, giúp tối ưu hóa việc xử lý dữ liệu.

Bước 3: Xử Lý Dữ Liệu Dòng Liên Tục bằng DStream: Dữ liệu trong mỗi lô được xử lý thông qua khái niệm DStream (Discretized Stream). DStream là một chuỗi các RDD (Resilient Distributed Dataset), mỗi RDD đại diện cho dữ liệu trong một lô cụ thể.

Bước 4: Tích Hợp với Apache Spark: Spark Streaming tích hợp tốt với Apache Spark, cho phép sử dụng các tính năng và thư viện mạnh mẽ của Spark như map, filter, reduceByKey, và các biến đổi khác để xử lý dữ liệu dòng liên tục.

Kết quả sau khi xử lý có thể được đẩy ra hệ thống tệp, cơ sở dữ liệu, hoặc bảng điều khiển trực tiếp.



Các thành phần cơ bản của Spark Streamming

- StreamingContext:

StreamingContext là một đối tượng quan trọng trong Spark Streaming, chịu trách nhiệm khởi tạo quá trình xử lý dữ liệu dòng liên tục. Đây là một đối tượng chính trong Spark Streaming API, cho phép tạo ra các DStream và cấu hình cách xử lý dữ liệu.

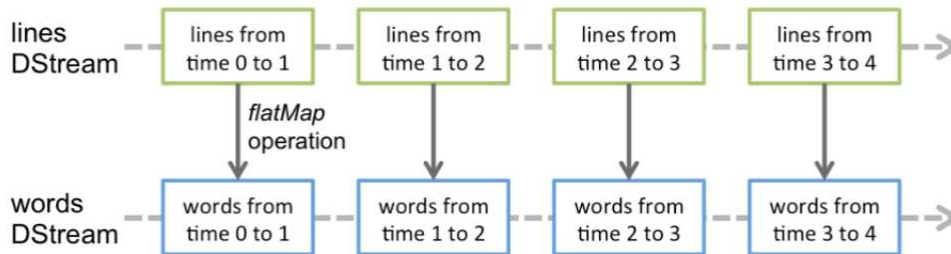
StreamingContext có thể được tạo ra từ một SparkContext đã tồn tại, hoặc thông qua việc tạo mới từ một cặp SparkConf và batch interval.

- DStream (Discretized Stream):

DStream (Discretized Stream) là một khái niệm trừu tượng trong Spark Streaming, biểu thị một luồng dữ liệu liên tục được chia thành các lô (batches) và xử lý theo từng khoảng thời gian. Mỗi DStream được biểu diễn bằng một chuỗi RDD (Resilient Distributed Dataset), là sự trừu tượng hóa của Spark về một tập dữ liệu phân tán và bất biến. Mỗi RDD trong DStream chứa dữ liệu từ một khoảng thời gian nhất định, cho phép xử lý dữ liệu dòng liên tục một cách linh hoạt và hiệu quả.



Các hoạt động áp dụng trên DStream được chuyển đổi thành các hoạt động trên RDD cơ bản. Ví dụ, khi chuyển đổi dòng thành từ, thao tác **flatMap** được áp dụng trên mỗi RDD trong DStream để tạo một RDD của wordsDStream



CHƯƠNG 3: CÀI ĐẶT HADOOP VÀ HADOOP CLUSTER

3.1 Cài đặt JDK

3.1.1 Tải JDK 1.8

Hadoop sử dụng JDK 1.8

Ta vào link sau: [Java Archive Downloads - Java SE 8 \(oracle.com\)](https://www.oracle.com/technetwork/java/javase-downloads-1344925.html)

Windows x64	211.58 MB	 jdk-8u202-windows-x64.exe
-------------	-----------	---

Sau khi tải về ta có file

 jdk-8u202-windows-x64	4/5/2024 8:02 AM	Application	216,653 KB
---	------------------	-------------	------------

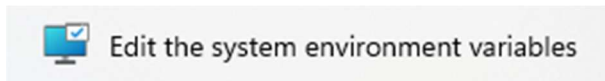
Sau đó chạy để cài đặt các gói về

 jdk-1.8	4/5/2024 8:12 AM	File folder
 jdk-22	3/27/2024 4:01 AM	File folder
 jre-1.8	4/5/2024 8:12 AM	File folder

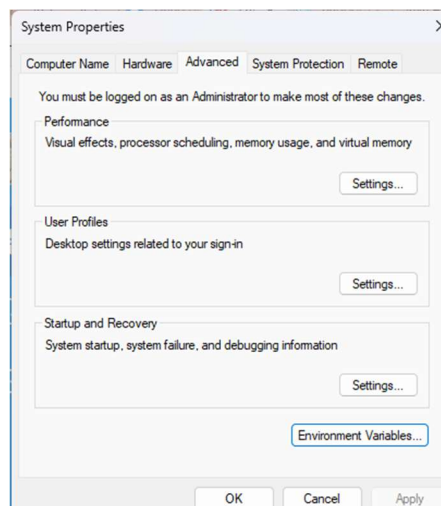
 Java 11	8/24/2023 2:15 PM	File folder
---	-------------------	-------------

3.1.2 Cài đặt môi trường JDK

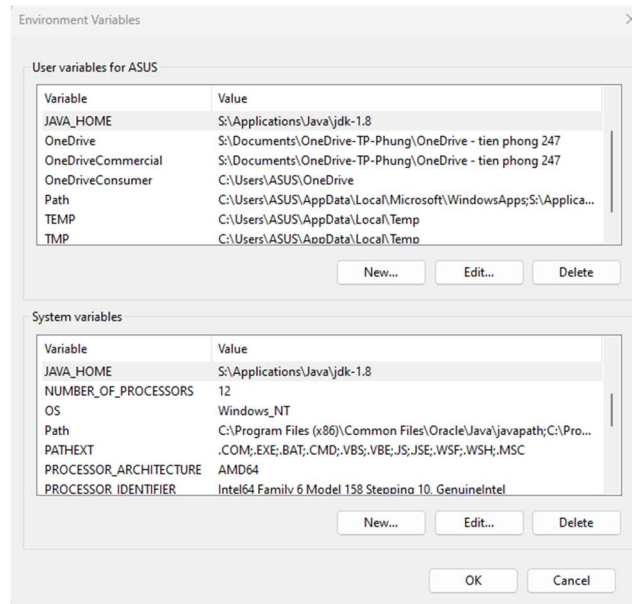
Có thể tìm ở thanh công cụ “Edit the system environment variables”



Chọn “Environment variables”



Dẫn đường dẫn phù hợp cho JAVA_HOME



Và Path



Kiểm tra xem hệ thống đã nhận jdk chưa

```
C:\Users\ASUS>java -version
java version "1.8.0_202"
Java(TM) SE Runtime Environment (build 1.8.0_202-b08)
Java HotSpot(TM) 64-Bit Server VM (build 25.202-b08, mixed mode)
```

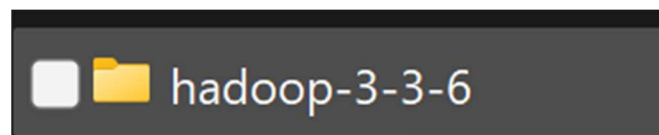
3.2 Cài đặt Hadoop

Đầu tiên chúng ta tải Hadoop

Hadoop sử dụng Hadoop-3.3.6

Ta vào link sau: [Apache Hadoop](#)

Sau khi tải về và giải nén

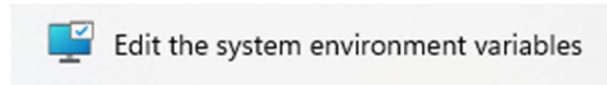


Cài đặt biến môi trường Hadoop

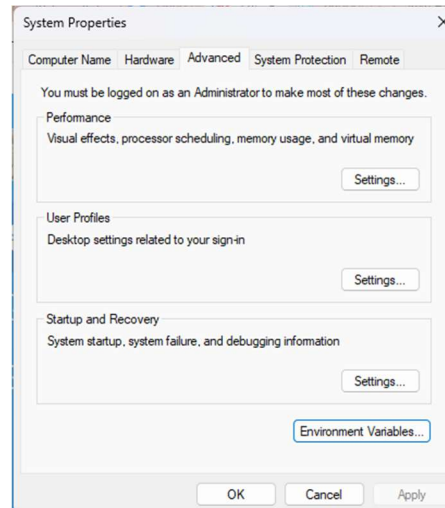
Máy master và máy slave đều phải cài đặt chung 1 đường dẫn

Ví dụ ở dưới là “C:\hadoop-3-3-6” thì cả máy master và slave đều lưu hadoop ở địa chỉ “C:\hadoop-3-3-6”

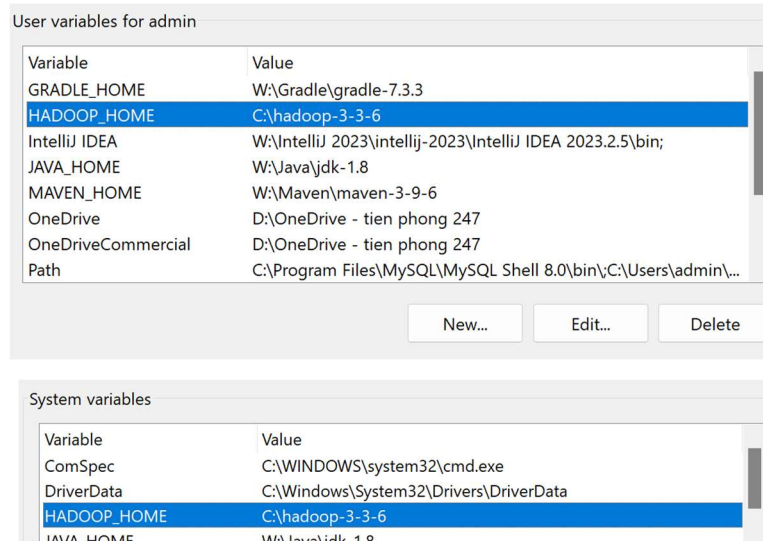
Có thể tìm ở thanh công cụ “Edit the system environment variables”



Chọn “Environment variables”



Dẫn đường dẫn phù hợp cho HADOOP



Và Path

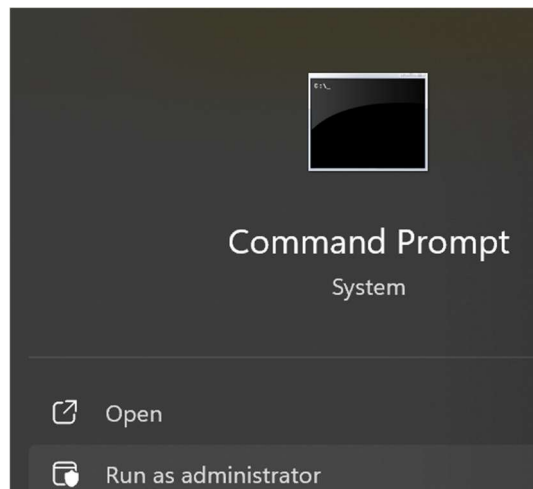
```
%HADOOP_HOME%\bin  
%HADOOP_HOME%\sbin
```

3.3 Cấu hình các file của Hadoop

3.3.1 Cấu hình địa chỉ IP

Mở command dưới quyền admin

Notepad C:\Windows\System32\Drivers\etc\hosts



Điền hostname

192.168.171.148 ngocphung-master

192.168.171.233 phuthanh-slave

192.168.171.201 trantien-slave

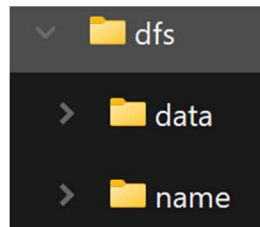
192.168.171.179 thuyquynh-slave

192.168.56.1 tientran-slave

```
# ... localhost
192.168.171.148 ngocphung-master
192.168.171.233 phuthanh-slave
192.168.171.201 trantien-slave
192.168.171.179 thuyquynh-slave
192.168.56.1 tientran-slave
```

Tại ổ C tạo folder dfs

Trong folder dfs có 2 folder con là name và data



Tạo thêm folder tmp vào ổ C

3.3.2 Cấu hình core-site.xml (cấu hình trên cả master và slave)

Vào địa chỉ "C:\hadoop-3-3-6\etc\hadoop\core-site.xml

```
<configuration>
  <property>
    <name>fs.default.name</name>
    <value>hdfs://ngocphung-master:9000</value>
  </property>
  <property>
    <name>dfs.permissions</name>
    <value>>false</value>
  </property>
  <property>
    <name>hadoop.tmp.dir</name>
    <value>/C:/tmp</value>
    <description>A base for other temporary directories.</description>
  </property>
</configuration>
```

```
<configuration>
  <property>
    <name>fs.default.name</name>
    <value>hdfs://ngocphung-master:9000</value>
  </property>
  <property>
    <name>dfs.permissions</name>
    <value>>false</value>
  </property>

  <property>
    <name>hadoop.tmp.dir</name>
    <value>/C:/tmp</value>
    <description>A base for other temporary directories.</description>
  </property>
</configuration>
```

3.3.3 Cấu hình mapred-site.xml (cấu hình trên cả master và slave)

Vào địa chỉ "C:\hadoop-3-3-6\etc\hadoop\mapred-site.xml"

```
<configuration>
  <property>
    <name>mapred.job.tracker</name>
    <value>ngocphung-master:9001</value>
  </property>
  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
  </property>
</configuration>
```

```
<configuration>
  <property>
    <name>mapred.job.tracker</name>
    <value>ngocphung-master:9001</value>
  </property>

  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
  </property>
</configuration>
```

3.3.4 Cấu hình hdfs-site.xml (cấu hình trên cả master và slave)

Vào địa chỉ "C:\hadoop-3-3-6\etc\hadoop\hdfs-site.xml"

```
<configuration>
  <property>
    <name>dfs.name.dir</name>
    <value>C:/dfs/name</value>
  </property>
  <property>
    <name>dfs.data.dir</name>
    <value>C:/dfs/data</value>
  </property>
  <property>
    <name>dfs.replication</name>
    <value>4</value>
  </property>
  <property>
    <name>dfs.permissions</name>
    <value>>false</value>
  </property>
  <property>
    <name>dfs.datanode.use.datanode.hostname</name>
    <value>>false</value>
  </property>
  <property>
    <name>dfs.namenode.datanode.registration.ip-hostname-check</name>
    <value>>false</value>
  </property>
  <property>
    <name>dfs.namenode.http-address</name>
    <value>ngocphung-master:50070</value>
    <description>Your NameNode hostname for httpaccess.</description>
  </property>
  <property>
    <name>dfs.namenode.secondary.http-address</name>
    <value>ngocphung-master:50090</value>
    <description>Your Secondary NameNode hostname for httpaccess.</description>
  </property>
</configuration>
```

```

<configuration>
  <property>
    <name>dfs.name.dir</name>
    <value>/C:/dfs/name</value>
  </property>

  <property>
    <name>dfs.data.dir</name>
    <value>/C:/dfs/data</value>
  </property>

  <property>
    <name>dfs.replication</name>
    <value>4</value>
  </property>

  <property>
    <name>dfs.permissions</name>
    <value>>false</value>
  </property>

  <property>
    <name>dfs.datanode.use.datanode.hostname</name>
    <value>>false</value>
  </property>

  <property>
    <name>dfs.namenode.datanode.registration.ip-hostname-check</name>
    <value>>false</value>
  </property>

  <property>
    <name>dfs.namenode.http-address</name>
    <value>ngocphung-master:50070</value>
    <description>Your NameNode hostname for httpaccess.</description>
  </property>

  <property>
    <name>dfs.namenode.secondary.http-address</name>
    <value>ngocphung-master:50090</value>
    <description>Your Secondary NameNode hostname for httpaccess.</description>
  </property>
</configuration>

```

3.3.5 Cấu hình yarn-site.xml (cấu hình trên cả master và slave)

Vào trong địa chỉ "C:\hadoop-3-3-6\etc\hadoop\yarn-site.xml"

```
<configuration>
  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
    <description>Long running service which executes on Node Manager(s) and provides
MapReduce Sort and Shuffle functionality.</description>
  </property>
  <property>
    <name>yarn.log-aggregation-enable</name>
    <value>true</value>
    <description>Enable log aggregation so application logs are moved onto hdfs and are
viewable via web ui after the application completed. The default location on hdfs is '/log' and can be
changed via yarn.nodemanager.remote-app-log-dir property</description>
  </property>
  <property>
    <name>yarn.resourcemanager.scheduler.address</name>
    <value>ngocphung-master:8030</value>
  </property>

  <property>
    <name>yarn.resourcemanager.resource-tracker.address</name>
    <value>ngocphung-master:8031</value>
  </property>
  <property>
    <name>yarn.resourcemanager.address</name>
    <value>ngocphung-master:8032</value>
  </property>
  <property>
    <name>yarn.resourcemanager.admin.address</name>
    <value>ngocphung-master:8033</value>
  </property>
  <property>
    <name>yarn.resourcemanager.webapp.address</name>
    <value>ngocphung-master:8088</value>
  </property>
</configuration>
```

```

<configuration>
  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
    <description>Long running service which executes on Node Manager(s) and provides MapReduce
Sort and Shuffle functionality.</description>
  </property>

  <property>
    <name>yarn.log-aggregation-enable</name>
    <value>true</value>
    <description>Enable log aggregation so application logs are moved onto hdfs and are viewable
via web ui after the application completed. The default location on hdfs is '/log' and can be changed via
yarn.nodemanager.remote-app-log-dir property</description>
  </property>

  <property>
    <name>yarn.resourcemanager.scheduler.address</name>
    <value>ngocphung-master:8030</value>
  </property>

  <property>
    <name>yarn.resourcemanager.resource-tracker.address</name>
    <value>ngocphung-master:8031</value>
  </property>

  <property>
    <name>yarn.resourcemanager.address</name>
    <value>ngocphung-master:8032</value>
  </property>

  <property>
    <name>yarn.resourcemanager.admin.address</name>
    <value>ngocphung-master:8033</value>
  </property>

  <property>
    <name>yarn.resourcemanager.webapp.address</name>
    <value>ngocphung-master:8088</value>
  </property>
</configuration>

```

3.3.6 Cấu hình hadoop-env.cmd (cấu hình trên cả master và slave)

Vào địa chỉ "C:\hadoop-3-3-6\etc\hadoop\hadoop-env.cmd"

Set up địa chỉ java giống vào java_home

```

@rem The java implementation to use.  Required.
set JAVA_HOME=W:\Java\jdk-1.8

```

Dán lệnh này ở cuối file

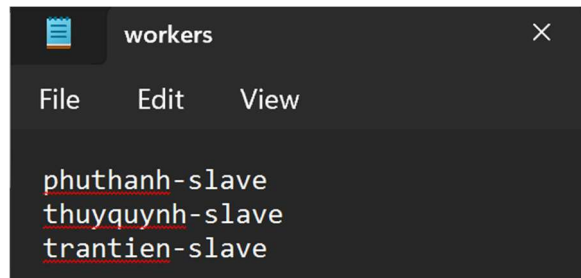
```
set HADOOP_IDENT_STRING=%USERNAME%
set HADOOP_PREFIX=%HADOOP_HOME%
set HADOOP_CONF_DIR=%HADOOP_PREFIX%\etc\hadoop
set YARN_CONF_DIR=%HADOOP_CONF_DIR%
set PATH=%PATH%;%HADOOP_PREFIX%\bin
```

```
@rem A string representing this instance of hadoop. %USERNAME% by default.
set HADOOP_IDENT_STRING=%USERNAME%
set HADOOP_PREFIX=%HADOOP_HOME%
set HADOOP_CONF_DIR=%HADOOP_PREFIX%\etc\hadoop
set YARN_CONF_DIR=%HADOOP_CONF_DIR%
set PATH=%PATH%;%HADOOP_PREFIX%\bin
```

Edit file workers (Làm trên máy master)

Vào địa chỉ "C:\hadoop-3-3-6\etc\hadoop\workers"

Trên máy master chỉ ghi tên máy slave



Edit file workers (Làm trên máy slave)

Vào địa chỉ "C:\hadoop-3-3-6\etc\hadoop\workers"

Trên máy slave chỉ ghi tên máy của mình

3.3.7 Cập nhật file BIN

Tải từ link này: <https://github.com/s911415/apache-hadoop-3.1.0-winutils>

Tải về và extract ra

Name	Date modified	Type	Size
winutils-master	4/5/2024 9:33 AM	File folder	
winutils-master	4/5/2024 7:31 AM	Compressed (zipp...	23,920 KB

Tìm folder gần với phiên bản của phiên bản mình dùng nhất

Em dùng hadoop-3.3.6 nên em sẽ dùng folder hadoop-3.3.5

Big Data > Week 10 > winutils-master > winutils-master >			
Sort View ...			
Name	Date modified	Type	
hadoop-3.3.5	4/5/2024 9:34 AM	File folder	

Copy file bin và đè lên file bin của “S:\Applications\Hadoop\hadoop-3-3-6”

Week 10 > winutils-master > winutils-master > hadoop-3.3.5 >			
Sort View ...			
Name	Date modified	Type	Size
bin	4/5/2024 9:34 AM	File folder	

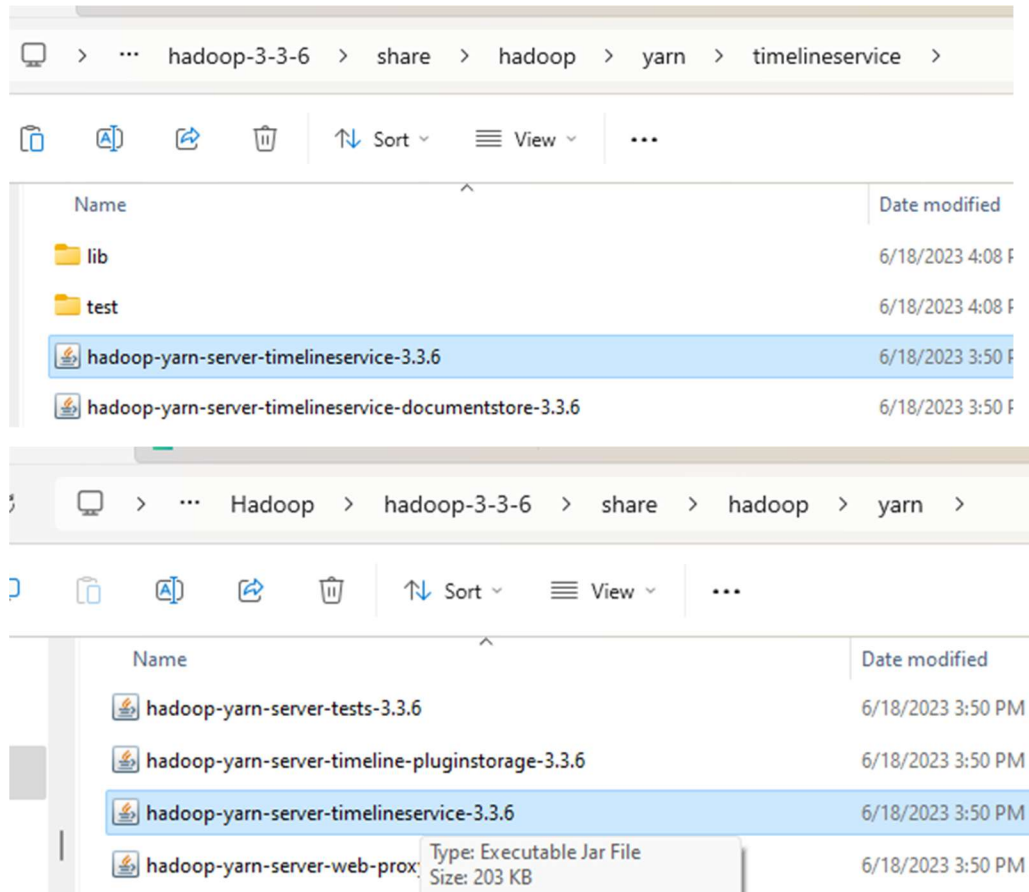
3.3.8 Format hệ thống

Hdfs namenode -format

```
S:\Applications\Hadoop\hadoop-3-3-6\bin>hdfs namenode -format
```

Sau đó ta Copy file hadoop-yarn-server-timelineservice-3.3.6

Và dán ra “S:\Applications\Hadoop\hadoop-3-3-6\share\hadoop\yarn”



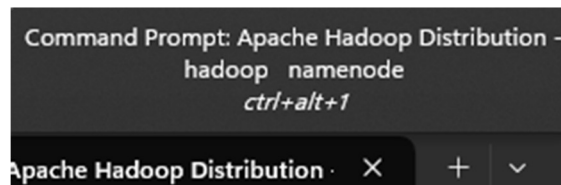
3.3.9 Chạy thử

Start-all

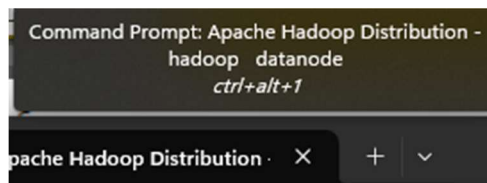
```
S:\Applications\Hadoop\hadoop-3-3-6\bin>start-all
This script is Deprecated. Instead use start-dfs.cmd and start-yarn.cmd
starting yarn daemons

S:\Applications\Hadoop\hadoop-3-3-6\bin>jps
2080 ResourceManager
3012 DataNode
13996 NameNode
17452 Jps
6076 NodeManager
```

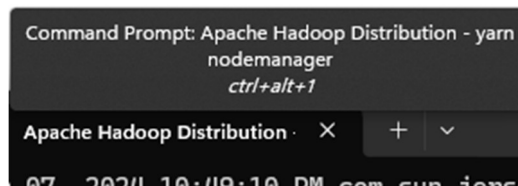
Namenode



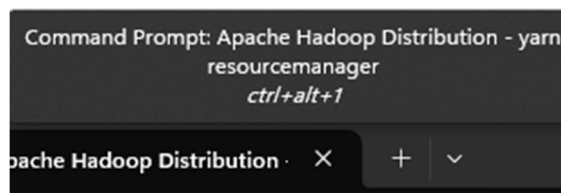
Datanode



Nodemanager

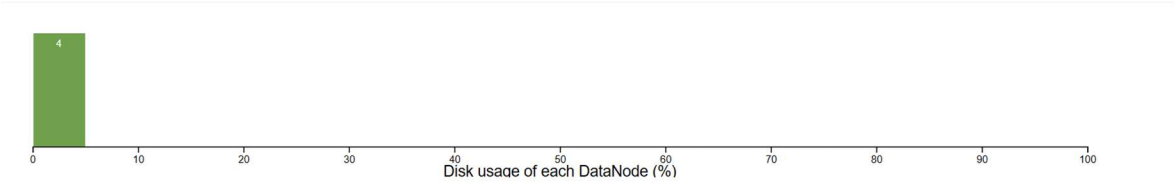


Resourcemanager



Kết quả, tạo Hadoop cluster thành công, bốn máy có trong hệ thống gồm 1 máy master và 3 máy slave

Datanode usage histogram



In operation

DataNode State: All Show: 25 entries Search:

Node	Http Address	Last contact	Last Block Report	Used	Non DFS Used	Capacity	Blocks	Block pool used	Version
✓/default-rack/trantien-slave:9866 (192.168.171.201:9866)	http://trantien-slave:9864	508s	34m	322 B	164.4 GB	216.57 GB	0	322 B (0%)	3.3.6
✓/default-rack/phuthanh-slave:9866 (192.168.171.233:9866)	http://phuthanh-slave:9864	0s	0m	322 B	289.75 GB	377.94 GB	0	322 B (0%)	3.3.6
✓/default-rack/192.168.232.1:9866 (192.168.171.148:9866)	http://192.168.232.1:9864	2s	78m	322 B	176.64 GB	275.54 GB	0	322 B (0%)	3.3.6
✓/default-rack/thuyquynh-slave:9866 (192.168.171.179:9866)	http://thuyquynh-slave:9864	1s	33m	322 B	191.53 GB	226.98 GB	0	322 B (0%)	3.3.6

Showing 1 to 4 of 4 entries

Previous **1** Next



Nodes of the cluster

Logged in as: dr:who

Cluster Metrics

Apps Submitted	0	Apps Pending	0	Apps Running	0	Apps Completed	0	Containers Running	0	Used Resources	<memory:0 B, vCores:0>	Total Resources	<memory:32 GB, vCores:32>	Reserved Resources	<memory:0 B, vCores:0>	Physical Mem Used %	83	Physical VCores Used %	0
----------------	---	--------------	---	--------------	---	----------------	---	--------------------	---	----------------	------------------------	-----------------	---------------------------	--------------------	------------------------	---------------------	----	------------------------	---

Cluster Nodes Metrics

Active Nodes	4	Decommissioning Nodes	0	Decommissioned Nodes	0	Lost Nodes	0	Unhealthy Nodes	0	Rebooted Nodes	0	Shutdown Nodes	0
--------------	---	-----------------------	---	----------------------	---	------------	---	-----------------	---	----------------	---	----------------	---

Scheduler Metrics

Scheduler Type	Capacity Scheduler	Scheduling Resource Type	[memory-mb (unit=M), vcores]	Minimum Allocation	<memory:1024, vCores:1>	Maximum Allocation	<memory:8192, vCores:4>	Maximum Cluster Application Priority	0	Scheduler Busy %	0
----------------	--------------------	--------------------------	------------------------------	--------------------	-------------------------	--------------------	-------------------------	--------------------------------------	---	------------------	---

Show: 20 entries Search:

Node Labels	Rack	Node State	Node Address	Node HTTP Address	Last health-update	Health-report	Containers	Allocation Tags	Mem Used	Mem Avail	Phys Mem Used %	VCores Used	VCores Avail	Phys VCores Used %	Version
/default-rack		RUNNING	ngocphung-master:57762	ngocphung-master:8042	Thu May 16 17:45:16 +0700 2024		0		0 B	8 GB	91	0	8	3	3.3.6
/default-rack		RUNNING	192.168.56.1:5793	192.168.56.1:8042	Thu May 16 17:45:11 +0700 2024		0		0 B	8 GB	86	0	8	3	3.3.6
/default-rack		RUNNING	phuthanh-slave:55126	phuthanh-slave:8042	Thu May 16 17:43:26 +0700 2024		0		0 B	8 GB	78	0	8	1	3.3.6
/default-rack		RUNNING	phuthanh-slave:55414	phuthanh-slave:8042	Thu May 16 17:44:13 +0700 2024		0		0 B	8 GB	78	0	8	0	3.3.6

Showing 1 to 4 of 4 entries

First Previous **1** Next Last

CHƯƠNG 4: THIẾT KẾ VÀ CÀI ĐẶT

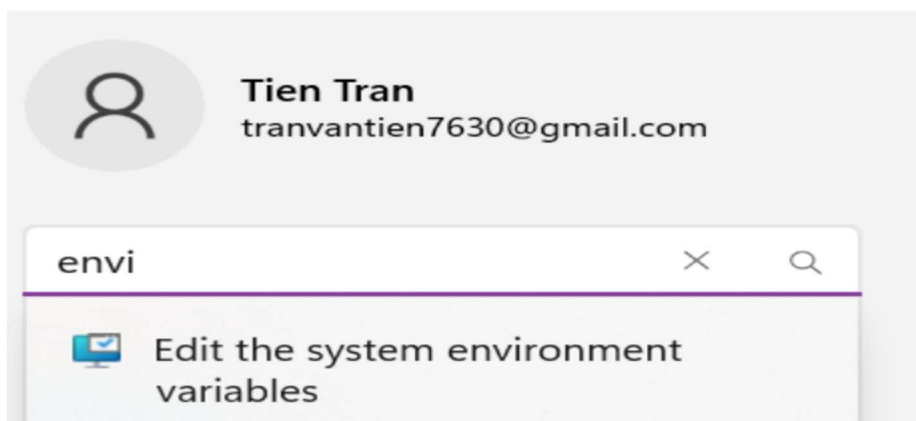
4.1 Cài đặt Apache Spark

4.1.1 Cài đặt JDK

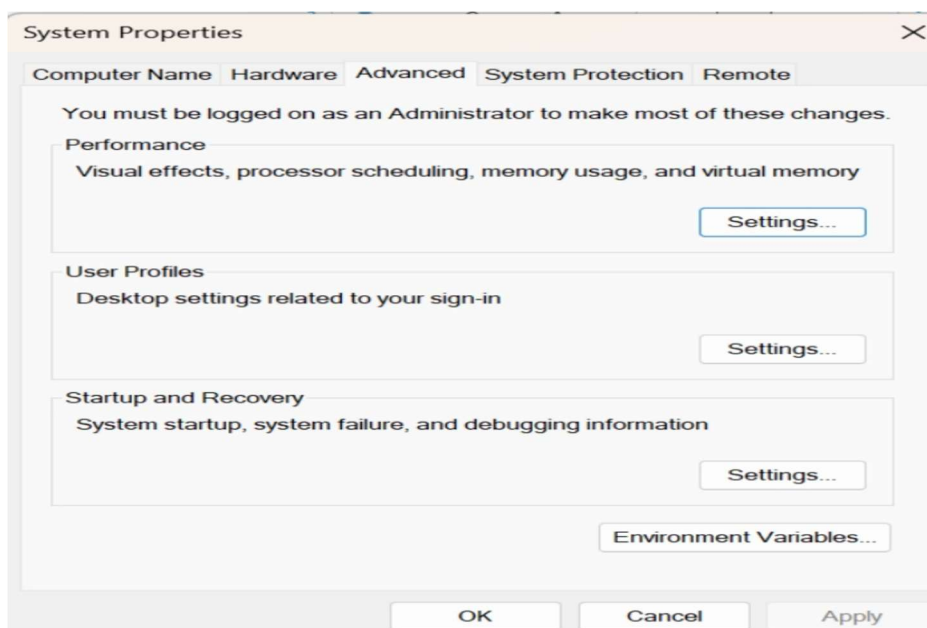
Spark sử dụng JDK 11, ta cài đặt các gói Java 11 về



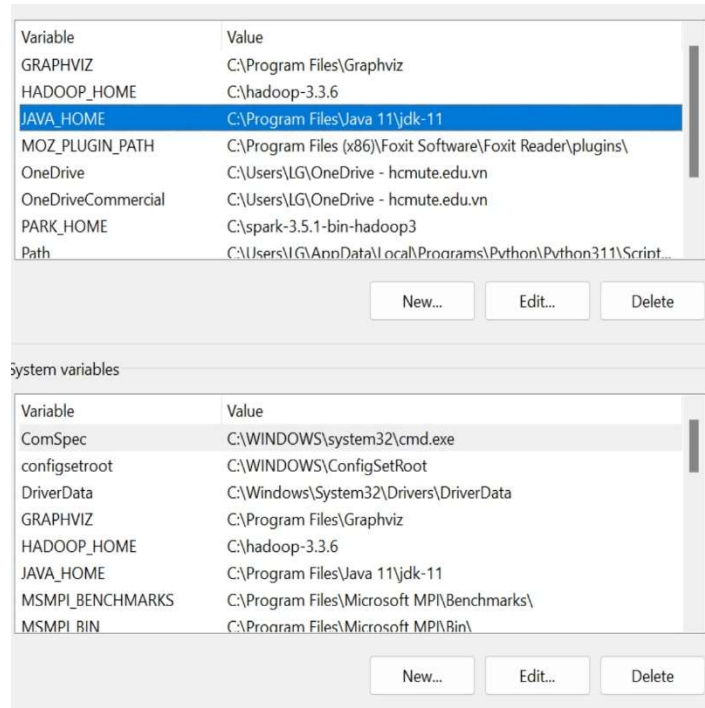
Tiếp theo, cài đặt các biến môi trường cho JDK 11. Có thể tìm ở thanh công cụ “Edit the system environment variables”



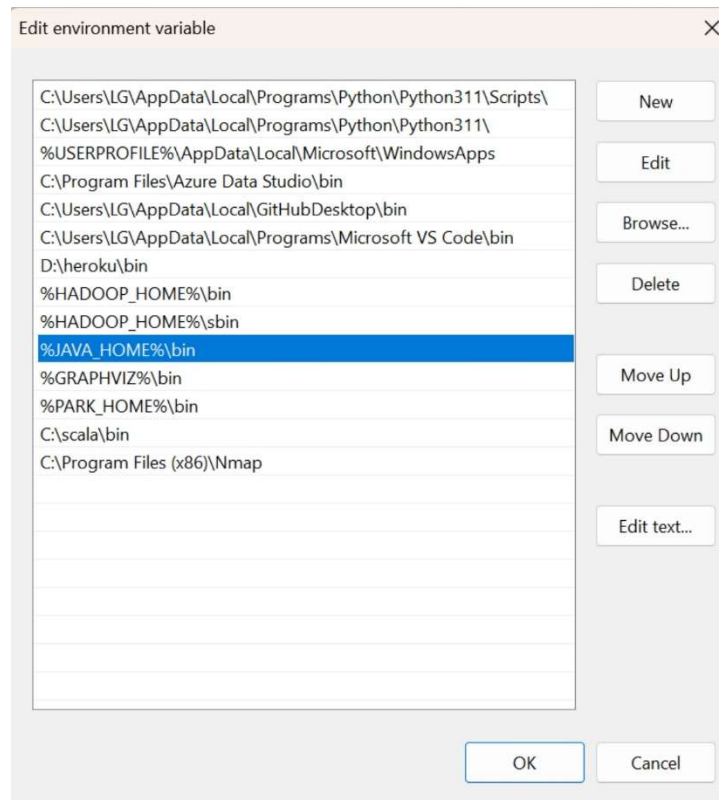
Chọn “Environment variables”



Thiết lập đường dẫn phù hợp cho JAVA_HOME



Thiết lập đường dẫn trong Path



Kiểm tra JDK đã cài

```
Using Scala version 2.12.18 (Java HotSpot(TM) 64-Bit Server VM, Java 11.0.19)
```

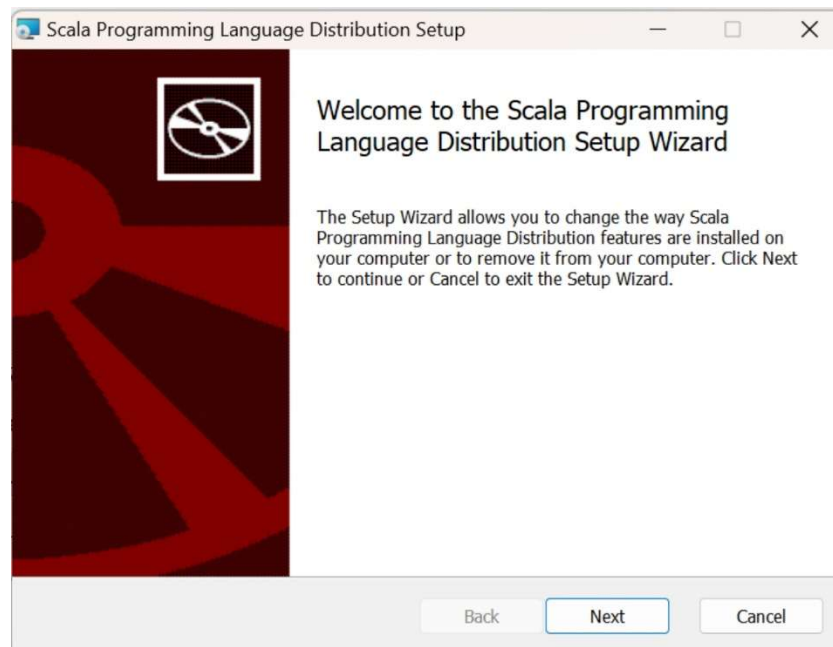
4.1.2 Cài đặt Scala

Sử dụng Scala 2.12.8

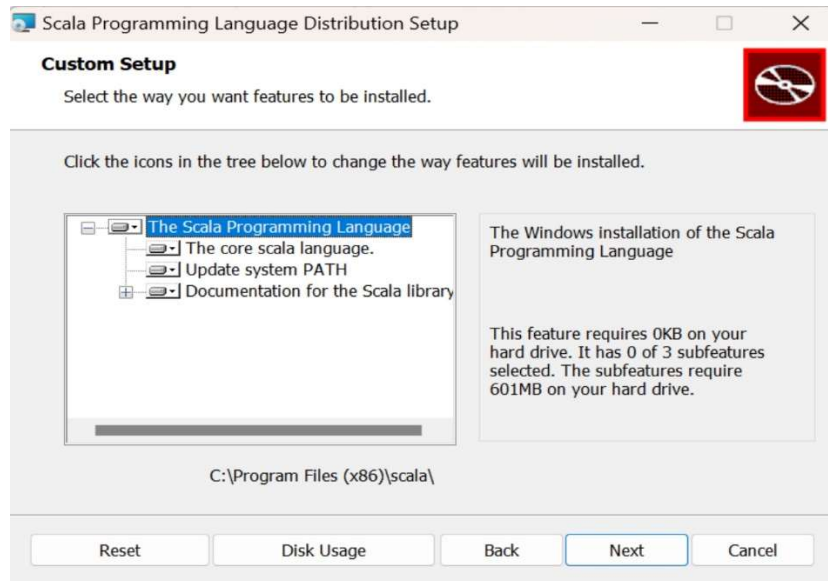
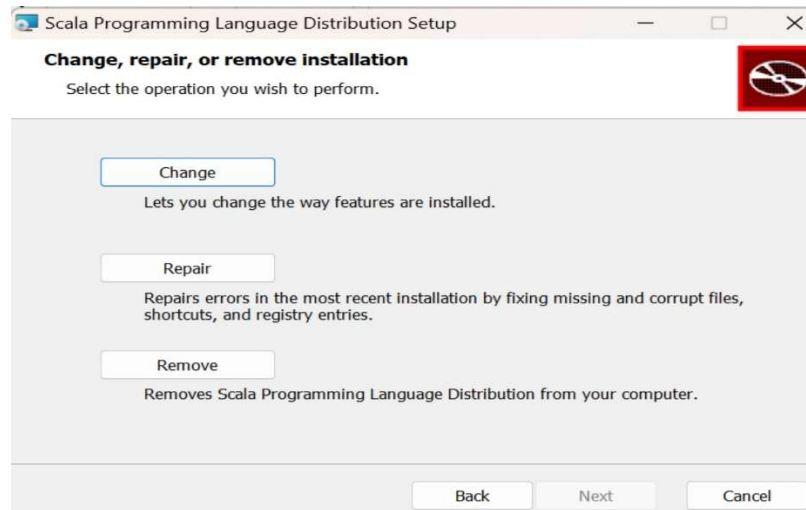
Ta vào link sau: <https://www.scala-lang.org/download/2.12.8.html>

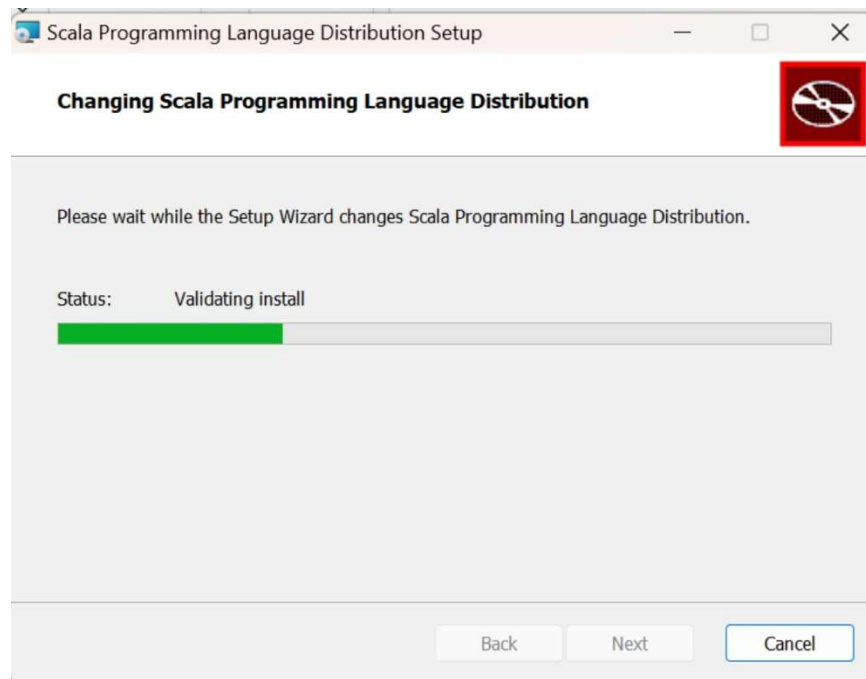
Sau khi tải về

scala-2.12.8 5/1/2024 4:54 PM Windows Installer Pa... 126,930 KB

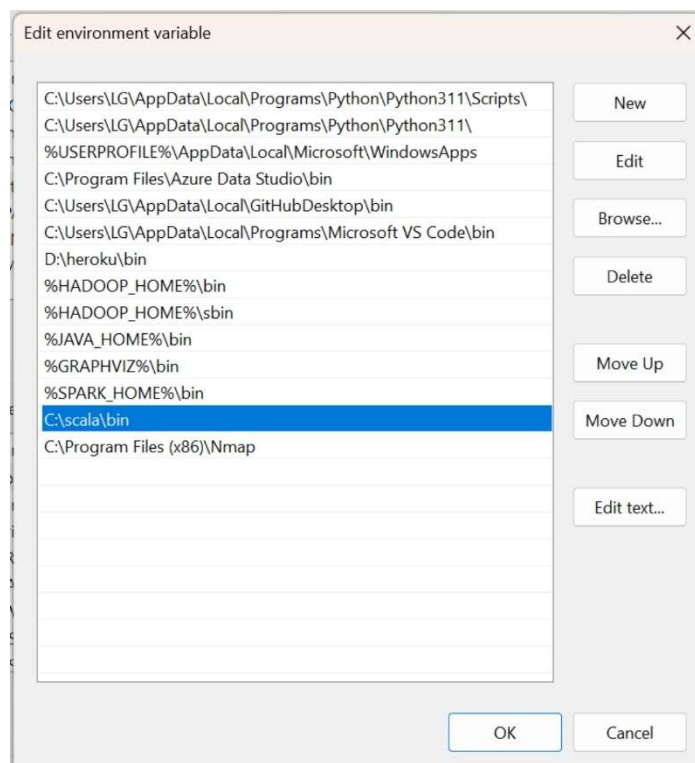


Nhấn Next

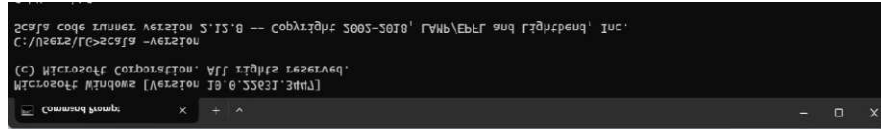




Tiếp theo, ta cài đặt biến môi trường cho Scala 2.12.8



Kiểm tra phiên bản Scala

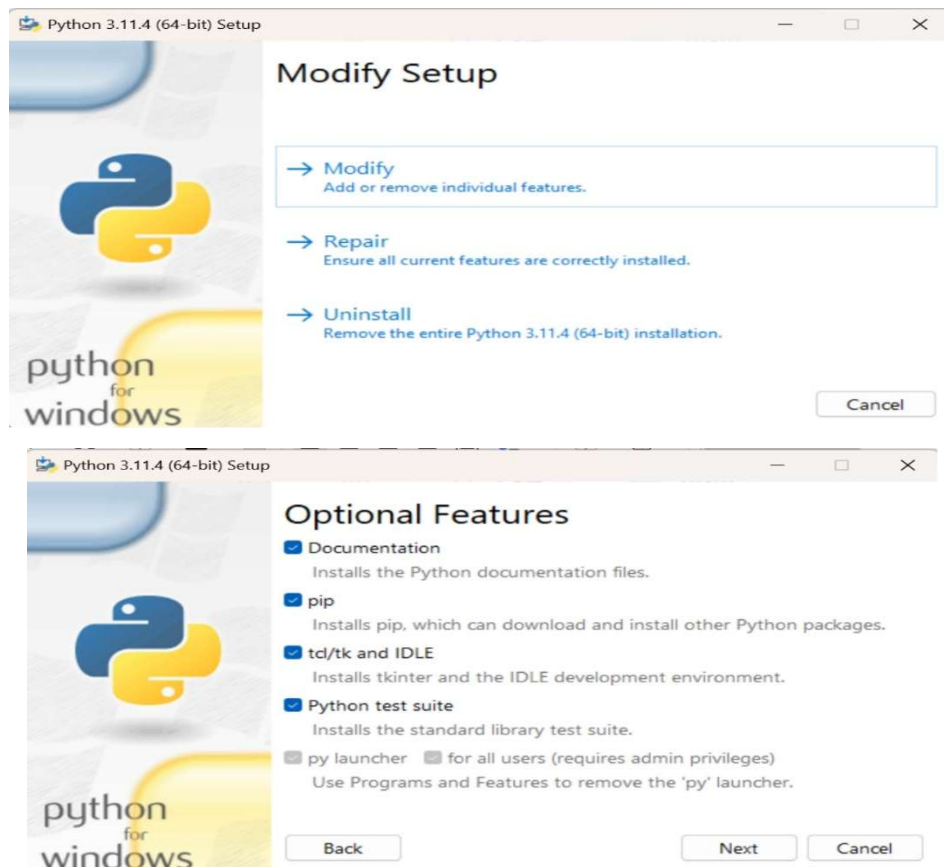


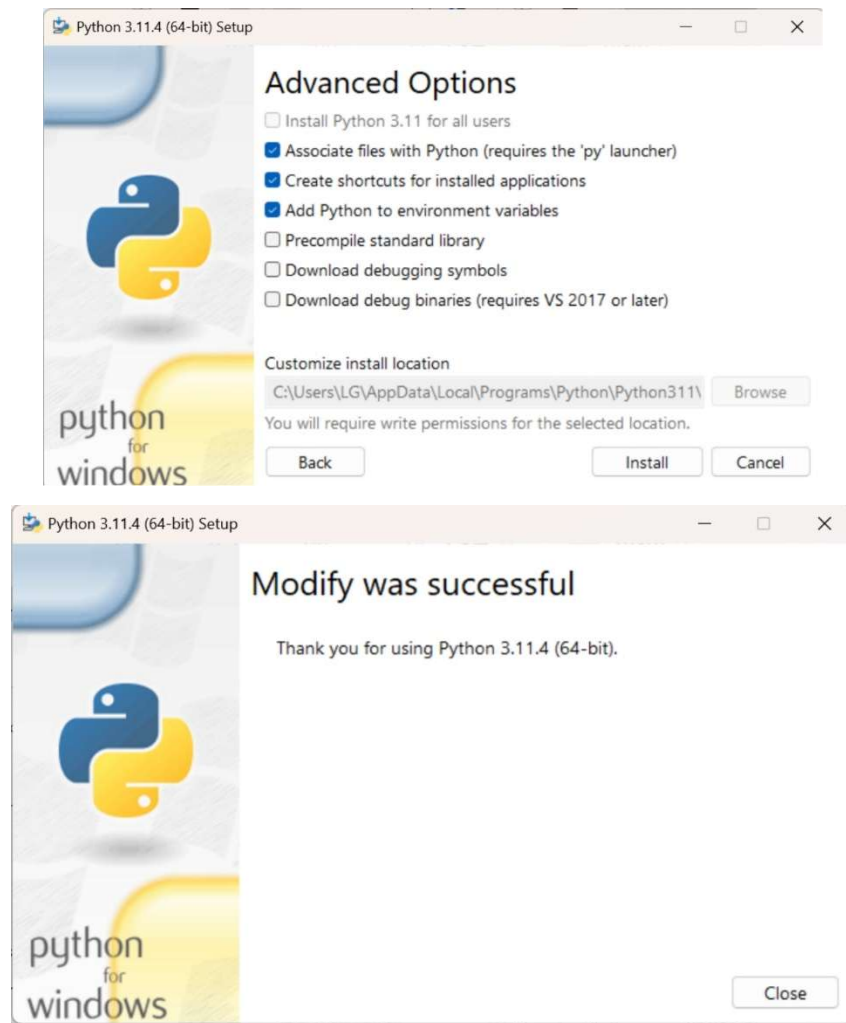
4.1.3 Cài đặt Python

Sử dụng Python 3.11.4

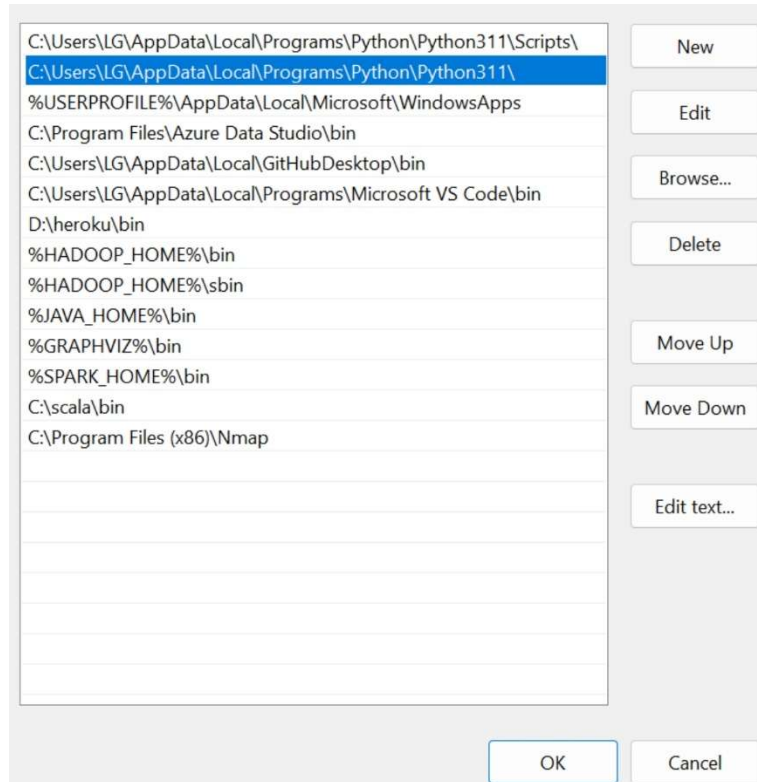
Ta vào link sau: <https://www.python.org/downloads/release/python-3114/>

Sau khi tải về, ta chạy tiến hành setup





Cài đặt các biến môi trường cho Python



Kiểm tra

```
C:\Users\LG>python
Python 3.11.4 (tags/v3.11.4:d2340ef, Jun 7 2023, 05:45:37) [MSC v.1934 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

4.1.4 Cài đặt Spark

Sử dụng Spark-3.5.1

Ta vào link sau: <https://www.apache.org/dyn/closer.lua/spark/spark-3.5.1/spark-3.5.1-bin-hadoop3.tgz>

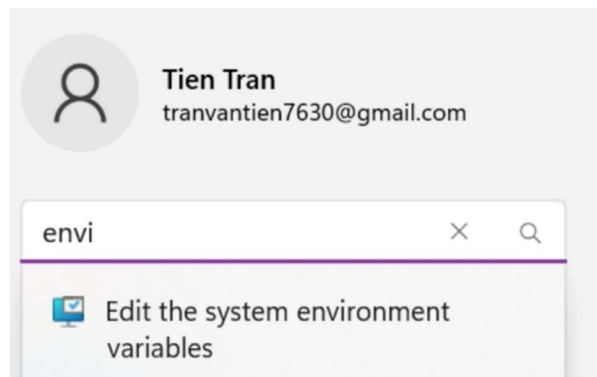
Sau khi tải về

 spark-3.5.1-bin-hadoop3	5/1/2024 3:24 PM	Compressed Archive ...	391,062 KB
---	------------------	------------------------	------------

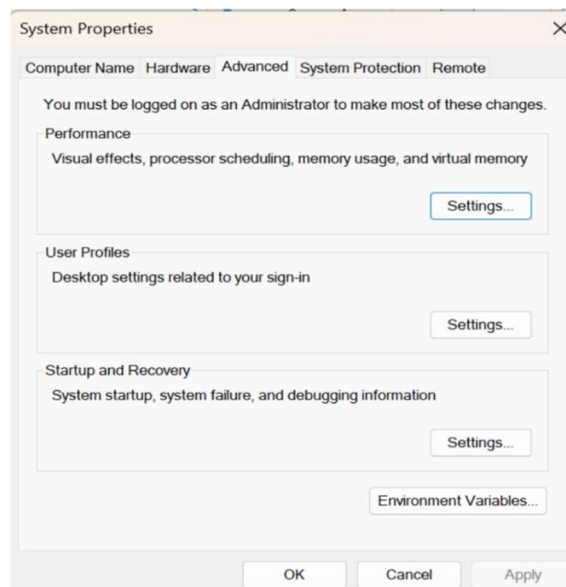
giải nén

 spark-3.5.1-bin-hadoop3	5/5/2024 3:39 PM	File folder
---	------------------	-------------

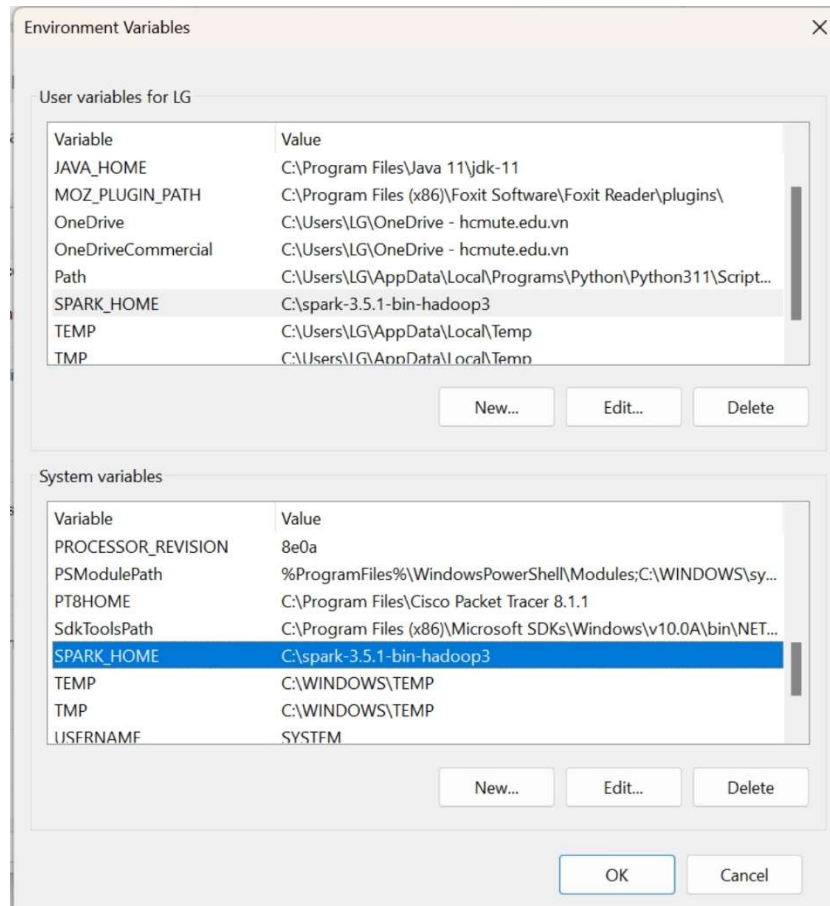
Ta cài đặt biến môi trường cho SPARK. Có thể tìm ở thanh công cụ “Edit the system environment variables”



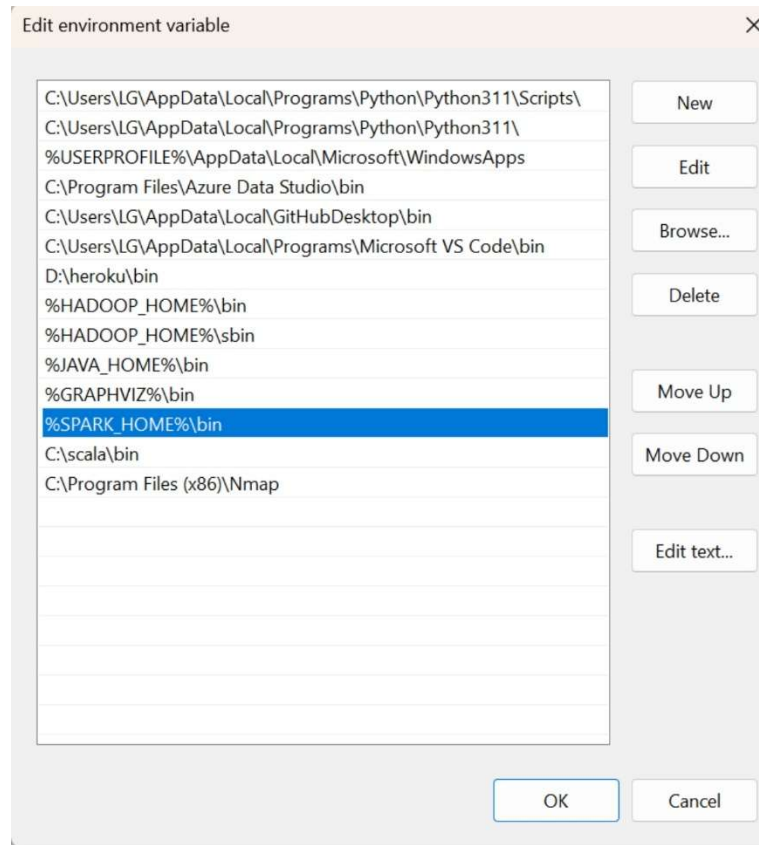
Chọn “Environment variables”



Dẫn đường dẫn phù hợp cho SPARK_HOME



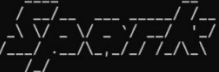
Thiết lập đường dẫn Path



4.1.5 Chạy thử Spark

```
Microsoft Windows [Version 10.0.22631.3447]
(c) Microsoft Corporation. All rights reserved.

C:\spark-3.5.1-bin-hadoop3>spark-shell
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
Spark context Web UI available at http://DESKTOP-JNUMA3T:4040
Spark context available as 'sc' (master = local[*], app id = local-1714973184710).
Spark session available as 'spark'.
Welcome to
```



```
version 3.5.1

Using Scala version 2.12.18 (Java HotSpot(TM) 64-Bit Server VM, Java 11.0.19)
Type in expressions to have them evaluated.
Type :help for more information.

scala>
```

Giao diện Spark shell: <http://desktop-jnuma3t:4040/>

The screenshot displays the Spark shell application UI with the following components:

- Spark Jobs**: Shows user information (User: LG, Total Uptime: 19 s, Scheduling Mode: FIFO) and an event timeline for executors and jobs.
- Stages for All Jobs**: A section for viewing job stages.
- Storage**: A section for viewing storage information.
- Environment**: A section for viewing runtime information and Spark properties.
- Executors**: A section for viewing executor information.

The **Environment** tab is currently selected, showing the following information:

Environment

▼ Runtime Information

Name	Value
Java Home	C:\Program Files\Java\jdk-11
Java Version	11.0.19 (Oracle Corporation)
Scala Version	version 2.12.18

▼ Spark Properties

Name	Value
spark.app.id	local-1714973566605
spark.app.name	Spark shell
spark.app.startTime	1714973563075
spark.app.submitTime	1714973549888
spark.driver.extraJavaOptions	-Djava.net.preferIPv6Addresses=false -XX:+IgnoreUnrecognizedVMOptions --add-opens=java.base/java.lang=ALL-UNNAMED --add-opens=java.base/java.lang.invoke=ALL-UNNAMED --add-opens=java.base/java.lang.reflect=ALL-UNNAMED --add-opens=java.base/java.io=ALL-UNNAMED --add-opens=java.base/java.net=ALL-UNNAMED --add-opens=java.base/java.nio=ALL-UNNAMED --add-opens=java.base/java.util=ALL-UNNAMED --add-opens=java.base/java.util.concurrent=ALL-UNNAMED

Stages for All Jobs

← → ↻ ⚠ Không bảo mật desktop-jnuma3t4040/storage/ 📄 ☆ 📁 👤 ⋮

SPARK 3.5.1 Jobs Stages **Storage** Environment Executors PySparkShell application UI

Storage



3.5.1

Jobs

Stages

Storage

Environment

Executors

PySparkShell application UI

Environment

▼ Runtime Information

Name	Value
Java Home	C:\Program Files\Java 11\jdk-11
Java Version	11.0.19 (Oracle Corporation)
Scala Version	version 2.12.18

▼ Spark Properties

Name	Value
spark.app.id	local-1714973343387
spark.app.name	PySparkShell
spark.app.startTime	1714973339953
spark.app.submitTime	1714973338598
spark.driver.extraJavaOptions	-Djava.net.preferIPv6Addresses=false -XX:+IgnoreUnrecognizedVMOptions --add-opens=java.base/java.lang=ALL-UNNAMED --add-opens=java.base/java.lang.invoke=ALL-UNNAMED --add-opens=java.base/java.lang.reflect=ALL-UNNAMED --add-opens=java.base/java.io=ALL-UNNAMED --add-opens=java.base/java.net=ALL-UNNAMED --add-

← → 🔍 Không bảo mật desktop-jnuma3t4040/executors/  3.5.1 Jobs Stages Storage Environment **Executors** PySparkShell application UI

Executors

▶ Show Additional Metrics

Summary

	▲ RDD Blocks ↕	Storage Memory ↕	Disk Used ↕	Cores ↕	Active Tasks ↕	Failed Tasks ↕	Complete Tasks ↕	Total Tasks ↕	Task Time (GC Time) ↕	Input ↕	Shuffle Read ↕	Shuffle Write ↕	Excluded ↕
Active(1)	0	0.0 B / 434.4 MiB	0.0 B	8	0	0	0	0	2.4 min (93.0 ms)	0.0 B	0.0 B	0.0 B	0
Dead(0)	0	0.0 B / 0.0 B	0.0 B	0	0	0	0	0	0.0 ms (0.0 ms)	0.0 B	0.0 B	0.0 B	0
Total(1)	0	0.0 B / 434.4 MiB	0.0 B	8	0	0	0	0	2.4 min (93.0 ms)	0.0 B	0.0 B	0.0 B	0

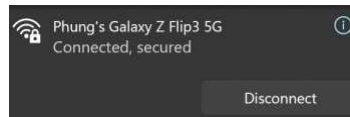
Executors

Show 20 entries

Search:

4.2 Cài đặt Spark Cluster

Bước 1: Connect chung 1 đường mạng (thực hiện trên master và workers)

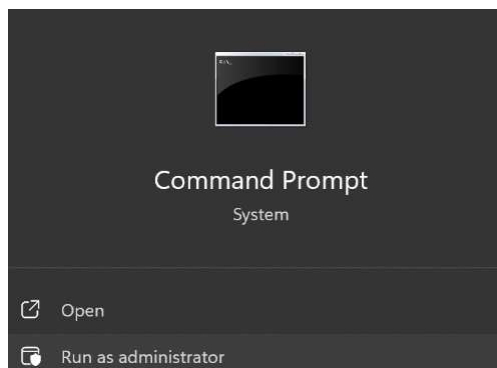


```
Wireless LAN adapter Wi-Fi:  
Connection-specific DNS Suffix . :  
Link-local IPv6 Address . . . . . : fe80::9ae8:402a:fdda:417b%18  
IPv4 Address. . . . . : 192.168.167.148  
Subnet Mask . . . . . : 255.255.255.0  
Default Gateway . . . . . : 192.168.167.12
```

```
Wireless LAN adapter Wi-Fi:  
Connection-specific DNS Suffix . :  
Link-local IPv6 Address . . . . . : fe80::7894:b6ed:6b57:e778%22  
IPv4 Address. . . . . : 192.168.167.201  
Subnet Mask . . . . . : 255.255.255.0  
Default Gateway . . . . . : 192.168.167.12
```

Bước 2: Đổi tên trên file host của máy master và các máy slave để dễ dàng quản lý (thực hiện trên master và workers)

Chạy command prompt dưới quyền admin



Mở file host ở địa chỉ C:\Windows\System32\drivers\etc\hosts

```
C:\> Administrator: Command Prompt  
Microsoft Windows [Version 10.0.22631.3447]  
(c) Microsoft Corporation. All rights reserved.  
C:\Windows\System32>notepad C:\Windows\System32\drivers\etc\hosts
```

Ghi nội dung chứa địa chỉ ip và tên máy mình muốn đặt

```
hosts
File Edit View
#
# This file contains the mappings of
names. Each
# entry should be kept on an individ
should
# be placed in the first column follo
host name.
# The IP address and the host name sh
least one
# space.
#
# Additionally, comments (such as the
individual
# lines or following the machine name
symbol.
#
# For example:
#
#      102.54.94.97      rhino.acme.co
server
#      38.25.63.10      x.acme.com
host

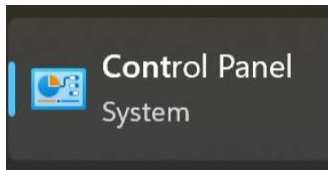
# localhost name resolution is handle
#      127.0.0.1      localhost
#      ::1      localhost
192.168.167.148 ngocphung-master
192.168.167.233 phuthanh-slave
192.168.167.201 trantien-slave
192.168.167.179 thuyquynh-slave
```

Việc này sẽ giúp ta dễ quản lý máy khi ping hay kiểm tra. Thay vì ping đến thẳng địa chỉ máy thì ta chỉ cần ping tên máy

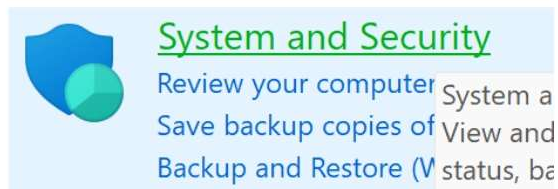
<pre>C:\Users\admin>ping 192.168.167.201 Pinging 192.168.167.201 with 32 bytes of data: Reply from 192.168.167.201: bytes=32 time=175ms TTL=128 Reply from 192.168.167.201: bytes=32 time=76ms TTL=128 Reply from 192.168.167.201: bytes=32 time=40ms TTL=128 Reply from 192.168.167.201: bytes=32 time=8ms TTL=128 Ping statistics for 192.168.167.201: Packets: Sent = 4, Received = 4, Lost = 0 (0% loss), Approximate round trip times in milli-seconds: Minimum = 8ms, Maximum = 175ms, Average = 74ms</pre>	<pre>C:\Users\admin>ping trantien-slave Pinging trantien-slave [192.168.167.201] with 32 bytes of data: Reply from 192.168.167.201: bytes=32 time=115ms TTL=128 Reply from 192.168.167.201: bytes=32 time=132ms TTL=128 Reply from 192.168.167.201: bytes=32 time=15ms TTL=128 Reply from 192.168.167.201: bytes=32 time=20ms TTL=128 Ping statistics for 192.168.167.201: Packets: Sent = 4, Received = 4, Lost = 0 (0% loss), Approximate round trip times in milli-seconds: Minimum = 15ms, Maximum = 132ms, Average = 70ms</pre>
--	---

Bước 3: Tắt tường lửa của máy master và các máy slave (thực hiện trên master và workers)

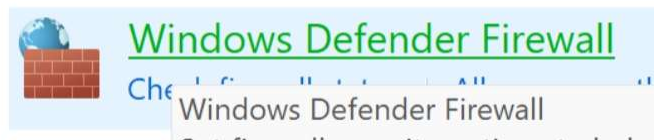
Mở control panel



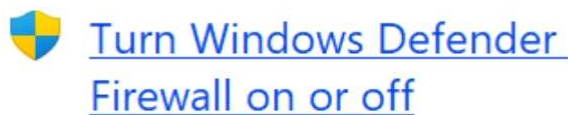
Chọn System and Security



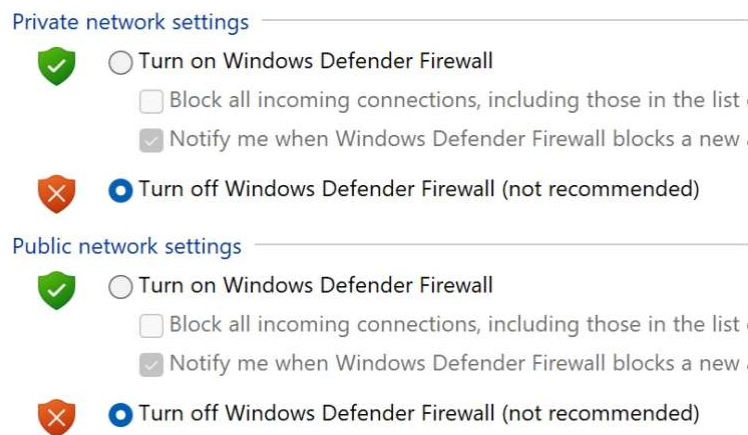
Chọn Windows Defender Firewall



Chọn Turn Windows Defender Firewall on or off



Turn off 2 mục và chọn OK



Bước 4: chạy lệnh deploy trên máy master (thực hiện trên máy master)

Di chuyển đến Spark_Home

- cd %SPARK_HOME%

```
W:\>cd %SPARK_HOME%
```

Khởi động máy master

bin\spark-class2.cmd org.apache.spark.deploy.master.Master

```
W:\Spark\spark-3.5.1-bin-hadoop3>bin\spark-class2.cmd org.apache.spark.deploy.master.Master
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
24/05/07 14:03:51 INFO Master: Started daemon with process name: 19268@NgocPhung
24/05/07 14:03:51 INFO SecurityManager: Changing view acls to: admin
24/05/07 14:03:51 INFO SecurityManager: Changing modify acls to: admin
24/05/07 14:03:51 INFO SecurityManager: Changing view acls groups to:
24/05/07 14:03:51 INFO SecurityManager: Changing modify acls groups to:
24/05/07 14:03:51 INFO SecurityManager: SecurityManager: authentication disabled; ui acls disabled; users with view perm
issions: admin; groups with view permissions: EMPTY; users with modify permissions: admin; groups with modify permission
s: EMPTY
24/05/07 14:03:52 INFO Utils: Successfully started service 'sparkMaster' on port 7077.
24/05/07 14:03:52 INFO Master: Starting Spark master at spark://192.168.167.148:7077
24/05/07 14:03:52 INFO Master: Running Spark version 3.5.1
24/05/07 14:03:52 INFO JettyUtils: Start Jetty 0.0.0.0:8080 for MasterUI
24/05/07 14:03:52 INFO Utils: Successfully started service 'MasterUI' on port 8080.
24/05/07 14:03:52 INFO MasterWebUI: Bound MasterWebUI to 0.0.0.0 and started at http://ngocphung-master:8080
24/05/07 14:03:52 INFO Master: I have been elected leader! New state: ALIVE
```

Mở trình duyệt vào cổng 8080 để xem giao diện UI của spark master

Spark Master at spark://192.168.167.148:7077

URL: spark://192.168.167.148:7077
Alive Workers: 0
Cores in use: 0 Total, 0 Used
Memory in use: 0.0 B Total, 0.0 B Used
Resources in use:
Applications: 0 Running, 0 Completed
Drivers: 0 Running, 0 Completed
Status: ALIVE

Workers (0)

Worker Id	Address	State	Cores	Memory	Resources
-----------	---------	-------	-------	--------	-----------

Running Applications (0)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
----------------	------	-------	---------------------	------------------------	----------------	------	-------	----------

Completed Applications (0)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
----------------	------	-------	---------------------	------------------------	----------------	------	-------	----------

Bước 5: chạy lệnh deploy trên máy slave (chỉ thực hiện trên máy slave)

Di chuyển đến Spark_Home

- cd %SPARK_HOME%

```
C:\Users\dinhq>cd %SPARK_HOME%
```

bin\spark-class org.apache.spark.deploy.worker.Worker

spark://192.168.167.148:7077


```
C:\spark-3.5.1-bin-hadoop3>.\bin\spark-class org.apache.spark.deploy.worker.Worker spark://192.168.167.148:7077
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
24/05/07 14:09:43 INFO Worker: Started daemon with process name: 16904@ASUS
24/05/07 14:09:44 INFO SecurityManager: Changing view acls to: dinhq
24/05/07 14:09:44 INFO SecurityManager: Changing modify acls to: dinhq
24/05/07 14:09:44 INFO SecurityManager: Changing view acls groups to:
24/05/07 14:09:44 INFO SecurityManager: Changing modify acls groups to:
24/05/07 14:09:44 INFO SecurityManager: SecurityManager: authentication disabled; ui acls disabled; users with view perm
issions: dinhq; groups with view permissions: EMPTY; users with modify permissions: dinhq; groups with modify permission
s: EMPTY
24/05/07 14:09:45 INFO Utils: Successfully started service 'sparkWorker' on port 42916.
24/05/07 14:09:45 INFO Worker: Worker decommissioning not enabled.
24/05/07 14:09:45 INFO Worker: Starting Spark worker 192.168.167.179:42916 with 4 cores, 6.7 GiB RAM
24/05/07 14:09:45 INFO Worker: Running Spark version 3.5.1
24/05/07 14:09:45 INFO Worker: Spark home: C:\spark-3.5.1-bin-hadoop3
24/05/07 14:09:45 INFO ResourceUtils: =====
24/05/07 14:09:45 INFO ResourceUtils: No custom resources configured for spark.worker.
24/05/07 14:09:45 INFO ResourceUtils: =====
24/05/07 14:09:45 INFO JettyUtils: Start Jetty 0.0.0.0:8081 for WorkerUI
24/05/07 14:09:46 INFO Utils: Successfully started service 'WorkerUI' on port 8081.
24/05/07 14:09:46 INFO WorkerWebUI: Bound WorkerWebUI to 0.0.0.0, and started at http://thuyquynh-slave:8081
24/05/07 14:09:46 INFO Worker: Connecting to master 192.168.167.148:7077...
24/05/07 14:09:46 INFO TransportClientFactory: Successfully created connection to /192.168.167.148:7077 after 407 ms (0
ms spent in bootstraps)
24/05/07 14:09:48 INFO Worker: Successfully registered with master spark://192.168.167.148:7077
```

Mở trình duyệt vào cổng 8081 để xem giao diện UI của spark slave

Spark Worker at 192.168.167.179:42916

ID: worker-20240507140945-192.168.167.179-42916
 Master URL: spark://192.168.167.148:7077
 Cores: 4 (0 Used)
 Memory: 6.7 GiB (0.0 B Used)
 Resources:

Back to Master

Running Executors (0)

ExecutorID	State	Cores	Memory	Resources
------------	-------	-------	--------	-----------

Bước 6: Kiểm tra các máy worker đã kết nối

Mở trình duyệt vào cổng 8081 để kiểm tra các workers

Spark Master at spark://192.168.167.148:7077

URL: spark://192.168.167.148:7077
 Alive Workers: 1
 Cores in use: 4 Total, 0 Used
 Memory in use: 6.7 GiB Total, 0.0 B Used
 Resources in use:
 Applications: 0 Running, 0 Completed
 Drivers: 0 Running, 0 Completed
 Status: ALIVE

Workers (1)

Worker Id	Address	State	Cores	Memory	Resources
worker-20240507140945-192.168.167.179-42916	192.168.167.179:42916	ALIVE	4 (0 Used)	6.7 GiB (0.0 B Used)	

4.3 Demo các bài Map Reduce

4.3.1 WordCount

```
import findspark
findspark.init()

from pyspark import SparkConf
from pyspark import SparkContext

conf = SparkConf()
#conf.setMaster('spark://192.168.1.91:7077')
conf.setMaster('local')
conf.setAppName('Comprehension')

# Khởi tạo SparkContext
sc = SparkContext(conf=conf)

# Đọc dữ liệu từ các tệp văn bản vào RDD
text_rdd = sc.wholeTextFiles("your_input_path/*")

# Define the tokenizer function
def tokenize(file_name, text):
    words = text.split()
    return [(word, file_name) for word in words]

# Tokenize each file
tokenized_rdd = text_rdd.flatMap(lambda file: tokenize(file[0], file[1]))

# Group by word
grouped_rdd = tokenized_rdd.groupByKey()
```


Concatenate file names

```
result_rdd = grouped_rdd.mapValues(lambda values: "|".join(values))
```

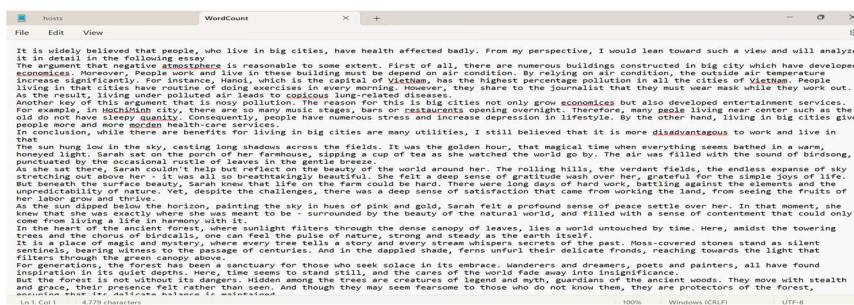
Lưu kết quả ra file văn bản

```
result_rdd.saveAsTextFile("your_output_path")
```

Dừng SparkContext

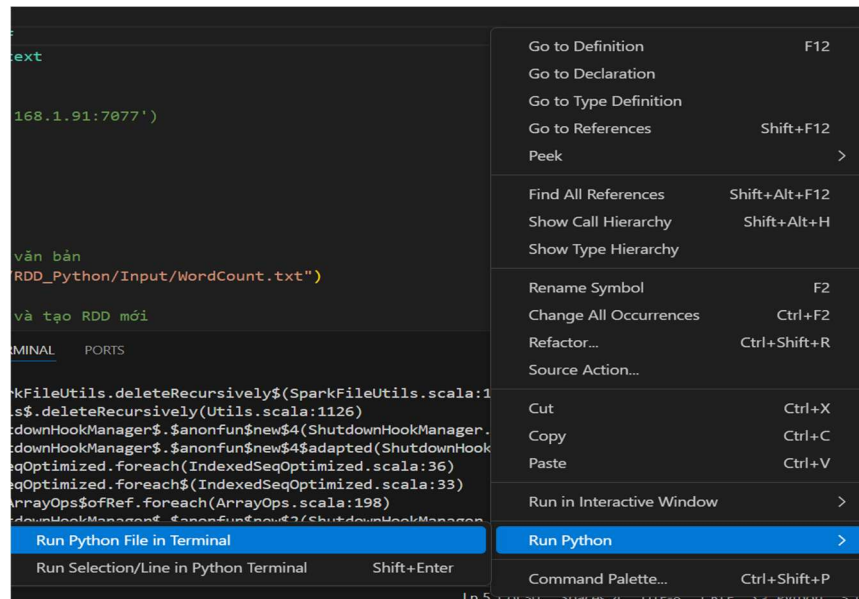
```
sc.stop()
```

Tập input:



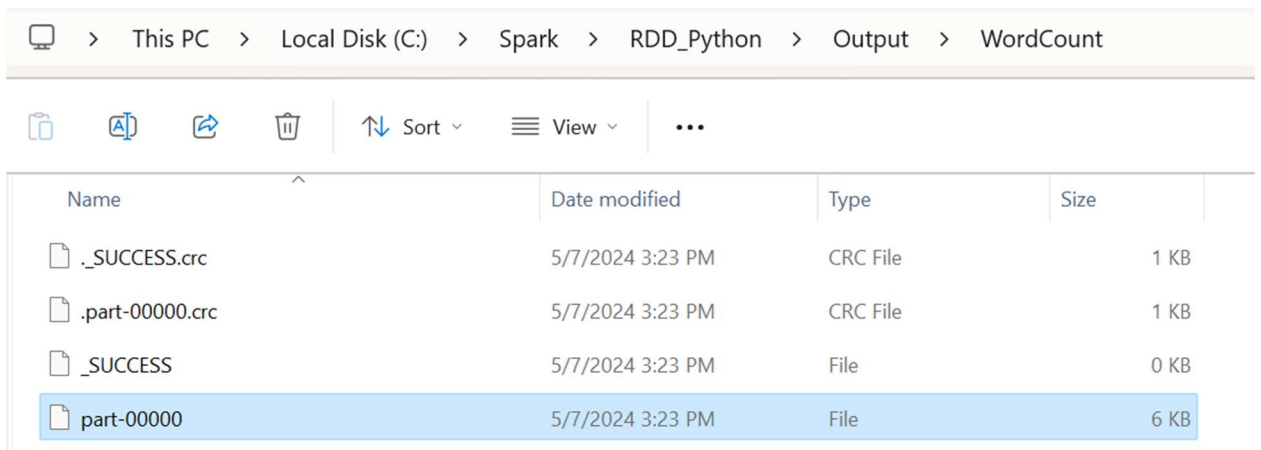
Mục tiêu của bài này là đếm số lần xuất hiện mỗi từ trong tập input đầu vào, ví dụ từ “It” xuất hiện bao nhiêu lần, từ “is” xuất hiện bao nhiêu lần,...

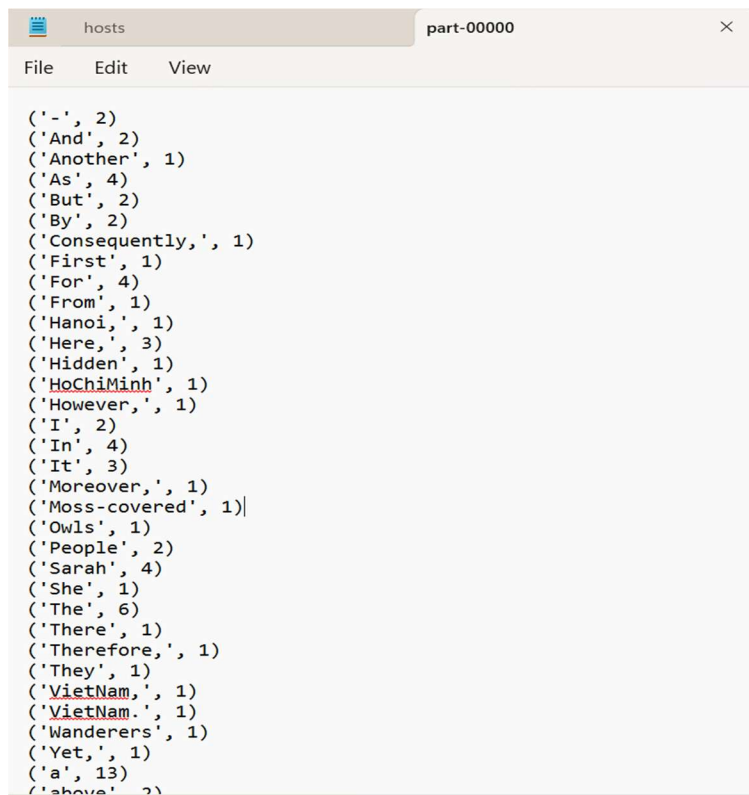
Ta nhân Run Python để chạy chương trình



```
SUCCESS: The process with PID 7424 (child process of PID 17812) has been terminated.
SUCCESS: The process with PID 17812 (child process of PID 14996) has been terminated.
SUCCESS: The process with PID 14996 (child process of PID 17708) has been terminated.
```

Output: Từ và số lần xuất hiện của từ đó





```
( '-', 2)
( 'And', 2)
( 'Another', 1)
( 'As', 4)
( 'But', 2)
( 'By', 2)
( 'Consequently', 1)
( 'First', 1)
( 'For', 4)
( 'From', 1)
( 'Hanoi', 1)
( 'Here', 3)
( 'Hidden', 1)
( 'HoChiMinh', 1)
( 'However', 1)
( 'I', 2)
( 'In', 4)
( 'It', 3)
( 'Moreover', 1)
( 'Moss-covered', 1)|
( 'Owls', 1)
( 'People', 2)
( 'Sarah', 4)
( 'She', 1)
( 'The', 6)
( 'There', 1)
( 'Therefore', 1)
( 'They', 1)
( 'VietNam', 1)
( 'VietNam.', 1)
( 'Wanderers', 1)
( 'Yet', 1)
( 'a', 13)
( 'above', 2)
```

3.3.3 WordLength

```
import findspark
findspark.init()

from pyspark import SparkConf
from pyspark import SparkContext

# Khởi tạo SparkConf
conf = SparkConf()
#conf.setMaster('spark://192.168.1.91:7077')
#conf.setMaster('local')
conf.setAppName("WordLength")

# Khởi tạo SparkContext
sc = SparkContext(conf=conf)

# Đọc dữ liệu từ tệp văn bản
text = sc.textFile("C:/Spark/RDD_Python/Input/WordLength.txt")

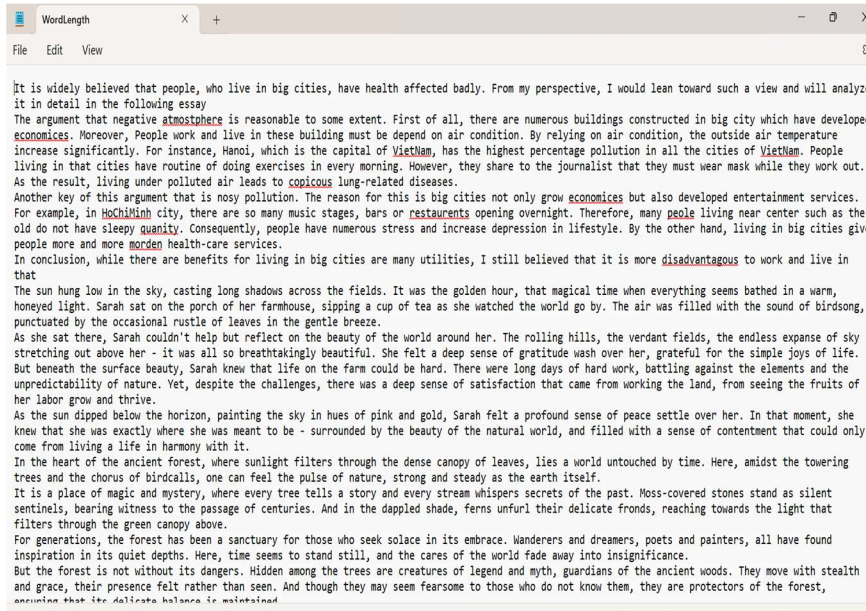
# Xử lý dữ liệu để tính độ dài của từ và phân loại nó
word_lengths = text.flatMap(lambda line: [(word, "tiny") if len(word) == 1 else
                                           (word, "small") if 1 < len(word) <= 4 else
                                           (word, "medium") if 5 <= len(word) <= 9 else
                                           (word, "big") for word in line.split()])

# Lọc ra các từ duy nhất
unique_word_lengths = word_lengths.distinct()

# Sắp xếp kết quả theo key
sorted_counts = unique_word_lengths.sortByKey()

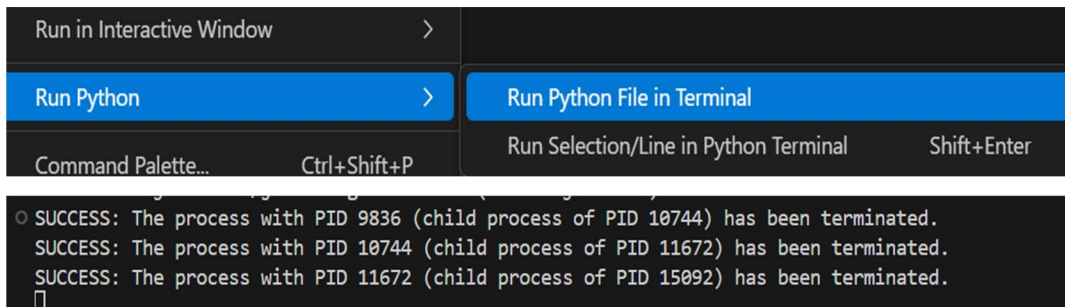
# Lưu kết quả vào file văn bản
sorted_counts.saveAsTextFile("C:/Spark/RDD_Python/Output/WordLength")
#output = sorted_counts.collect()
#for (word, category) in output:
#    print("%s: %s" % (word, category))
# Dừng SparkContext
sc.stop()
```

Tập input:



Mục tiêu của bài này là đếm số lần xuất hiện mỗi loại từ “small”, “tiny”, “medium”, “big” xuất hiện bao nhiêu lần

Các bước chạy:



Output

This PC > Local Disk (C:) > Spark > RDD_Python > Output > WordLength				
Sort View ...				
Name	Date modified	Type	Size	
._SUCCESS.crc	5/7/2024 3:10 PM	CRC File	1 KB	
.part-00000.crc	5/7/2024 3:10 PM	CRC File	1 KB	
.part-00001.crc	5/7/2024 3:10 PM	CRC File	1 KB	
._SUCCESS	5/7/2024 3:10 PM	File	0 KB	
part-00000	5/7/2024 3:10 PM	File	6 KB	
part-00001	5/7/2024 3:10 PM	File	4 KB	

WordLength part-00000

File Edit View

```
(('-', 'tiny')
('And', 'small')
('Another', 'medium')
('As', 'small')
('But', 'small')
('By', 'small')
('Consequently,', 'big')
('First', 'medium')
('For', 'small')
('From', 'small')
('Hanoi,', 'medium')
('Here,', 'medium')
('Hidden', 'medium')
('HoChiMinh', 'medium')
('However,', 'medium')
('I', 'tiny')
('In', 'small')
('It', 'small')
('Moreover,', 'medium')
('Moss-covered', 'big')
('Owls', 'small')
('People', 'medium')
('Sarah', 'medium')
('She', 'small')
('The', 'small')
('There', 'medium')
('Therefore,', 'big')
('They', 'small')
('VietNam', 'medium')
('VietNam.', 'medium')
('Wanderers', 'medium')
('Yet,', 'small')
('a', 'tiny')
('above', 'medium'))
```

4.3.3 FriendCount

```
import findspark
findspark.init()

from pyspark import SparkConf
from pyspark import SparkContext

conf = SparkConf()
#conf.setMaster('spark://192.168.1.91:7077')
conf.setMaster('local')
conf.setAppName('FriendCount')

# Khởi tạo SparkContext
sc = SparkContext(conf=conf)

# Đọc dữ liệu từ một tập tin văn bản chứa danh sách bạn bè
friend_list = sc.textFile("C:/Spark/RDD_Python/Input/FriendCount.txt")

# Chia các dòng thành các cặp bạn bè và tạo RDD mới
friend_pairs = friend_list.flatMap(lambda line: line.split(",")).map(lambda pair: (pair.split()[0], 1))

# Tính tổng số bạn của mỗi người
friend_counts = friend_pairs.reduceByKey(lambda a, b: a + b)

# Sắp xếp kết quả theo key
sorted_counts = friend_counts.sortByKey()

# Lưu kết quả ra file văn bản
sorted_counts.saveAsTextFile("C:/Spark/RDD_Python/Output/FriendCount")
output = sorted_counts.collect()
for (person, count) in output:
    print("%s: %i" % (person, count))

# Dừng SparkContext
sc.stop()
```

Về phần code, đầu tiên sẽ đọc dữ liệu từ file input và tạo ra RDD. Tiến hành quá trình map từng dòng của file input (dùng phương thức split để tách chuỗi thành từng phần tử riêng biệt với dấu phân cách là dấu “,”) thành từng cặp key-value với key là tên người bạn và value là 1. Tiếp tục thực hiện tiến trình Reduce để gom nhóm các value có cùng key lại và tạo ra một RDD mới. Cuối cùng sắp xếp các cặp key-value theo thứ tự chữ cái của key và lưu vào thư mục FriendCount đồng thời in ra màn hình

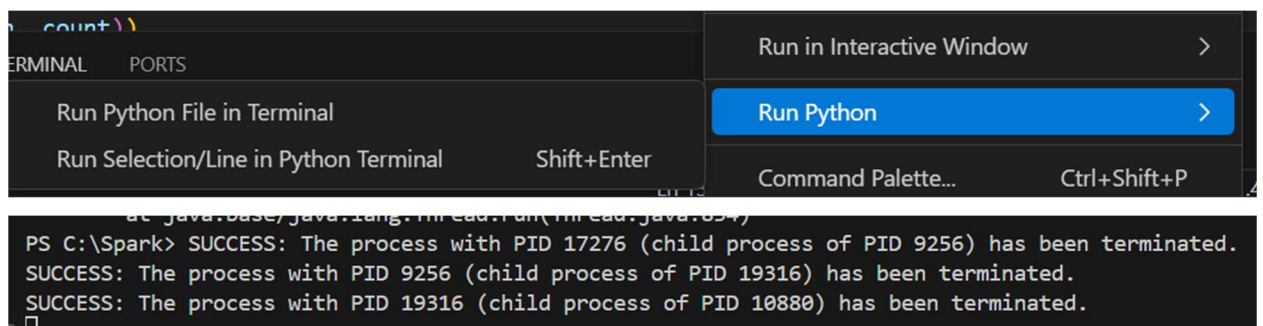
Tập input là file FriendCount.txt chứa thông tin về một người và bạn bè của người đó. Ví dụ: Dat là bạn của An, Cam là bạn của Dung, Hanh là bạn của Binh,



```
File Edit View
An, Dat
Dung, Cam
Binh, Hanh
Hanh, Cam
ZhangSan, LiSi
WangWu, ZhaoLiu
LiSi, WangWu
ZhouQi, ZhangSan
ZhaoLiu, ZhangSan
WangWu, ZhouQi
ZhangSan, ZhaoLiu
ZhouQi, LiSi
ChenBa, LiSi
LiuJiu, ZhouQi
SunShi, ZhangSan
ZhangSan, ChenBa
LiSi, LiuJiu
WangWu, ZhangSan
ZhangSan, LiuJiu
ZhouQi, ChenBa
ZhaoLiu, LiuJiu
ZhangSan, SunShi
LiSi, WangWu
ZhangSan, ZhaoLiu
WangWu, SunShi
ZhouQi, ZhaoLiu
LiuJiu, LiSi
ZhaoLiu, LiSi
SunShi, WangWu
ZhangSan, WangWu
ZhouQi, ZhaoLiu
LiuJiu, SunShi
LiSi, ZhangSan
ZhaoLiu, ZhangSan
```

Mục tiêu của bài này là đếm số lượng bạn bè của mỗi người, output sẽ in ra tên của một người và số lượng bạn bè của người đó là bao nhiêu

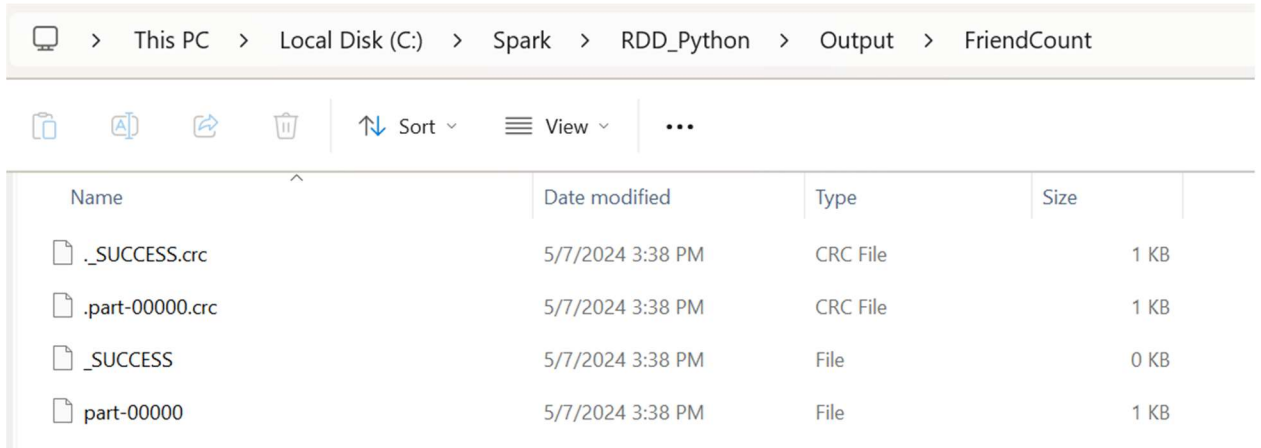
Ta nhấn chuột vào Run Python để chạy



```
count.py
TERMINAL PORTS
Run Python File in Terminal
Run Selection/Line in Python Terminal Shift+Enter
Run in Interactive Window
Run Python
Command Palette... Ctrl+Shift+P

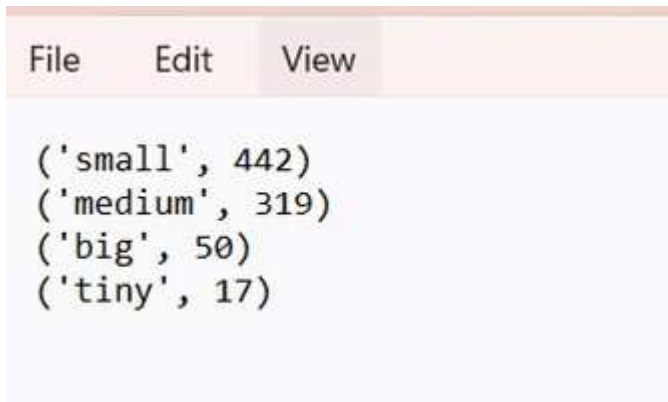
PS C:\Spark> SUCCESS: The process with PID 17276 (child process of PID 9256) has been terminated.
SUCCESS: The process with PID 9256 (child process of PID 19316) has been terminated.
SUCCESS: The process with PID 19316 (child process of PID 10880) has been terminated.
```


Output:



Name	Date modified	Type	Size
._SUCCESS.crc	5/7/2024 3:38 PM	CRC File	1 KB
.part-00000.crc	5/7/2024 3:38 PM	CRC File	1 KB
._SUCCESS	5/7/2024 3:38 PM	File	0 KB
part-00000	5/7/2024 3:38 PM	File	1 KB

Output sẽ hiện ra tên của người bạn và số lượng bạn bè của người đó. Ví dụ như Alice có 23 người bạn, An có 10 người bạn, Binh có 8 người bạn,...



```
('small', 442)
('medium', 319)
('big', 50)
('tiny', 17)
```

4.3.4 Comprehension

```
import findspark
findspark.init()

from pyspark import SparkConf
from pyspark import SparkContext
import os

conf = SparkConf()
#conf.setMaster('spark://192.168.1.91:7077')
conf.setMaster('local')
conf.setAppName('Comprehension')

# Khởi tạo SparkContext
sc = SparkContext(conf=conf)

# Đọc dữ liệu từ các tệp văn bản vào RDD
text_rdd = sc.wholeTextFiles("C:/Spark/RDD_Python/Input/Comprehension/*")

# Trích xuất tên tệp từ đường dẫn đầy đủ
def extract_file_name(file_path):
    return os.path.basename(file_path)

# Define the tokenizer function
def tokenize(file_name, text):
    words = text.split()
    return [(word, file_name) for word in set(words)] # Sử dụng set để chỉ gán một lần cho mỗi từ

# Tokenize each file
tokenized_rdd = text_rdd.flatMap(lambda file: tokenize(extract_file_name(file[0]), file[1]))

# Group by word
grouped_rdd = tokenized_rdd.groupByKey()

# Concatenate file names
result_rdd = grouped_rdd.mapValues(lambda values: "|".join(values))

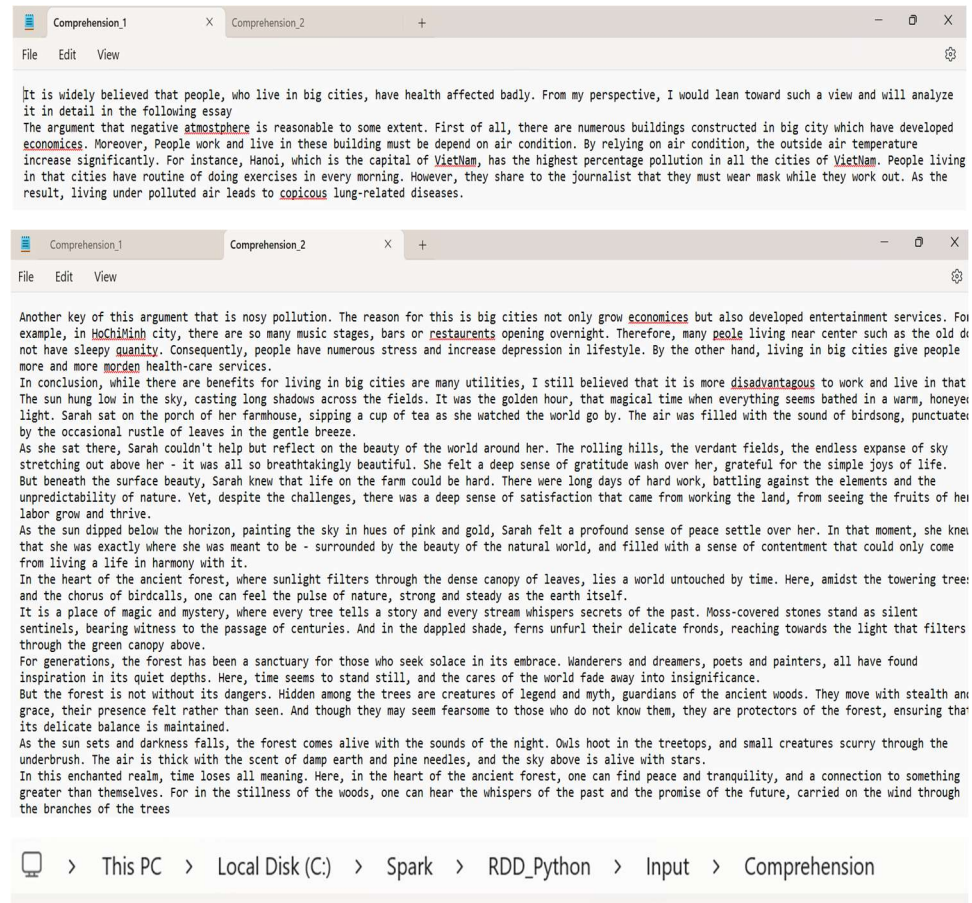
#
sorted_rdd = result_rdd.sortByKey()

# Lưu kết quả ra file văn bản
sorted_rdd.saveAsTextFile("C:/Spark/RDD_Python/Output/Comprehension")

# In ra kết quả trên terminal
output = sorted_rdd.collect()
for (word, files) in output:
    print(f"{word}: {files}")

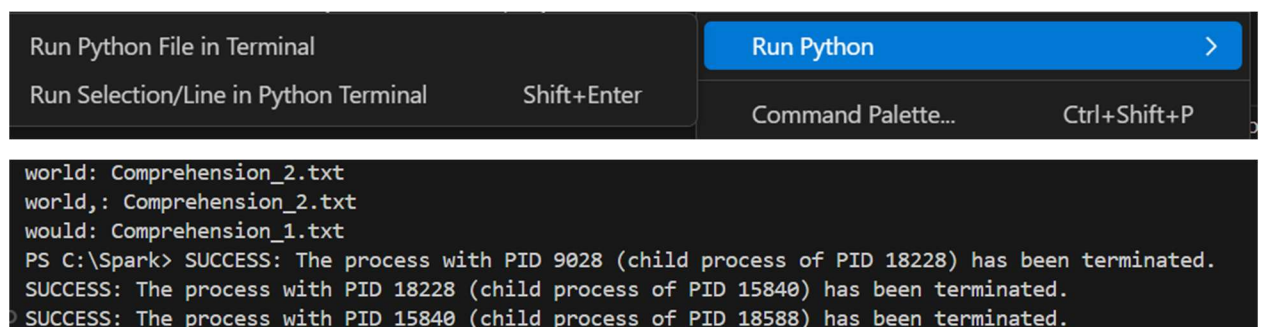
# Dừng SparkContext
sc.stop()
```

Tập input là thư mục Comprehension chứa 2 file txt là Comprehension1.txt và Comprehension2.txt

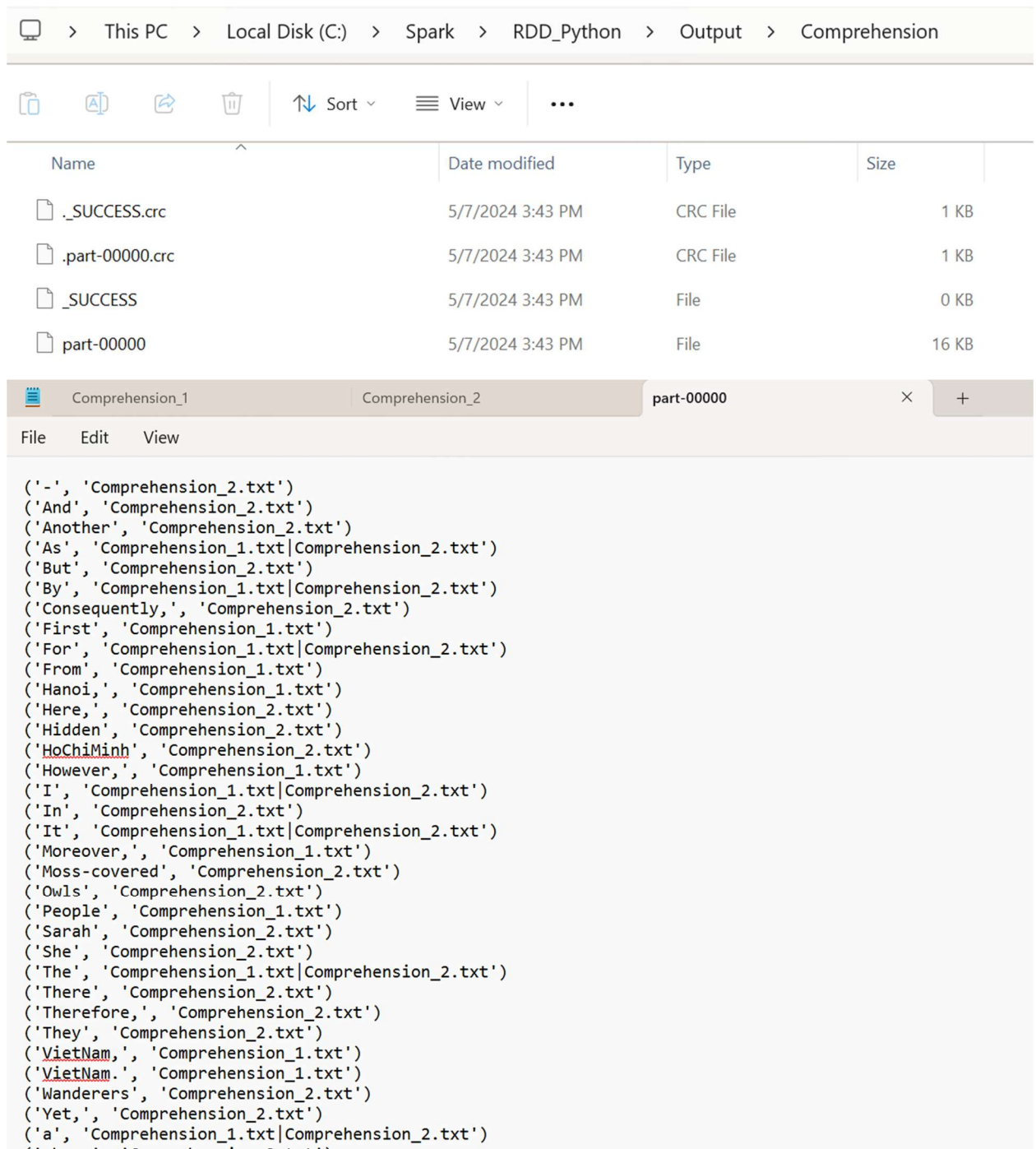


Mục đích của bài này là cho biết từ đó thuộc file nào

Các bước chạy:



Output: Hiện ta từ đó và file mà có chứa từ đó, ví dụ từ “And” thuộc
Comprehension2.txt



4.4 Demo RDD Actions

```
package org.example.RDD
import org.apache.spark.sql.Session
import scala.collection.mutable
```

```

object RDDActions extends App{

  // Create RDD from a List Using parallelize
  val spark = SparkSession.builder()
    .appName("SparkByExample")
    .master("local")
    .getOrCreate()

  //Khoi tao 2 RDD
  spark.sparkContext.setLogLevel("ERROR")
  val inputRDD = spark.sparkContext.parallelize(List(("Z", 1),("A", 20),("B", 30),("C", 40),("B",
30),("B", 60)))
  inputRDD.foreach(println)
  val listRdd = spark.sparkContext.parallelize(List(1,7,3,4,5,3,2))
  listRdd.foreach(println)


  // aggregate() the elements of each partition, and then the results for all the partitions.
  //aggregate
  private def param0 = (accu: Int, v: Int) => accu + v
  private def param1 = (accu1: Int, accu2: Int) => accu1 + accu2
  println("aggregate listRdd : " + listRdd.aggregate(0)(param0, param1))
  //Output: aggregate : 20


  //aggregate
  private def param3 = (accu: Int, v: (String, Int)) => accu + v._2
  private def param4 = (accu1: Int, accu2: Int) => accu1 + accu2
  println("aggregate inputRdd: " + inputRDD.aggregate(0)(param3, param4))
  //Output: aggregate : 181


  //treeAggregate() – Aggregates the elements of this RDD in a multi-level tree pattern.
  // The output of this function will be similar to the aggregate function.
  //treeAggregate. This is similar to aggregate
  private def param8 = (accu: Int, v: Int) => accu + v
  private def param9 = (accu1: Int, accu2: Int) => accu1 + accu2
  println("treeAggregate : " + listRdd.treeAggregate(0)(param8, param9))
  //Output: treeAggregate : 20


  //fold() – Aggregate the elements of each partition, and then the results for all the partitions.
  //fold
  println("fold : " + listRdd.fold(0) { (acc, v) =>
    val sum = acc + v
    sum
  })
  //Output: fold : 20


  println("fold : " + inputRDD.fold(("Total", 0)) { (acc: (String, Int), v: (String, Int)) =>
    val sum = acc._2 + v._2
    ("Total", sum)
  })
  //Output: fold : (Total,181)
}

```

```

//reduce() – Reduces the elements of the dataset using the specified binary operator.
//reduce
println("reduce : " + listRdd.reduce(_ + _))
//Output: reduce : 20
println("reduce alternate : " + listRdd.reduce((x, y) => x + y))
//Output: reduce alternate : 20
println("reduce : " + inputRDD.reduce((x, y) => ("Total", x._2 + y._2)))
//Output: reduce : (Total,181)

//treeReduce() – Reduces the elements of this RDD in a multi-level tree pattern.
//treeReduce. This is similar to reduce
println("treeReduce : " + listRdd.treeReduce(_ + _))
//Output: treeReduce : 20

//collect() -Return the complete dataset as an Array.
//Collect
val data: Array[Int] = listRdd.collect()
data.foreach(println)

/*
count() – Return the count of elements in the dataset.
countApprox() – Return approximate count of elements in the dataset, this method returns incomplete
when execution time meets timeout.
countApproxDistinct() – Return an approximate number of distinct elements in the dataset.
*/
//count, countApprox, countApproxDistinct
println("Count : " + listRdd.count)
//Output: Count : 7
println("countApprox : " + listRdd.countApprox(1200))
//Output: countApprox : (final: [7.000, 7.000])
println("countApproxDistinct : " + listRdd.countApproxDistinct())
//Output: countApproxDistinct : 5
println("countApproxDistinct : " + inputRDD.countApproxDistinct())
//Output: countApproxDistinct : 5

/*
countByValue() – Return Map[T,Long] key representing each unique value in dataset and value
represents count each value present.
countByValueApprox() – Same as countByValue() but returns approximate result.
*/
//countByValue, countByValueApprox
println("countByValue : " + listRdd.countByValue())
//Output: countByValue : Map(5 -> 1, 1 -> 1, 2 -> 2, 3 -> 2, 4 -> 1)
//println(listRdd.countByValueApprox())

//first
println("first : " + listRdd.first())
//Output: first : 1
println("first : " + inputRDD.first())
//Output: first : (Z,1)

//top

```

```

println("top : " + listRdd.top(2).mkString(", "))
//Output: take : 5,4
println("top : " + inputRDD.top(2).mkString(", "))
//Output: take : (Z,1),(C,40)

//min
println("min : " + listRdd.min())
//Output: min : 1
println("min : " + inputRDD.min())
//Output: min : (A,20)

//max
println("max : " + listRdd.max())
//Output: max : 5
println("max : " + inputRDD.max())
//Output: max : (Z,1) */

/*
take() – Return the first num elements of the dataset.
takeOrdered() – Return the first num (smallest) elements from the dataset and this is the opposite of
the take() action.
*/

//take, takeOrdered, takeSample
println("take : " + listRdd.take(2).mkString(", "))
//Output: take : 1,7
println("takeOrdered : " + listRdd.takeOrdered(2).mkString(", "))
//Output: takeOrdered : 1,2
}

```

Mục đích minh họa các hành động (actions) có sẵn trong Apache Spark khi làm việc với RDD (Resilient Distributed Dataset). Mục đích chính của đoạn mã là:

1. Khởi tạo môi trường Spark:

- Tạo SparkSession để tương tác với Spark.
- Thiết lập mức độ log để chỉ hiển thị các lỗi.

2. Tạo RDD:

- Tạo hai RDD từ danh sách:
 - inputRDD từ danh sách các cặp giá trị.
 - listRdd từ danh sách các số nguyên.

3. Thực hiện các hành động trên RDD:

- **aggregate():** Tính toán tổng các phần tử trong RDD bằng cách sử dụng hàm tổng hợp tùy chỉnh.
- **treeAggregate():** Tương tự aggregate nhưng sử dụng cấu trúc cây để tối ưu hóa việc tính toán trên nhiều partition.
- **fold():** Tính toán tổng các phần tử trong mỗi partition và sau đó tổng hợp kết quả từ các partition.
- **reduce():** Giảm các phần tử của dataset bằng cách sử dụng toán tử nhị phân đã xác định.
- **treeReduce():** Tương tự reduce nhưng sử dụng cấu trúc cây.
- **collect():** Trả về toàn bộ dataset dưới dạng mảng.
- **count(), countApprox(), countApproxDistinct():** Đếm số lượng phần tử, đếm xấp xỉ và đếm số phần tử khác nhau xấp xỉ.
- **countByValue():** Trả về bản đồ đếm số lần xuất hiện của mỗi giá trị trong dataset.
- **first():** Lấy phần tử đầu tiên của dataset.
- **top():** Lấy các phần tử có giá trị cao nhất.
- **min(), max():** Tìm giá trị nhỏ nhất và lớn nhất trong dataset.
- **take(), takeOrdered(), takeSample():** Lấy một số phần tử đầu tiên, phần tử nhỏ nhất và mẫu ngẫu nhiên từ dataset.

Các hành động này giúp người dùng hiểu rõ hơn về cách thao tác và xử lý dữ liệu

lớn bằng Spark, thông qua các phép tính tổng hợp, lấy mẫu, và đếm phần tử trong RDD.

Kết quả sau khi chạy chương trình

The screenshot shows an IDE with a Scala file named `RDDActions.scala`. The code defines an input RDD with 7 partitions, each containing a string and an integer. It then creates a list RDD with 7 partitions, each containing a single integer. The output of the program is displayed in the console, showing the contents of the RDDs and the results of various Spark actions.

```
//Khoi tao 2 RDD
spark.sparkContext.setLogLevel("ERROR")
val inputRDD = spark.sparkContext.parallelize(List(("Z", 1), ("A", 20), ("B", 30), ("C", 40), ("B", 30), ("B", 60)))
inputRDD.foreach(println)
val listRdd = spark.sparkContext.parallelize(List(1,7,3,4,5,3,2))
println("list RDD: ")
listRdd.foreach(println)
```

Run RDDActions

```
SLF4J: See http://www.slf4j.org/codes.html#no_static_mdc_binder for further details.
(Z,1)
(A,20)
(B,30)
(C,40)
(B,30)
(B,60)
list RDD:
1
7
3
4
5
3
2
aggregate listRdd : 25
aggregate inputRdd : 181
treeAggregate : 25
fold : 25
fold : (Total,181)
reduce : 25
reduce alternate : 25
reduce : (Total,181)
treeReduce : 25

list RDD by using Collect:
1
7
3
4
5
3
2
Count : 7
countApprox : (final: [7.000, 7.000])
countApproxDistinct : 6
countApproxDistinct : 5
first : 1
first : (Z,1)
top : 7,5
top : (Z,1),(C,40)
min : 1
min : (A,20)
max : 7
max : (Z,1)
take : 1,7
takeOrdered : 1,2

Process finished with exit code 0
```

Một ví dụ khác về RDD Repartition và Coalesce

```
package org.example.RDD

import org.apache.spark.sql.Session

object RDDRepartition extends App {

  val spark:Session = Session.builder()
    .master("local[5]")
    .appName("SparkByExamples.com")
    .getOrCreate()

  spark.sparkContext.setLogLevel("ERROR")

  val rdd = spark.sparkContext.parallelize(Range(0,20))
  println("From local[5] : "+rdd.partitions.size)

  val rdd1 = spark.sparkContext.parallelize(Range(0,20), 6)
  println("parallelize : "+rdd1.partitions.size)

  rdd1.partitions.foreach(f=> f.toString)
  val rddFromFile =
  spark.sparkContext.textFile("E:/TAI_LIEU_DAI_HOC/SEMESTER_6/BigData/Demo/src/main/scala/
  org/example/data/test.txt",10)

  println("TextFile : "+rddFromFile.partitions.size)

  rdd1.saveAsTextFile("E:/TAI_LIEU_DAI_HOC/SEMESTER_6/BigData/Demo/src/main/scala/org/exa
  mple/output/partition")
  val rdd2 = rdd1.repartition(4)
  println("Repartition size : "+rdd2.partitions.size)

  rdd2.saveAsTextFile("E:/TAI_LIEU_DAI_HOC/SEMESTER_6/BigData/Demo/src/main/scala/org/exa
  mple/output/re-partition")

  val rdd3 = rdd1.coalesce(4)
  println("Repartition size : "+rdd3.partitions.size)

  rdd3.saveAsTextFile("E:/TAI_LIEU_DAI_HOC/SEMESTER_6/BigData/Demo/src/main/scala/org/exa
  mple/output/coalesce")

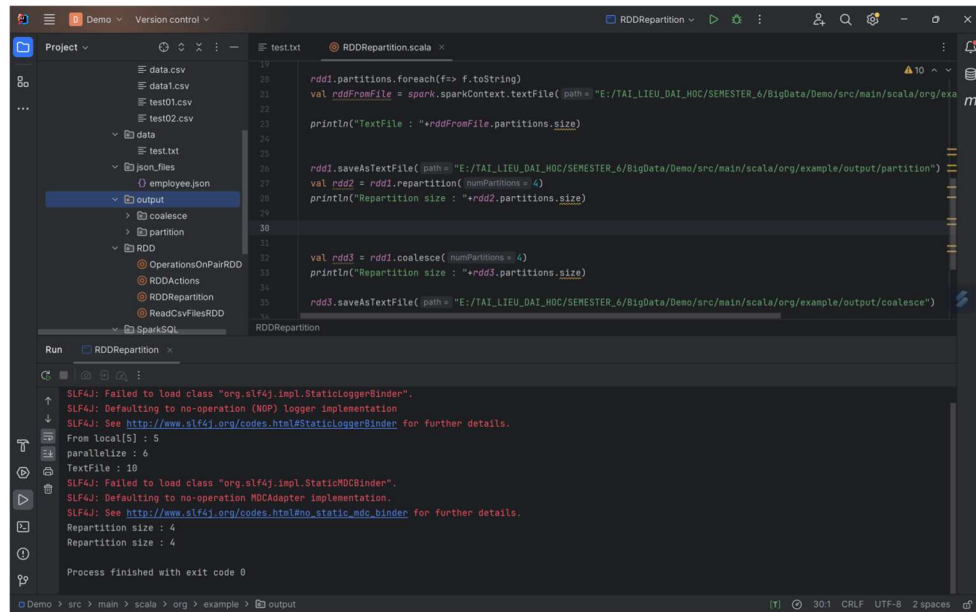
  spark.stop()
}
```

Mục đích để phân biệt

- **repartition():** Tạo một RDD mới với số lượng partition thay đổi, thường được sử dụng để tăng hoặc giảm số lượng partition nhằm tối ưu hóa việc phân phối dữ liệu.
- **coalesce():** Tạo một RDD mới với số lượng partition giảm, thường được sử dụng

đề hợp nhất partition nhằm giảm chi phí tính toán.

Kết quả sau khi chạy chương trình



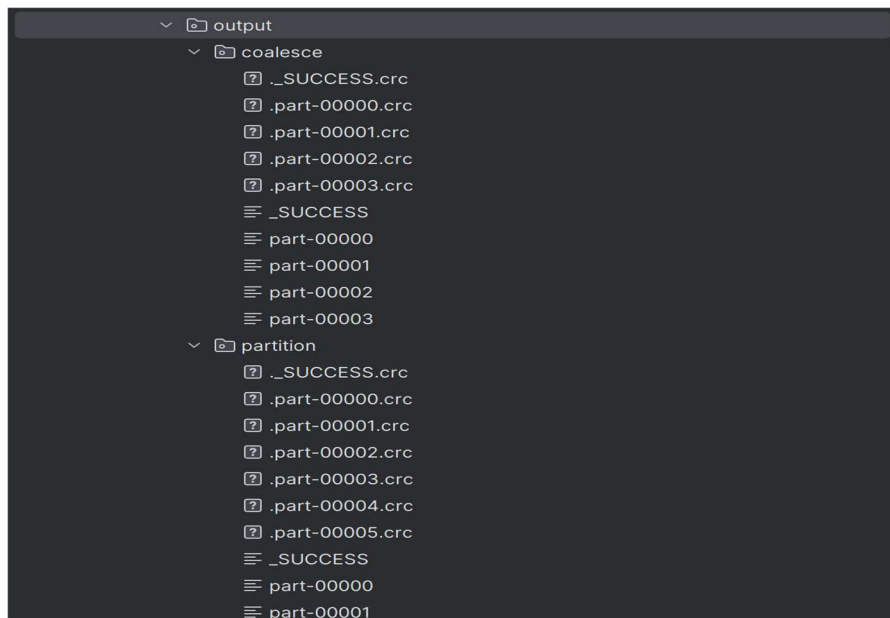
The screenshot shows an IDE with a project named 'Demo'. The file 'RDDRepartition.scala' is open, showing the following code:

```
19 rdd1.partitions.foreach(f=> f.toString)
20 val rddFromFile = spark.sparkContext.textFile(path = "E:/TAI_LIEU_DAI_HOC/SEMESTER_6/BigData/Demo/src/main/scala/org/example/output/partition")
21
22 println("TextFile : "+rddFromFile.partitions.size)
23
24
25
26 rdd1.saveAsTextFile(path = "E:/TAI_LIEU_DAI_HOC/SEMESTER_6/BigData/Demo/src/main/scala/org/example/output/partition")
27 val rdd2 = rdd1.repartition(numPartitions = 4)
28 println("Repartition size : "+rdd2.partitions.size)
29
30
31
32 val rdd3 = rdd1.coalesce(numPartitions = 4)
33 println("Repartition size : "+rdd3.partitions.size)
34
35 rdd3.saveAsTextFile(path = "E:/TAI_LIEU_DAI_HOC/SEMESTER_6/BigData/Demo/src/main/scala/org/example/output/coalesce")
36
```

The Run console shows the following output:

```
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
From local[5] : 5
parallelize : 6
TextFile : 10
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
Repartition size : 4
Repartition size : 4
Process finished with exit code 0
```

Nhìn output ta có thể thấy đối với coalesce sẽ tối ưu hóa và hiệu suất cao hơn và giảm chi phí tính toán, chỉ tạo ra 3 partition



4.5 Demo SparkSQL

Dữ liệu đầu vào là 1 file csv **address.csv** chứa các thông tin về Id, Address Line1, City, State, Zipcode và 1 file json **employee.json** chứa các thông tin name, department, state, salary, age, bonus

```
≡ address.csv × employee.json
1 Id,Address Line1,City,State,Zipcode
2 1,9182 Clear Water Rd,Fayetteville,AR,72704
3 2,9724 E Landon Ln,Kennewick,WA,99338
4 3,9509 Clay Creek Ln,Fort Worth,TX,76177
5 4,98016 S Garnsey St,Santa Ana,CA,92707
6 5,9920 State Highway 89,Kingman,OK,73456
7 6,9234 Lakeview Dr,Springfield,MO,65802
8 7,6541 Pinehurst Ave,Boulder,CO,80301
9 8,7743 Maple St,Albany,NY,12203
10 9,5643 Oakwood Dr,Greenville,SC,29601
11 10,8822 Elm St,Omaha,NE,68124
12 11,3401 Cedar Ave,Fresno,CA,93702
13 12,1101 Magnolia St,Shreveport,LA,71101
14 13,7834 Willow Dr,Richmond,VA,23219
15 14,6312 Birchwood Ave,Nashville,TN,37201
16 15,4523 Pine St,Des Moines,IA,50309
17 16,5422 Chestnut St,Charlotte,NC,28202
18 17,8819 Walnut St,Miami,FL,33125
19 18,7734 Cherry Ave,Detroit,MI,48201
20 19,9301 Oak St,Portland,OR,97201
21 20,4412 Birch St,Seattle,WA,98101
22
```

```
≡ address.csv × employee.json ×
1 [
2   {
3     "name": "James",
4     "department": "Sales",
5     "state": "NY",
6     "salary": 90000,
7     "age": 34,
8     "bonus": 10000
9   },
10  {
11    "name": "Michael",
12    "department": "Sales",
13    "state": "NY",
14    "salary": 86000,
15    "age": 56,
16    "bonus": 20000
17  },
18  {
19    "name": "Robert",
20    "department": "Sales",
21    "state": "CA",
22    "salary": 81000,
23    "age": 30,
24    "bonus": 23000
25  },
26  {
27    "name": "Maria",
28    "department": "Finance",
```

```

package org.example.SparkSQL

import org.apache.log4j.{Level, Logger}
import org.apache.spark.sql.SparkSession
import org.apache.spark.sql.functions.lit
import org.apache.spark.sql.functions.col
import org.apache.spark.sql.functions._

object SelectExample {
  def main(args: Array[String]): Unit = {
    Logger.getLogger("org").setLevel(Level.ERROR);
    val spark = SparkSession.builder().appName("SparkSelectExample").master("local")
      .getOrCreate();

    val df = spark.read.format("csv")
      .option("header", "true")

    .load("E:/TAI_LIEU_DAI_HOC/SEMESTER_6/BigData/Demo/src/main/scala/org/example/csv_files/
address.csv");

    df.printSchema() //kieu du lieu cho tung cot
    df.show() //show de kiem tra du lieu, chi mac dinh 20 hang

    println("=====")
    println("Select function using column name as string and just get 5 rows")
    df.select("Id", "Address Line1", "City").show(5,false)

    println("=====")
    // su dung cau lenh where
    println("where() with SQL Expression")
    df.where("state='WA'").show(false)
    // Multiple condition
    df.where(df("state") === "CA" && df("City") === "Santa Ana")
      .show(false)

    println("=====")
    //Add a New Column to DataFrame
    // lit() function is used to add a constant value to a DataFrame column
    println("withColumn() with SQL")

    val df1=df.withColumn("Country", lit("USA"))
    df1.show(5,false)

    // Change Value of an Existing Column
    val df2=df.withColumn("Zipcode", col("Zipcode") + 20)
    df2.show(5, false)

    // Add, Replace, or Update multiple Columns
    df.createOrReplaceTempView("ADDRESS")
  }
}

```

```

spark.sql("SELECT Zipcode+20 as CopiedColumn, 'USA' as country FROM
ADDRESS").show()

//rename DataFrame column name
val df3 = df.withColumnRenamed("City", "ThanhPho")
               .withColumnRenamed("State", "Bang")
df3.printSchema()

// Drop a Column
val df4=df.drop("Country")
df4.show(5,false)

//Distinct a Column
val distinctValuesDF = df.select("State").distinct()
println("Distinct count: " + distinctValuesDF.count())

println("=====")
println("GROUP BY")
val df_emp =
spark.read.option("multiline","true").json("E:/TAI_LIEU_DAI_HOC/SEMESTER_6/BigData/Demo/src/main/scala/org/example/json_files/employee.json")
df_emp.show(false)

println("group by department")
df_emp.groupBy("department")
      .agg(
        sum("salary").as("sum_salary"),
        avg("salary").as("avg_salary"),
        sum("bonus").as("sum_bonus"),
        max("bonus").as("max_bonus"))
      .where(col("sum_bonus") >= 50000)
      .show(false)

println("Join DF and DF_Emp")
df.join(df_emp, df("State") === df_emp("state"), "inner")
  .show(false)

//"inner" can be swap outer,full,fullouter,left,leftouter,right,rightouter
//Use SQL Express
df.createOrReplaceTempView("ADD")
df_emp.createOrReplaceTempView("EMP")
//SQL JOIN
val joinDF = spark.sql("select * from ADD a, EMP e where a.State == e.state")
joinDF.show(false)

println("Pivot: ")
val pivotDF = df_emp.groupBy("department").pivot("state").sum("salary")
pivotDF.show()

spark.stop()
}
}

```

Mục đích mục đích minh họa một số hoạt động phổ biến khi làm việc với dữ liệu trong Apache Spark SQL. Dưới đây là mục đích và chức năng của từng phần trong đoạn mã:

1. Khởi tạo môi trường Spark:

- Tạo một phiên **SparkSession** để tương tác với Spark.
- Thiết lập mức độ log để chỉ hiển thị các lỗi.

2. Đọc dữ liệu và khám phá dữ liệu:

- Sử dụng phương thức **read** để đọc dữ liệu từ tệp CSV và in schema của DataFrame.
- Sử dụng **show()** để hiển thị một số hàng đầu tiên của DataFrame.

3. Chọn cột và hiển thị DataFrame:

- Sử dụng phương thức **select** để chọn một số cột cụ thể của DataFrame và hiển thị.
- Sử dụng **where** để lọc dữ liệu dựa trên điều kiện SQL và hiển thị kết quả.
- Thực hiện nhiều điều kiện và hiển thị kết quả.

4. Thêm và thay đổi cột:

- Sử dụng **withColumn** để thêm một cột mới với giá trị cố định.
- Sử dụng **withColumn** để thay đổi giá trị của một cột hiện có.
- Sử dụng SQL expression để thêm cột mới và hiển thị kết quả.

5. Đổi tên và loại bỏ cột:

- Sử dụng **withColumnRenamed** để đổi tên của một hoặc nhiều cột.
- Sử dụng **drop** để loại bỏ một hoặc nhiều cột và hiển thị kết quả.

6. Tính toán thống kê và phân tích dữ liệu:

- Sử dụng **distinct** để lấy giá trị duy nhất của một cột và đếm số lượng giá trị đó.
- Sử dụng **groupBy** và các hàm thống kê như **sum**, **avg**, **max** để tính toán thống kê nhóm và lọc kết quả.

7. Kết hợp dữ liệu:

- Sử dụng **join** để kết hợp hai DataFrame dựa trên điều kiện nhất định và hiển

thị kết quả.

- Sử dụng SQL expression để thực hiện kết hợp và hiển thị kết quả.

8. Pivot:

- Sử dụng **pivot** để biến đổi dữ liệu từ dạng dài (long) thành dạng rộng (wide) và hiển thị kết quả.

Kết quả sau khi chạy chương trình

```
SLF4J: See http://www.slf4j.org/codes.html#no_static_mdc_binder for further details.
root
|-- Id: string (nullable = true)
|-- Address Line1: string (nullable = true)
|-- City: string (nullable = true)
|-- State: string (nullable = true)
|-- Zipcode: string (nullable = true)

+-----+-----+-----+
| Id|      Address Line1|      City|State|Zipcode|
+-----+-----+-----+
| 1| 9182 Clear Water Rd|Fayetteville| AR| 72704|
| 2| 9724 E Landon Ln| Kennewick| WA| 99338|
| 3| 9509 Clay Creek Ln| Fort Worth| TX| 76177|
| 4| 98016 S Garnsey St| Santa Ana| CA| 92707|
| 5|9920 State Highwa...| Ringling| OK| 73456|
| 6| 9234 Lakeview Dr| Springfield| MO| 65802|
| 7| 6541 Pinehurst Ave| Boulder| CO| 80301|
| 8| 7743 Maple St| Albany| NY| 12203|
| 9| 5643 Oakwood Dr| Greenville| SC| 29601|
|10| 8822 Elm St| Omaha| NE| 68124|
|11| 3401 Cedar Ave| Fresno| CA| 93702|
|12| 1101 Magnolia St| Shreveport| LA| 71101|
|13| 7834 Willow Dr| Richmond| VA| 23219|
|14| 6312 Birchwood Ave| Nashville| TN| 37201|
|15| 4523 Pine St| Des Moines| IA| 50309|
|16| 5422 Chestnut St| Charlotte| NC| 28202|
|17| 8819 Walnut St| Miami| FL| 33125|
|18| 7734 Cherry Ave| Detroit| MI| 48201|
|19| 9301 Oak St| Portland| OR| 97201|
|20| 4412 Birch St| Seattle| WA| 98101|
+-----+-----+-----+

=====
Select function using column name as string and just get 5 rows
+-----+-----+-----+
| Id|Address Line1|      |City|      |
+-----+-----+-----+
| 1|9182 Clear Water Rd|      |Fayetteville|
| 2|9724 E Landon Ln|      |Kennewick|
| 3|9509 Clay Creek Ln|      |Fort Worth|
| 4|98016 S Garnsey St|      |Santa Ana|
| 5|9920 State Highway 89|      |Ringling|
+-----+-----+-----+
only showing top 5 rows

=====
where() with SQL Expression
+-----+-----+-----+
| Id|Address Line1|      |City|      |State|Zipcode|
+-----+-----+-----+
| 2|9724 E Landon Ln|      |Kennewick|WA|99338|
|20|4412 Birch St|      |Seattle|WA|98101|
+-----+-----+-----+

+-----+-----+-----+
| Id|Address Line1|      |City|      |State|Zipcode|
+-----+-----+-----+
| 4|98016 S Garnsey St|      |Santa Ana|CA|92707|
+-----+-----+-----+
```



```

withColumn() with SQL
=====
|Id|Address Line1|City|State|Zipcode|Country|
=====
|1|9182 Clear Water Rd|Fayetteville|AR|72704|USA|
|2|9724 E Landon Ln|Kennewick|WA|99338|USA|
|3|9509 Clay Creek Ln|Fort Worth|TX|76177|USA|
|4|98016 S Garnsey St|Santa Ana|CA|92707|USA|
|5|9920 State Highway 89|Ringling|OK|73456|USA|
=====
only showing top 5 rows

|Id|Address Line1|City|State|Zipcode|
=====
|1|9182 Clear Water Rd|Fayetteville|AR|72724.0|
|2|9724 E Landon Ln|Kennewick|WA|99358.0|
|3|9509 Clay Creek Ln|Fort Worth|TX|76197.0|
|4|98016 S Garnsey St|Santa Ana|CA|92727.0|
|5|9920 State Highway 89|Ringling|OK|73476.0|
=====
only showing top 5 rows

|CopiedColumn|country|
=====
|72724.0|USA|
|99358.0|USA|
|76197.0|USA|
|92727.0|USA|

```

```

|29621.0|USA|
|68144.0|USA|
|93722.0|USA|
|71121.0|USA|
|23239.0|USA|
|37221.0|USA|
|50329.0|USA|
|28222.0|USA|
|33145.0|USA|
|48221.0|USA|
|97221.0|USA|
|98121.0|USA|
=====
root
|-- Id: string (nullable = true)
|-- Address Line1: string (nullable = true)
|-- ThanhPho: string (nullable = true)
|-- Bang: string (nullable = true)
|-- Zipcode: string (nullable = true)

|Id|Address Line1|City|State|Zipcode|
=====
|1|9182 Clear Water Rd|Fayetteville|AR|72704|
|2|9724 E Landon Ln|Kennewick|WA|99338|
|3|9509 Clay Creek Ln|Fort Worth|TX|76177|
|4|98016 S Garnsey St|Santa Ana|CA|92707|
|5|9920 State Highway 89|Ringling|OK|73456|
=====
only showing top 5 rows

```

```

Distinct count: 18
=====
GROUP BY
=====
|age|bonus|department|name|salary|state|
=====
|34|10000|Sales|James|90000|NY|
|56|20000|Sales|Michael|86000|NY|
|30|23000|Sales|Robert|81000|CA|
|24|23000|Finance|Maria|90000|CA|
|40|24000|Finance|Raman|99000|CA|
|36|19000|Finance|Scott|83000|NY|
|53|15000|Finance|Jen|79000|NY|
|25|18000|Marketing|Jeff|80000|CA|
|50|21000|Marketing|Kumar|91000|NY|
=====

group by department
=====
|department|sum_salary|avg_salary|sum_bonus|max_bonus|
=====
|Sales|257000|85666.66666666667|53000|23000|
|Finance|351000|87750.0|81000|24000|
=====

Join DF and DF_Emp
=====
|Id|Address Line1|City|State|Zipcode|age|bonus|department|name|salary|state|
=====
|8|7743 Maple St|Albany|NY|12203|34|10000|Sales|James|90000|NY|
|8|7743 Maple St|Albany|NY|12203|56|20000|Sales|Michael|86000|NY|

```

```
Distinct count: 18
=====
GROUP BY
+---+---+---+---+---+---+---+---+---+
|age|bonus|department|name  |salary|state|
+---+---+---+---+---+---+---+---+---+
|34 |10000|Sales    |James |90000 |NY  |
|56 |20000|Sales    |Michael|86000 |NY  |
|30 |23000|Sales    |Robert |81000 |CA  |
|24 |23000|Finance  |Maria  |90000 |CA  |
|40 |24000|Finance  |Raman  |99000 |CA  |
|36 |19000|Finance  |Scott  |83000 |NY  |
|53 |15000|Finance  |Jen    |79000 |NY  |
|25 |18000|Marketing|Jeff   |80000 |CA  |
|50 |21000|Marketing|Kumar  |91000 |NY  |
+---+---+---+---+---+---+---+---+---+

group by department
+---+---+---+---+---+---+---+---+---+
|department|sum_salary|avg_salary  |sum_bonus|max_bonus|
+---+---+---+---+---+---+---+---+---+
|Sales     |257000    |85666.66666666667|53000    |23000    |
|Finance   |351000    |87750.0        |81000    |24000    |
+---+---+---+---+---+---+---+---+---+

Join DF and DF_Emp
+---+---+---+---+---+---+---+---+---+
|Id |Address Line1 |City  |State|Zipcode|age|bonus|department|name  |salary|state|
+---+---+---+---+---+---+---+---+---+
|8  |7743 Maple St |Albany|NY  |12203 |34 |10000|Sales    |James |90000 |NY  |
|8  |7743 Maple St |Albany|NY  |12203 |56 |20000|Sales    |Michael|86000 |NY  |
```

Demo > src > main > scala > org > example > json_files > employee.json

75.1 CRLF UTF-8 2 spaces No JSON schema

4.6 Demo Spark Streaming

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import *

# Create the SparkSession
spark = SparkSession.\
builder.\
appName("sparkstream").\
getOrCreate()

# Define input sources
lines = (spark.\
.readStream.format("socket").\
#option("host", "192.168.1.133").\
.option("host", "localhost").\
.option("port", 9999).\
.load())

# Transform data
words = lines.select(explode(split(col("value"), " ")).alias("word"))

# Get the count of published words
counts = words.groupBy("word").count()

# Define the checkpoint directory
checkpointDir = "C:/Spark/Spark_Python/Output/Streaming"

# Start streaming defining the necessary configurations
streamingQuery = (counts
.writeStream
.format("console")
.outputMode("complete")
.trigger(processingTime="1 second")
.option("checkpointLocation", checkpointDir)
.start())
streamingQuery.awaitTermination()
```

Input:

Nhập từng từ

```
C:\Users\LG>ncat -l 9999
hello
hadoop
spark
word
hello
```

```
C:\Users\LG>ncat -l 9999
hello
hadoop
spark
word
hello
haha
```

Bước chạy:

Mở cổng trước khi chạy chương trình

```
Microsoft Windows [Version 10.0.22631.3447]
(c) Microsoft Corporation. All rights reserved.

C:\Users\LG>ncat -l 9999
```

Output:

```
& C:/Users/LG/AppData/Local/Programs/Python/Python311/python.exe c:/Spark/Streaming_Python/Spark_Streaming.py
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
24/05/07 15:52:44 WARN TextSocketSourceProvider: The socket source should not be used for production applications! It does not support recovery.
24/05/07 15:52:48 WARN ResolveWriteToStream: spark.sql.adaptive.enabled is not supported in streaming DataFrames/Datasets and will be disabled.
-----
Batch: 0
-----
+----+-----+
|word|count|
+----+-----+
+----+-----+

24/05/07 15:53:45 WARN ProcessingTimeExecutor: Current batch is falling behind. The trigger interval is 1000 milliseconds, but spent 55027 milliseconds
[Stage 3:=====> (76 + 8) / 200]
```

```
24/05/07 15:53:45 WARN ProcessingTimeExecutor: Current batch is falling behind. The trigger interval is 1000 milliseconds, but spent 55027 milliseconds
-----
Batch: 1
-----
+----+-----+
| word|count|
+----+-----+
| hello| 2|
| spark| 1|
| word| 1|
| | 1|
|hadoop| 1|
+----+-----+
```

```
Batch: 2
-----
+----+-----+
| word|count|
+----+-----+
| hello| 2|
| spark| 1|
| haha| 1|
| word| 1|
| | 1|
|hadoop| 1|
+----+-----+

24/05/07 15:59:23 WARN ProcessingTimeExecutor: Current batch is falling behind. The trigger interval is 1000 milliseconds, but spent 36799 milliseconds
[]
```

PHẦN KẾT LUẬN

1. Ưu điểm

Nắm được các kiến thức cũng như những vấn đề liên quan về Apache Spark. Áp dụng kiến thức để cài đặt Spark, cấu hình Spark Cluster. Thiết kế và chạy demo các chương trình MapReduce trên Spark. Ngoài ra nhóm còn tìm hiểu thêm các ứng dụng khác của Spark như: RDD, SparkSQL, Spark Streaming,...

2. Nhược điểm

Do thời gian tìm hiểu giới hạn, nhóm chưa tìm hiểu sâu vào các khía cạnh khác của Apache Spark như MLlib, Graphx. Chưa kết hợp được với các Cơ sở dữ liệu như Hbase, Hive,...

3. Hướng phát triển

Từ những nghiên cứu ban đầu về Apache Spark, nhóm sẽ phát triển thêm các đề tài kết hợp Database. Tìm hiểu thêm về các tính năng MLlib và Graphx của Spark.

TÀI LIỆU THAM KHẢO

1. Matei Zaharia, Mosharaf Chowdhury, Michael J. Franklin, Scott Shenker, Ion Stoica
“Spark: Cluster Computing with Working Sets”, 2010
[Zaharia.pdf \(usenix.org\)](#)
2. Grzegorz Piwowarek, *“Introduction to Apache Spark”*, Baeldung, 2024.
[Introduction to Apache Spark | Baeldung](#)
3. SharkByExamples, *“Apache Spark 3.5 Tutorial with Examples”*, [Apache Spark Tutorial with Examples - Spark By {Examples} \(sparkbyexamples.com\)](#)
4. Databricks, *“What is Spark SQL”*, [What is Spark SQL? \(databricks.com\)](#)